



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



TOPOLOGIES CHARACTERIZATION AND EVOLUTION OF MULTI-AGENT SYSTEMS FOR AUTOMATED PROGRAM REPAIR

XIANG FENG YE PAN

Thesis supervisor

MATÍAS SEBASTIÁN MARTÍNEZ MARTÍNEZ (Department of Service and Information System Engineering)

Thesis co-supervisor

VINCENZO DE MARTINO (Department of Service and Information System Engineering)

Degree

Bachelor's Degree in Informatics Engineering (Computing)

Bachelor's thesis

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

This thesis is submitted to the Computer Science Department, Universitat Politècnica de Catalunya in fulfilment of the requirements for the degree in Informatics Engineering with a mention in Computing.

Xiang Feng Ye Pan
June 21, 2026

Copyright © 2026
Xiang Feng Ye Pan

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Matías Sebastián Martínez, and my co-supervisor, Vincenzo De Martino, for their invaluable guidance, support, and encouragement throughout this project.

I am also grateful to my GEP tutor, Belles Ros Francesc Xavier, for his insightful advice and assistance in project management, which greatly contributed to the organization and success of this work.

I would like to acknowledge the *SWE-bench* leaderboard and the open-source repositories that formed the basis of my analysis, which were essential to the progress of this work.

I also acknowledge the use of generative AI tools as assistants throughout the development of this thesis. The specific tools, the way each was used, and its contribution are declared in Table [B.2](#).

Finally, I would like to thank my colleagues, friends, and family, whose encouragement and understanding made this project possible.

Abstract

Resum

Els models de llenguatge grans (LLM) han avançat ràpidament en tasques relacionades amb el codi, però un únic model encara té dificultats amb la reparació de programes en incidències de programari reals. *SWE-bench*, un *benchmark* construït a partir d'incidències reals de GitHub, s'ha convertit en la referència per mesurar aquesta capacitat, i el seu rànquing està dominat per sistemes multi-agent (MAS) que coordinen diversos agents especialitzats. La manera com aquests agents es connecten, la seva *topologia*, varia molt entre propostes i poques vegades es caracteritza de manera sistemàtica.

Aquest treball persegueix dos objectius. Primer, analitza empíricament els MAS de codi obert presentats al rànquing de *SWE-bench*, i n'identifica els rols dels agents, els nivells d'autonomia i les topologies de comunicació. Segon, dissenya i implementa *EvoMas*, un marc multi-agent evolutiu per a la reparació automàtica de programes (APR) els agents i topologies del qual es poden afegir, substituir o reconfigurar sense redissenyar el sistema global.

L'estudi empíric es va dur a terme per extreure idees d'implementació que es transposen al disseny del marc. *EvoMas* es va construir i avaluar sobre l'entorn de *SWE-bench* amb models petits allotjats localment mitjançant Ollama. No obstant això, el seu disseny modular també permet operar més enllà de l'àmbit de l'APR, com demostren tasques alienes a la reparació com ara la traducció i la cerca web.

Els resultats experimentals indiquen que aquests models poden resoldre incidències reals, encara que amb limitacions clares, i que la topologia d'un sistema multi-agent té un impacte substancial en el resultat. *EvoMas* proporciona una plataforma oberta i reconfigurable sobre la qual es poden estudiar i reproduir aquests efectes de la topologia.

Resumen

Los modelos de lenguaje grandes (LLM) han avanzado rápidamente en tareas relacionadas con el código, pero un único modelo todavía tiene dificultades con la reparación de programas en incidencias de software reales. *SWE-bench*, un *benchmark* construido a partir de incidencias reales de GitHub, se ha convertido en la referencia para medir esta capacidad, y su clasificación está dominada por sistemas multiagente (MAS) que coordinan varios agentes especializados. El modo en que estos agentes se conectan, su *topología*, varía mucho entre propuestas y rara vez se caracteriza de forma sistemática.

Este trabajo persigue dos objetivos. Primero, analiza empíricamente los MAS de código abierto presentados a la clasificación de *SWE-bench*, identificando los roles de los agentes, los niveles de autonomía y las topologías de comunicación. Segundo, diseña e implementa *EvoMas*, un marco multiagente evolutivo para la reparación automática de programas (APR) cuyos agentes y topologías pueden añadirse, sustituirse o reconfigurarse sin rediseñar el sistema global.

El estudio empírico se llevó a cabo para extraer ideas de implementación que se transponen al diseño del marco. *EvoMas* se construyó y evaluó sobre el entorno de *SWE-bench* con modelos pequeños alojados localmente mediante Ollama. No obstante, su diseño modular también permite operar más allá del ámbito de la APR, como demuestran tareas ajenas a la reparación como la traducción y la búsqueda web.

Los resultados experimentales indican que estos modelos pueden resolver incidencias reales, aunque con limitaciones claras, y que la topología de un sistema multiagente tiene un impacto sustancial en el resultado. *EvoMas* proporciona una plataforma abierta y reconfigurable sobre la que se pueden estudiar y reproducir dichos efectos de la topología.

Abstract

Large Language Models (Large Language Model (LLM)) have advanced rapidly on code-related tasks, yet a single model still struggles with program repair on real-world software issues. *SWE-bench*, a benchmark built from real GitHub issues, has become the reference for measuring this ability, and its leaderboard is dominated by multi-agent systems (Multi-Agent System (MAS)) that coordinate several specialized agents. How these agents are connected, their *topology*, varies widely across submissions and is rarely characterized systematically.

This thesis pursues two objectives. First, it empirically analyzes open-source MAS submitted to the *SWE-bench* leaderboard, identifying their agent roles, levels of autonomy, and communication topologies. Second, it designs and implements *EvoMas*, an evolutionary multi-agent framework for Automated Program Repair (Automated Program Repair (APR)) whose agents and topologies can be added, replaced, or reconfigured without redesigning the overall system.

The empirical study was conducted to extract implementation ideas that are transposed to the framework design. *EvoMas* was built and evaluated on the *SWE-bench* environment using small, locally hosted models served through Ollama. However, its modular design also allows it to operate beyond the APR scope, as demonstrated by non-repair tasks such as translation and web search.

The experimental results indicate that these models can solve real-world issues, although with clear limitations, and that the topology of a multi-agent system has a substantial impact on the outcome. *EvoMas* provides an open, reconfigurable platform on which such topology effects can be studied and reproduced.

Contents

1	Introduction, Motivation, and Objectives	1
1.1	Context and Introduction	1
1.1.1	The Landscape of AI-Assisted Software Engineering	1
1.1.2	Foundational Concepts	2
1.1.3	Identifying the Problem	2
1.1.4	Stakeholders	4
1.2	Motivation and Problem Statement	4
1.2.1	Limitations of Existing Approaches	5
1.2.2	Value of this Work	5
1.3	Scope	6
1.3.1	Objectives and Sub-objectives	6
1.3.2	Potential Obstacles and Risks	6
1.3.3	Temporal Planning	7
2	State of the Art	9
2.1	Large Language Models	9
2.1.1	Hyperparameters	10
2.1.2	Prompt Engineering	11
2.1.3	Local versus Cloud Inference	13
2.1.4	LLMOps	13
2.1.5	LLMs for Code	15
2.2	Multi-Agent Systems (MAS)	15
2.2.1	Topology Structures	16
2.2.1.1	Centralized (Star)	16
2.2.1.2	Decentralized and Distributed	17
2.2.1.3	Hierarchical (Tree)	19
2.2.2	Advanced Coordination and Orchestration Architecture	19
2.2.2.1	Hybrid Topologies	20

2.2.2.2	Dynamic Topologies	20
2.3	Automated Program Repair (APR)	22
2.4	Frameworks for LLM Applications	22
2.5	<i>SWE-bench</i> Leaderboard Problems Subset and Evaluation Metrics . .	24
2.5.1	Verified	24
2.5.2	Lite	25
2.5.3	Full	25
2.6	Similar Products and Empirical Studies	25
2.6.1	Similar Existing Products: Agent Workflow Builder	26
2.6.2	Similar Empirical Studies: Topology and Agent Organization	26
3	Empirical Study of Architecture and Topologies of MAS for APR	29
3.1	Research Questions	29
3.2	Methodology	31
3.2.1	Repository Selection	33
3.2.2	Framework Methodology	34
3.2.3	Agent Methodology	34
3.2.4	Prompt Methodology	35
3.2.5	Agent Role Methodology	36
3.2.6	Autonomy Methodology	37
3.2.7	Topology Methodology	38
3.2.8	Graph Visualization Methodology	39
3.3	Analysis Results	39
3.3.1	Results of the Repository Selection	40
3.3.2	RQ1: Results of the Framework Analysis	42
3.3.3	Results of the Agent Analysis	45
3.3.4	RQ2: Results of the Prompt Analysis	46
3.3.5	RQ3: Results of the Agent Roles Analysis	52
3.3.6	RQ4: Results of the Autonomy Analysis	56
3.3.7	RQ5: Results of the Topology Analysis	58
3.3.8	Results of the Graph Visualization	59
4	Specification and Design of the Solution	64
4.1	Methodology	64
4.1.1	Development Process	64
4.1.2	Monitoring Tools	64
4.2	System Requirements	65

4.2.1	Functional Requirements	65
4.2.2	Non-Functional Requirements	66
4.3	System Specification	68
4.3.1	Use Case Diagram	68
4.3.2	Conceptual Schema	68
4.3.3	Data Persistence	68
4.3.4	Taxonomy of MAS Designs and Analysis Dimensions	69
4.4	System Design	69
4.4.1	Backend	69
4.4.2	Frontend	69
4.4.3	Behavioral Views	70
4.5	Architectural Design	70
4.5.1	Design Ideas Adopted from the Empirical Analysis	70
4.5.2	Analysis of Alternatives	71
4.5.3	Physical Architecture	73
5	Development of the Proposal	74
5.1	Environment Setup	74
5.2	Testing	76
5.3	Deployment	78
6	Experimentation and Evaluation	79
6.1	Evaluation Metrics	79
6.2	Experimental Setup	79
6.2.1	Instances	79
6.2.2	Topology	80
6.2.3	Language Models	81
6.2.4	Agent Configuration	82
6.2.5	Evaluation Scripts	82
6.2.6	Hardware	82
6.2.7	Reporting Pipeline	82
6.3	Results Analysis	83
6.3.1	Different LLMs on the Chain Topology	83
6.3.2	Temperature and Seed Sensitivity	86
6.3.3	Topology Comparison at qwen3.5:9b	89
6.3.4	Real-World Benchmark	91
6.3.5	Token Efficiency Across Resolved Instances	95

6.4	Framework Extensibility Beyond APR	98
6.4.1	Translation Demo	98
6.4.2	Web-Research Demo	98
6.5	Limitations	99
6.6	Discussion and Implications	99
7	Sustainability and Ethical Implications	101
7.1	Self-Assessment	101
7.2	Economic Impact	102
7.3	Environmental Impact	103
7.4	Social Impact and Ethics	105
7.5	Sustainability Matrix	106
7.6	Identification of Laws and Regulations	106
8	Conclusions	109
8.1	Research Outcomes and Contributions	109
8.2	Degree of Achievement of the Objectives	110
8.3	Future Work	111
8.4	Personal Reflection	112
A	Glossary	114
B	Supplementary Figures and Tables	116
C	UML Diagrams of the <i>EvoMas</i> Framework	121
D	Topology Graphs of the Experiment Notebooks	137
	Bibliography	144

List of Figures

1.1	Part I deliverables map to Part II framework components	3
2.1	LLMOps lifecycle	14
2.2	Centralized (star) topology	16
2.3	Decentralized topology	17
2.4	Distributed topologies (mesh, ring, chain)	18
2.5	Hierarchical (tree) topology	19
2.6	Hybrid topology	20
2.7	Dynamic topology	21
3.1	Compact overview of the empirical study research process	32
3.2	Reduction of the repository sample across the four analyses	42
3.3	Consolidated graph (radial layout)	60
3.4	HyperAgent subgraph	61
3.5	Prometheus subgraph	62
3.6	trae-agent subgraph	63
4.1	Deployment diagram of the <i>EvoMas</i> framework	73
5.1	Technology stack of the <i>EvoMas</i> framework	74
5.2	Backend unit suite captured output	77
5.3	Frontend unit suite captured output	77
5.4	CI/CD pipeline of the <i>EvoMas</i> framework	78
6.1	Reporting pipeline	84
6.2	Chain-topology LLM-comparison scatter plot	85
6.3	Side-by-side patches for the <i>medium</i> instance	85
6.4	Temperature consistency across repetitions	88
6.5	Temperature-variance comparison	88
6.6	Topology ranking at qwen3.5:9b	90

6.7	Topology ranking on <i>SWE-bench</i> Lite	92
6.8	Per-instance duration: chain vs hyperagent_star on <i>SWE-bench</i> Lite	93
6.9	Execution trace: chain on sqlfluff-1625	93
6.10	Execution trace: hyperagent_star on sqlfluff-1625	94
6.11	Total tokens vs cell wall-clock	96
6.12	Total tokens vs total LLM calls	97
6.13	Input vs output token share per prediction	97
6.14	Websearch-demo output: answer-py.md	99
B.1	Project Gantt chart	117
B.2	Taxonomy of MAS analysis dimensions	118
B.3	Empirical study process overview	119
B.4	Translation-demo diff: intro.md, English to Spanish	120
C.1	<i>EvoMas</i> use case diagram	122
C.2	Conceptual data model of the <i>EvoMas</i> framework	123
C.3	<i>EvoMas</i> backend, level-1 architecture	124
C.4	<i>EvoMas</i> backend, agent hierarchy	125
C.5	<i>EvoMas</i> backend, API server	126
C.6	<i>EvoMas</i> backend, MCP server and tools	127
C.7	<i>EvoMas</i> frontend, level-1 architecture	128
C.8	<i>EvoMas</i> frontend, Topology page	129
C.9	<i>EvoMas</i> frontend, Inference page	130
C.10	<i>EvoMas</i> frontend, Evaluation page	131
C.11	<i>EvoMas</i> frontend, Results page	132
C.12	<i>EvoMas</i> sequence diagram, Run Inference	133
C.13	<i>EvoMas</i> sequence diagram, Run Evaluation	134
C.14	<i>EvoMas</i> activity diagram, backend repair pipeline	135
C.15	<i>EvoMas</i> activity diagram, evaluation pipeline	136
D.1	chain topology graph	140
D.2	agentscope_hybrid topology graph	140
D.3	experepair_star topology graph	141
D.4	hyperagent_star topology graph	141
D.5	joycode_star topology graph	141
D.6	lingxi_star topology graph	142

D.7	openhands_star topology graph	142
D.8	prometheus_tree topology graph	143
D.9	translate topology graph	143
D.10	websearch topology graph	143

List of Tables

2.1	Ollama inference hyperparameters	10
2.2	Visual LLM-application platform comparison	27
3.1	<i>SWE-bench</i> repository candidate pool	41
3.2	Framework adoption and infrastructure	44
3.3	Agent count per analyzed repository	45
3.4	Uppercase-emphasis pattern by repository	47
3.5	Directive-words pattern by repository	47
3.6	Punctuation pattern by repository	48
3.7	Markup-tags pattern by repository	49
3.8	Role-definition pattern by repository	49
3.9	Negation pattern by repository	50
3.10	Output-format pattern by repository	51
3.11	Numbered-list pattern by repository	51
3.12	Prompt-pattern frequency	52
3.13	Agent-role distribution	53
3.14	Autonomy-level distribution	57
3.15	Topology classification	58
4.1	Design ideas adopted from the empirical analysis	70
6.1	Topologies exercised in the experiments	81
6.2	Language models used	81
6.3	Agent-configuration parameters	82
6.4	Evaluation scripts per instance set	83
6.5	Hardware configuration	83
6.6	Configuration: LLM comparison	84
6.7	Chain-topology LLM-comparison results	84
6.8	Configuration: temperature and seed sensitivity	87

6.9	Temperature and seed sensitivity results	87
6.10	Configuration: topology comparison	89
6.11	Topology comparison at <code>qwen3.5:9b</code>	90
6.12	Configuration: topology comparison on <i>SWE-bench</i> Lite	92
6.13	Topology comparison on <i>SWE-bench</i> Lite	92
6.14	Configuration: token efficiency aggregation	95
7.1	Detailed project budget	102
7.2	UPC sustainability matrix	107
8.1	Sub-objective achievement status	110
8.2	Requirements satisfaction status	111
B.1	Project task breakdown	116
B.2	Declaration of AI tools used	117
D.1	Dataset: all 107 instances used in the experiments	137

Chapter 1

Introduction, Motivation, and Objectives

*"If debugging is the process of removing software bugs,
then programming must be the process of putting them in."*

- Edsger W. Dijkstra

1.1 Context and Introduction

1.1.1 The Landscape of AI-Assisted Software Engineering

Artificial intelligence (AI) has reshaped the daily practice of software development. Large Language Models (LLMs) have lowered the barrier to generating functional code from natural-language descriptions, demonstrating the viability of automating code synthesis at scale [Chen et al. 2021]. For tasks with well-defined scope and recurring patterns drawn from publicly available code, LLMs perform reliably. Yet the complexity of real-world software systems exposes a fundamental ceiling. Coordinating changes across multiple modules, reasoning about long execution traces, and ensuring that a patch does not introduce regressions are challenges that a single model prompt cannot adequately address [Jimenez et al. 2024].

It is in this setting that Automated Program Repair (APR) becomes essential. Rather than asking a developer to manually trace, localize, and fix a bug, APR techniques aim to automate the full repair pipeline of fault localization, patch generation, and patch validation [Xia et al. 2024].

This Bachelor's thesis is carried out at the Barcelona School of Informatics (Facultat d'Informàtica de Barcelona) of the Universitat Politècnica de Catalunya, un-

der the supervision of Matías Sebastián Martínez and Vincenzo De Martino. It investigates the intersection of two emerging paradigms: **MAS** and **APR**. Specifically, it analyzes the MAS architectures deployed in state-of-the-art APR solutions and characterizes their topologies. Building on that characterization, it constructs an evolutionary MAS framework capable of adapting its agent topology to different repair scenarios.

1.1.2 Foundational Concepts

The technical discussion that follows relies on a set of domain-specific terms used consistently throughout the thesis; formal definitions of all terms are collected alphabetically in Appendix [A](#).

1.1.3 Identifying the Problem

LLMs have outpaced existing methods for evaluating code-related tasks, yet understanding their practical limits remains essential for guiding the design of more capable automated repair systems. The *SWE-bench* benchmark [\[Jimenez et al. 2024\]](#) is an evaluation framework consisting of 2,294 software engineering problems derived from real GitHub issues and pull requests across popular Python repositories. Given a codebase snapshot and a natural-language issue description, a system must modify the source code to resolve the issue. The difficulty arises from the need to coordinate changes across multiple functions, classes, and files, to interpret long execution contexts, and to perform multi-step reasoning well beyond standard code completion.

According to its authors, “our evaluations show that both state-of-the-art proprietary models and our fine-tuned model SWE-Llama can resolve only the simplest issues. The best-performing model, Claude 2, is able to solve a mere 1.96% of the issues” [\[Jimenez et al. 2024\]](#). This result motivates the use of MAS, which can distribute reasoning tasks across specialized agents to deal with the complexity that a single model cannot handle.

The *SWE-bench* leaderboard [\[SWE-bench Team 2024a\]](#) publishes submissions from the research community and industry, organized into five categories: Verified [\[Chowdhury et al. 2024\]](#), Lite, Full, Bash Only, and Multimodal. Inspecting these open-source submissions reveals substantial diversity in MAS architecture and topology. For instance, Trae Agent by ByteDance ¹, a single-agent system,

¹[bytedance/trae-agent](https://github.com/bytedance/trae-agent)

achieves 78.80% on the Verified category [SWE-bench Team 2024a], while Agentless by OpenAutoCoder ², a non-agent pipeline, attains 40.67% on the Lite category. This spread illustrates that neither agent count nor framework choice alone determines performance. The agent *topology*, a critical yet largely unexplored determinant of APR outcomes, is the focus of this thesis; recent evidence confirms that task performance varies measurably across topological structures for LLM-based MAS [Yang et al. 2025].

To investigate this systematically, the work is divided into two complementary parts. Before designing an evolutionary topology framework, the project first needs a grounded characterization of how state-of-the-art APR systems are actually constructed. Each of these characterizations becomes a concrete component of the framework built in Part II, as summarized in Figure 1.1.

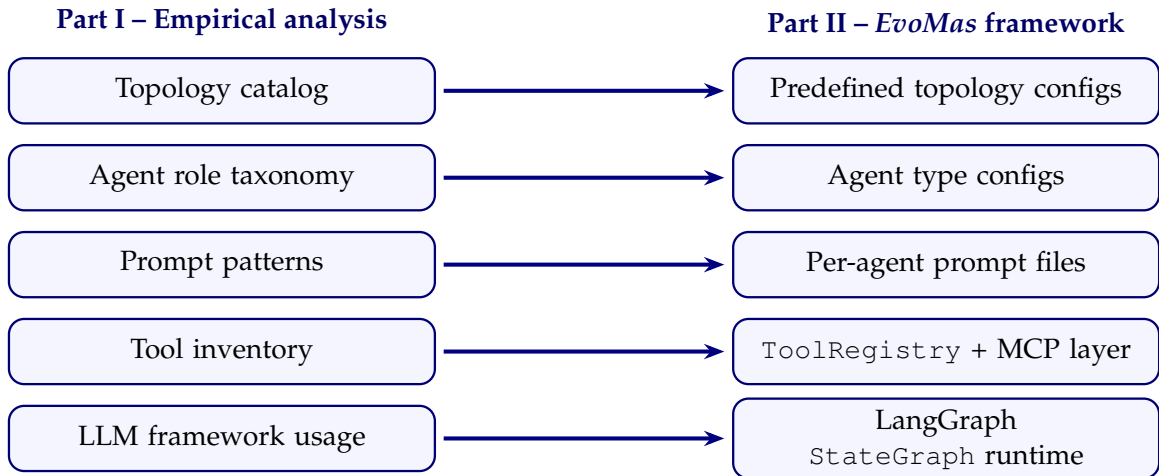


Figure 1.1: Each characterization produced by the empirical analysis of Part I (left) becomes a concrete configuration surface or code component of the evolutionary framework built in Part II (right).

Part I - Empirical Analysis

The first part is an empirical study of open-source repositories submitted to the *SWE-bench* leaderboard. The analysis focuses on the Verified, Lite, and Full categories, examining 48 repositories. For each repository, the characterization covers the topology type (centralized, decentralized and distributed, hierarchical, hybrid, or dynamic), the level of agent autonomy, the roles of individual agents, the

²OpenAutoCoder/Agentless

LLM frameworks employed, and the structure of prompts used to call the models. The full empirical study is reported in Chapter 3.

Part II - Framework Development

The second part builds on the empirical findings to design and implement *Evo-Mas*, an evolutionary MAS framework for APR using LangChain and LangGraph. Unlike existing solutions, which fix a single topology, this framework is designed to support multiple topologies within the same system. Agents can be added, replaced, or reconfigured between runs by editing the configuration, allowing different coordination strategies to be compared under controlled conditions. Full runtime topology evolution is identified as future work (see Section 8.3). The framework is specified in Chapter 4, implemented in Chapter 5, and evaluated in Chapter 6.

1.1.4 Stakeholders

The primary beneficiaries of this work are software development teams and DevOps organizations responsible for continuous integration and delivery pipelines, where automated defect resolution can reduce cycle times and lower maintenance costs. Researchers in APR and MAS gain a publicly available framework and a characterization study that establishes a comparative baseline for future work. AI and machine learning engineers building LLM-based agent systems can configure local Ollama models, adjust hyperparameters, and tailor prompt and tool designs. In addition, the prompts and tools extracted in Part I from the analyzed leaderboard repositories can be reused or adapted directly as agent templates. End users of software products benefit indirectly, as more reliable automated repair translates to fewer latent defects in the final product.

1.2 Motivation and Problem Statement

The breadth of software engineering tasks to which LLMs have been applied is documented in a systematic literature review of 395 studies spanning 2017 to 2024 [Hou et al. 2024]. That review identifies code generation, fault localization, and program repair as recurring application areas. APR has emerged as a particularly promising approach to reduce the cost and time associated with software maintenance by automating the detection, localization, and correction of

program faults. However, many existing APR solutions are implemented as rigid systems with fixed workflows, limiting their flexibility to adapt to the diversity of real-world defects and codebases.

A second motivating concern, complementary to flexibility, is the **efficiency challenge** identified by the TopoAgent study [Yang et al. 2025]: naïve scaling of agent counts without adaptive structural design results in redundant inter-agent communication and inflated token and hardware costs.

1.2.1 Limitations of Existing Approaches

OpenHands [OpenHands 2024] is a representative open-source platform for AI-driven software development. It exposes a set of Python and REST APIs for constructing agents that interact with code, supporting multiple execution modalities including a command-line interface, a local graphical user interface, and cloud-based deployment³. OpenHands is competitive across a broad range of benchmarks, including *SWE-bench*. Nevertheless, the platform enforces predefined agent workflows that are difficult to reconfigure at runtime, constraining experimentation with alternative coordination topologies.

More broadly, inspection of the *SWE-bench* leaderboard reveals that many submitted systems adopt a fixed or implicit agent topology, where the structure of agent interactions is embedded in the implementation and cannot be changed without significant refactoring.

1.2.2 Value of this Work

The value of *EvoMas* lies in its architectural flexibility (see Section 1.1.3). Unlike existing MAS implementations for APR, it does not commit to a fixed topology. As a consequence, different APR strategies can be explored and compared by reconfiguring agent communication patterns, without modifying the underlying system architecture. This design supports experimentation, scalability, and reusability, making the framework particularly suited to research-oriented APR evaluation on standardized benchmarks.

³All-Hands-AI/OpenHands

1.3 Scope

1.3.1 Objectives and Sub-objectives

The primary objective of this thesis is to analyze MAS architectures and topologies in the context of APR and to design an evolutionary MAS framework capable of adapting its agent topology and composition to different APR scenarios. To achieve this primary objective, the following sub-objectives are defined.

Empirical Analysis Sub-objectives

1. Analyze agent-based APR systems present in selected *SWE-bench* leaderboard repositories.
2. Identify and classify the agents involved in these systems, including their roles and responsibilities.
3. Evaluate the level of autonomy and coordination within the analyzed agent systems.
4. Discover topologies resulting from agent interactions and compare them to identify the advantages and disadvantages of each topology.

Framework Development Sub-objectives

1. Based on the empirical analysis, design a MAS framework that supports topological evolution and flexible agent orchestration.
2. Ensure the framework can solve the problem of APR.
3. Test and compare different topologies and agent configurations on the *SWE-bench* harness. Leaderboard submission with a frontier-class LLM is left to future work, since the local hardware used in this thesis is limited to small open-weight models.

1.3.2 Potential Obstacles and Risks

Any project can encounter difficulties, and a Bachelor's thesis is especially constrained by limited time and the need to acquire expertise in MAS frameworks, LLM tooling, and empirical software engineering concurrently. The following risks have been identified with their estimated impact and proposed alternatives.

- **Inexperience with MAS Frameworks (High).** Difficulty in understanding complex open-source repositories. Alternative: consult with experienced supervisors in the first in-person meeting and ask an LLM for help with specific complex concepts.
- **Computational Limitations (Medium).** Experiments may take too long to execute using a local LLM from Ollama. Alternative: optimize code, use a smaller test system initially, or use cloud-based high-performance computing resources. Additional resources: access to HPC clusters or cloud computing credits.
- **No Variability in Analyzed Repositories (Medium).** Limited diversity in repositories may reduce the effectiveness of APR experiments. Alternative: select repositories that exhibit architectural diversity. Additional resources: another source of repositories besides the *SWE-bench* leaderboard.
- **Difficulty Integrating Multiple Topologies (High).** Integrating agents with different topologies may be complex and could lead to incompatibilities. Alternative: incremental development, starting with a single topology and integrating more later.
- **Results below the State of the Art (Medium).** This project relies on open-source repositories rather than frontier industrial ones. While this ensures transparency and reproducibility, it may limit performance compared to state-of-the-art approaches. Alternative: cross-validate results and run multiple repair strategies. Additional resources: specific synthetic issues.

1.3.3 Temporal Planning

The project spans approximately 540 hours distributed across 19 weeks (10 February - 23 June 2026), organized into project management (PM), repository analysis (AR), framework construction (FC), tests and experiments (TE), and final documentation (FD). Table B.1 lists each sub-task with its estimated effort and scheduled period; Figure B.1 presents the full schedule as a Gantt chart.

Deviations from the GEP planning

Two changes with respect to the GEP final document are documented below. First, the hardware setup was extended with an HP ZBook Fury G8 (i7-11850H, 32 GB

RAM, RTX A3000 with 6 GB VRAM) as a dedicated Ollama inference server to release the home workstation from running a local LLM (see Section 4.5.3). Second, the Gantt chart was updated to the final schedule (Figure B.1). An early framework prototype was built to probe the feasibility of running APR on a local LLM before committing to the full framework build, as described in Section 4.1.1.

Chapter 2

State of the Art

2.1 Large Language Models

When LLMs first emerged, they were text-based models pre-trained on large corpora of natural language. Early systems were used primarily as chatbots and for classical **Natural Language Processing (NLP)** tasks, such as machine translation, sentiment analysis, and question answering, where the goal was simply to enable machines to understand and generate human language. Contemporary LLMs have evolved into **multimodal** systems, meaning they are trained on multiple data modalities beyond text, including images, audio, and video. The capability of an LLM scales with three interdependent factors: the number of model parameters, the volume of training data, and the amount of compute used during pre-training, a relationship known as **scaling laws** [Kaplan et al. 2020].

At its core, an LLM is a token predictor: given an input *prompt*, it generates a response token¹ by token. Multimodality is achieved by encoding non-text inputs, such as images and audio, into token sequences that the same architecture can process, rather than by changing the model implementation. The practical utility of an LLM extends considerably when it is equipped with *tools*: external functions that the model can invoke to execute terminal commands, web search, or interpret structured data. It is precisely this tool-use capability that enables LLMs to act as autonomous agents within the MAS architectures [Wang et al. 2024].

¹A *token* is the basic unit of text that a language model processes. Depending on the tokenizer, a token corresponds roughly to a word, a sub-word fragment, or a single character. For example, the word “unbelievable” may be split into the tokens “un”, “believ”, and “able”. Modern LLMs operate on sequences of such tokens rather than on raw characters or whole words.

2.1.1 Hyperparameters

An LLM exposes different categories of hyperparameters depending on the stage of its lifecycle. Understanding this taxonomy is relevant to MAS design since different agents in a multi-agent pipeline can be assigned different inference settings to suit their specialized roles. The five principal categories are (1) **model architecture**, (2) **training**, (3) **attention and transformer-specific**, (4) **inference**, and (5) **fine-tuning** hyperparameters. Of these, only inference hyperparameters are within the design scope of this project. The framework uses pre-trained models served through Ollama, so architecture, training, attention, and fine-tuning hyperparameters are fixed by the model provider and not revisited here.

Inference hyperparameters, also called **decoding hyperparameters** [Arias et al. 2024], are user-defined values that shape the response generation process at runtime, tunable per-call and without modifying the model weights. Table 2.1 lists the inference hyperparameters exposed by Ollama ².

Table 2.1: Ollama inference hyperparameters, listing for each parameter its default value, accepted range, and effect on token generation.

Parameter	Description	Value Type	Example Usage
num_ctx	Sets the size of the context window used to generate the next token. (Default: 2048)	int	num_ctx 4096
repeat_last_n	Sets how far back the model looks to prevent repetition. (Default: 64, 0 = disabled, -1 = num_ctx)	int	repeat_last_n 64
repeat_penalty	Sets how strongly to penalize repetitions. A higher value (e.g., 1.5) will penalize repetitions more strongly, while a lower value (e.g., 0.9) will be more lenient. (Default: 1.1)	float	repeat_penalty 1.1
temperature	The temperature of the model. Increasing the temperature will make the model answer more creatively. (Default: 0.8)	float	temperature 0.7
seed	Sets the random number seed to use for generation. Setting this to a specific number will make the model generate the same text for the same prompt. (Default: 0)	int	seed 42
stop	Sets the stop sequences to use. When this pattern is encountered the LLM will stop generating text and return. Multiple stop patterns may be set by specifying multiple separate stop parameters in a model file.	string	stop "AI assistant:"
num_predict	Maximum number of tokens to predict when generating text. (Default: -1, infinite generation)	int	num_predict 42

(continued on next page)

²ollama/ollama/docs/modelfile.mdx

(continued from previous page)

Parameter	Description	Value Type	Example Usage
<code>top_k</code>	Reduces the probability of generating nonsense. A higher value (e.g., 100) will give more diverse answers, while a lower value (e.g., 10) will be more conservative. (Default: 40)	int	<code>top_k 40</code>
<code>top_p</code>	Works together with <code>top_k</code> . A higher value (e.g., 0.95) will lead to more diverse text, while a lower value (e.g., 0.5) will generate more focused and conservative text. (Default: 0.9)	float	<code>top_p 0.9</code>
<code>min_p</code>	Alternative to <code>top_p</code> , aiming to ensure a balance of quality and variety. The parameter p represents the minimum probability for a token to be considered, relative to the probability of the most likely token. For example, with $p = 0.05$ and the most likely token having a probability of 0.9, logits with a value less than 0.045 are filtered out. (Default: 0.0)	float	<code>min_p 0.05</code>

In MAS-based APR, these hyperparameters can be set independently per agent. For example, a fault-localization agent may use low temperature to produce consistent file-path identifiers, while a patch-generation agent may use a higher temperature to explore a broader set of candidate fixes.

2.1.2 Prompt Engineering

Prompt engineering is the discipline of designing input text to elicit desired behaviors from an LLM without modifying its weights [Schulhoff et al. 2025]. Because the sole interface to an LLM is natural language, the content of a prompt substantially affects the quality of the output. Key techniques cataloged by Schulhoff et al. (2025) include:

1. **Zero-shot prompting:** task description only, with no worked examples. Example from Lingma-SWE-GPT’s file locator³:

```
OUTPUT AT MOST 10 LOCATIONS that need to be modified in
their workspace and sort them by modification priority.
```

2. **Few-shot prompting:** prepending input-output examples to guide generation. Example from aider’s patch coder⁴:

³agent_locate_file_class_func.py#L7

⁴patch_prompts.py#L11

Example 1

User: "Change get_factorial() to use math.factorial"

Assistant: "Import the math module; replace the custom recursive factorial() helper with math.factorial; update get_factorial() to call math.factorial."

Example 2

User: "Refactor hello() into its own file."

Assistant: ...

3. **Chain-of-thought prompting:** instructing the model to reason step by step before producing an answer. Example from ExpeRepair's patch writer⁵:

Explain your reasoning step by step, then write your patch using the following strict format: each modification must be enclosed in <file>...</file>, <original>...</original>, <patched>...</patched>.

4. **Role prompting:** assigning an expert persona through an opening instruction. Example from HyperAgent's navigator⁶:

You are an expert in finding all relevant information inside a code repository to answer the query from a planner agent. No need to propose a solution.

In the context of MAS-based APR, prompts are the primary mechanism through which agents receive identity, procedural knowledge, and task-specific objectives. A single model invocation may carry up to three structurally distinct prompt types, each contributing a different layer of information:

- **System** prompts establish the model's role and behavioral constraints (the agent's identity and operating constraints).
- **Proxy** prompts supply procedural scaffolding (the *how*) such as tool descriptions, output format requirements, or intermediate reasoning templates. This category neither establishes identity nor specifies the concrete task; rather, it bridges the system context and the user request.
- **Human** prompts (also called **User** prompts) carry the concrete objective issued by an end user or an upstream agent (the *what*).

⁵agent.write_patch.py#L119

⁶plan.seeking.py#L12

This three-level taxonomy is applied in the empirical analysis to classify the prompts found across the open-source *SWE-bench* leaderboard repositories examined in this thesis.

2.1.3 Local versus Cloud Inference

LLMs can be served in two fundamentally different deployment modes: locally on personal hardware, or remotely via a cloud provider’s API.

Local inference, used in this project via Ollama [Ollama 2024] (a lightweight runtime that downloads and serves open-weight models), keeps all data on the machine, providing full privacy and eliminating per-token API costs. The principal constraint is available hardware memory: the maximum model size that can be loaded is bounded by RAM and VRAM. **Scaling laws** predict that capability grows with model size, so this hardware ceiling directly limits achievable performance. For the development machines used here (AMD Ryzen 5 5600G, 16 GB RAM; HP ZBook Fury G8, Intel i7-11850H, 32 GB RAM, NVIDIA RTX A3000 Mobile), this restricts inference to models in the 7–13 B parameter range.

Cloud inference, offered by providers such as OpenAI, Anthropic, and Google, runs models on high-performance computing clusters, granting access to frontier-scale models (100 B+ parameters) with substantially larger context windows. The trade-offs are that data leaves the local environment, raising privacy and confidentiality concerns, and usage incurs a monetary cost billed per token consumed.

For the framework prototype in this project, local inference via Ollama is preferred to avoid third-party data exposure and API costs. Cloud models remain a viable option for the final *SWE-bench* evaluation, where performance is the priority.

2.1.4 LLMOps

LLMOps (Large Language Model Operations) is the discipline of operating LLM-based applications in production, extending classical **MLOps** practices to the specific concerns of language models [Stone et al. 2025]. The lifecycle of an operational LLM deployment is summarized in Figure 2.1 as a seven-step iterative loop:

1. **Data Privacy Governance:** classifying the data that the system is allowed to see (public, internal, regulated) and establishing the corresponding access controls before the data is exposed to any model.

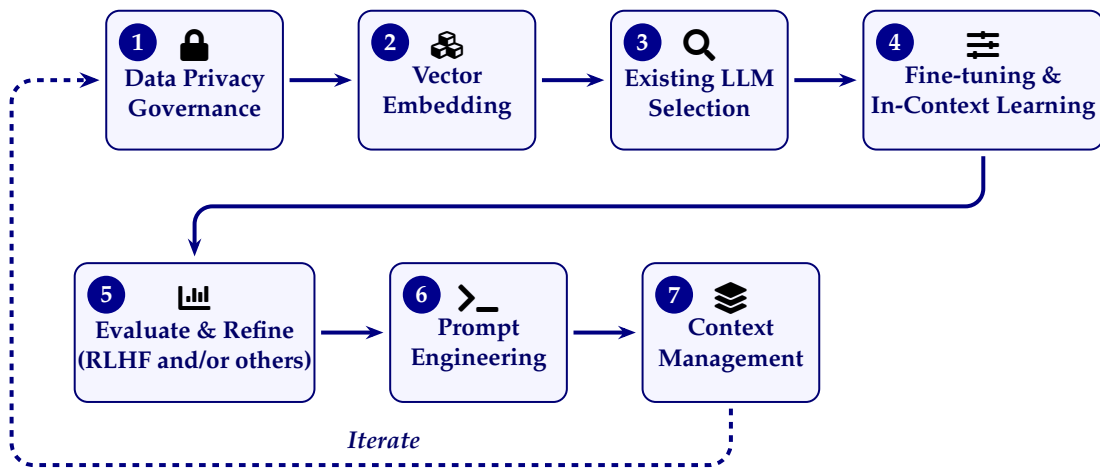


Figure 2.1: LLMOps lifecycle: the seven iterative steps that operationalize an LLM-based application. Adapted from [Stone et al. 2025].

2. **Vector Embedding:** converting each text chunk of the knowledge base into a high-dimensional numerical vector that captures its semantic meaning. The vector space is structured so that semantically similar texts produce nearby vectors. These vectors are then stored in an index that the agents query at inference time to retrieve the chunks most relevant to a given prompt, which is the foundation of **retrieval-augmented generation**.
3. **Existing LLM Selection:** surveying the available pre-trained models and picking the one whose capability, latency, and cost fit the task.
4. **Fine-tuning and In-Context Learning:** two complementary ways to specialize the chosen model for the target domain. **Fine-tuning** updates a subset of the model’s weights through additional training on domain-specific data, permanently changing what the model knows. **In-context learning** leaves the weights frozen and instead conditions the model at inference time by injecting worked examples into the prompt. The model itself is unchanged, so the conditioning can be reverted by clearing the prompt, unlike fine-tuning, which requires retraining to undo.
5. **Evaluate and Refine:** measuring the deployed system against held-out benchmarks, such as *SWE-bench*, and refining it through techniques like reinforcement learning from human feedback (RLHF) or other preference-tuning methods.

6. **Prompt Engineering:** designing the system, proxy, and user prompts that drive each agent, following the techniques cataloged in Section 2.1.2.
7. **Context Management:** deciding how the finite context window is allocated across system prompts, retrieved snippets, intermediate agent messages, and tool outputs, so that the most relevant information remains visible to the model.

In the context of MAS-based APR, LLMOps concerns are sharpened by the per-agent structure of the system: each agent carries its own prompt, its own inference hyperparameters (see Table 2.1), and its own tool set. The framework developed in this thesis externalizes all per-agent configuration to a single JSON topology specification, which makes the LLMOps surface explicit: changing an agent’s model, prompt, or tool list is an edit to a configuration file rather than a change to source code.

2.1.5 LLMs for Code

Beyond general-purpose language modeling, a family of models has been pre-trained or fine-tuned specifically on large collections of source code and software engineering workflows [Hou et al. 2024].

In the context of APR, code LLMs are typically deployed across three inter-dependent sub-tasks: **fault localization**, in which the model identifies the probable defect site given a failing test and a repository snapshot; **patch generation**, in which it proposes one or more candidate fixes; and **patch validation**, in which it assesses patch correctness before or alongside test execution.

A fundamental constraint shared by all current code LLMs is their finite context window. A full software repository far exceeds what any single prompt can accommodate, so practical APR systems must implement retrieval or summarization strategies to reduce the repository to a prompt-sized representation without discarding the context required for a correct repair [Jimenez et al. 2024]. This constraint directly motivates both the empirical analysis of existing systems and the architectural decisions made in the framework developed in this thesis.

2.2 Multi-Agent Systems (MAS)

When a repair task requires inspecting multiple files, tracing execution paths across modules, and producing a syntactically correct patch that passes the tests,

no single LLM invocation can absorb the full context [Jimenez et al. 2024]. A **Multi-Agent System** addresses this constraint by distributing the repair work across a workflow of cooperating agents, each with a specific system prompt and tools [Wang et al. 2024]. Rather than compressing the entire problem into one prompt, the MAS decomposes it, assigns each sub-task to a specialist agent, and coordinates the results.

The structural arrangement of agents, specifically how they communicate and how control authority is distributed among them, constitutes the **topology** of the MAS. Centralized arrangements simplify global coordination but introduce bottlenecks as the agent count grows, while decentralized and distributed configurations improve resilience and enable parallelism at the cost of higher communication overhead [Moore 2025]. This section surveys the principal topology classes and their trade-offs.

2.2.1 Topology Structures

Modern literature classifies agent network topologies based on their structural connectivity and agent mobility [Zhang et al. 2006]. The following subsections define the primary topology classes.

2.2.1.1 Centralized (Star)

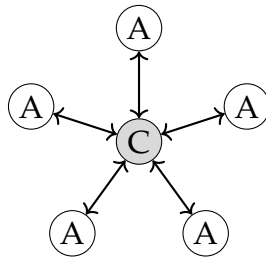


Figure 2.2: Centralized (star) topology: a single central node holds the global state and exchanges messages with each spoke agent.

- **Description:** As illustrated in Figure 2.2, all subordinate agents communicate directly with a single central coordinating node. In LLM-based systems, this architecture relies strictly on radial communication:
 - *Center Node (Router/Proxy):* The center node acts as the master. It holds the global state, processes all incoming user requests, and dispatches specific prompts to the corresponding agents.

- *Spoke Nodes (Agents)*: Specialized workers (e.g., “Coder”, “Reviewer”) that only receive inputs from and return outputs to the center. They do not communicate with each other.
- **Advantages:** The architecture is simple to design and implement, and enables efficient resource allocation because the hub maintains a global view of all ongoing sub-tasks [Tran et al. 2025, Moore 2025].
- **Disadvantages:** The central hub is a single point of failure: if the central node fails, the entire system may collapse, leaving it less resilient to disruptions [Tran et al. 2025]. As the number of agents or the length of the repair task grows, the hub’s context window fills rapidly, forcing lossy summarization of earlier exchanges [Liu et al. 2023].

2.2.1.2 Decentralized and Distributed

- **Description:** In a decentralized and distributed topology, control authority is spread across multiple agents rather than concentrated in a single hub. Tran et al. (2025) treat these configurations as one category in which collaboration decisions are distributed among multiple peers, in contrast to the centralized case where a single authority controls all interactions. Two structural patterns characterize this architecture.

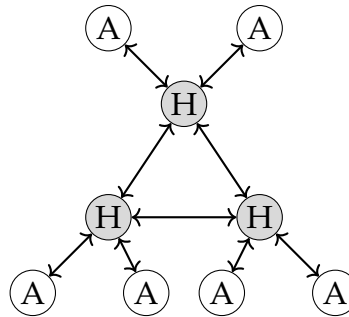


Figure 2.3: Decentralized topology: control authority is partitioned across multiple regional hubs that coordinate through hub-to-hub edges and delegate execution to worker agents.

- *Multi-hub*: As illustrated in Figure 2.3, control authority is partitioned among several regional hubs. Hub-to-hub edges carry high-level coordination between manager-level nodes, while hub-to-agent edges delegate execution to worker agents that report exclusively to their designated hub rather than to a global master.

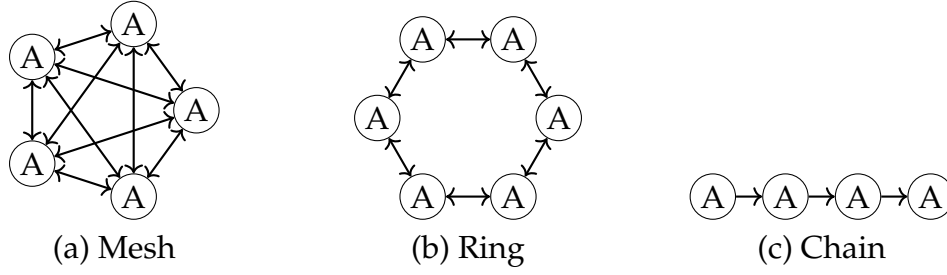


Figure 2.4: Common distributed topologies: (a) mesh, where agents are fully or partially interconnected; (b) ring, a closed loop suited to round-robin tasks; (c) chain, a sequential pipeline that passes information in one direction.

- *Peer-to-peer (Mesh, Ring, and Chain)*: As shown in Figure 2.4, this family of patterns eliminates dedicated hub nodes entirely. In a **mesh**, agents are fully or partially interconnected, enabling multi-path peer negotiations. A **ring** forms a closed loop suited to round-robin tasks and natural load balancing. A **chain** (sequential pipeline) passes information step by step in one direction.
- **Advantages**: Distributing coordination reduces dependence on any single decision-maker; if one hub produces incorrect reasoning, its sub-tasks can be reassigned to another without stalling the entire system [Tran et al. 2025]. Architectures that favor sparser communication graphs have been shown to reduce token consumption by up to 95% while maintaining competitive task performance [Zhang et al. 2025], and dense mesh structures support tasks that benefit from peer debate [Du et al. 2023].
- **Disadvantages**: Coordinating state across multiple authoritative hubs introduces high communication overhead, and achieving coherent global behavior becomes harder as the number of hubs grows [Tran et al. 2025]. Sparser topologies suppress error propagation, yet the optimal sparsity level is non-trivial: topologies that are too sparse also restrict beneficial information diffusion, while moderately sparse structures achieve the best balance [Shen et al. 2025]. Dense topologies such as meshes exhibit greater susceptibility to error propagation, wherein erroneous outputs from individual agents are more likely to compromise the final prediction [Shen et al. 2025], and their denser communication graphs consume substantially more tokens than sparser alternatives [Zhang et al. 2025].

2.2.1.3 Hierarchical (Tree)

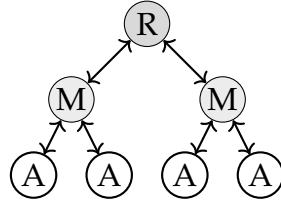


Figure 2.5: Hierarchical (tree) topology: agents are organized into layered tiers with strictly vertical communication paths from root managers down to leaf workers.

- **Description:** As shown in Figure 2.5, agents are organized into multi-layered tiers characterized by strictly vertical communication paths. Hierarchical MAS can be characterized along five axes: control hierarchy, information flow, role and task delegation, temporal layering, and communication structure [Moore 2025]. In practice, two recurring node types emerge:
 - *Root / Managers:* Top-level agents that do not execute concrete tasks. Instead, they decompose large goals into smaller sub-tasks and delegate them downward.
 - *Leaf Agents:* Specialized workers that execute the highly specific sub-tasks delegated to them and pass results back up the branch.
- **Advantages:** Distributing tasks across levels reduces the bottleneck present in star topologies, and a divide-and-conquer strategy enables efficient resource allocation at each level [Tran et al. 2025, Moore 2025].
- **Disadvantages:** Mid-level manager agents become critical points: poor reasoning by a manager propagates corrupted sub-tasks to every agent beneath it [Shen et al. 2025]. The multi-level routing also introduces latency and architectural complexity that grows with tree depth [Tran et al. 2025].

2.2.2 Advanced Coordination and Orchestration Architecture

The three fundamental structures defined in Section 2.2.1 (centralized, decentralized and distributed, and hierarchical) each fix the communication graph at design

time. This subsection covers two advanced arrangements that move beyond a single static structure: **hybrid topologies**, which combine two or more of the fundamental structures within one system, and **dynamic topologies**, which reconfigure the communication graph at runtime.

2.2.2.1 Hybrid Topologies

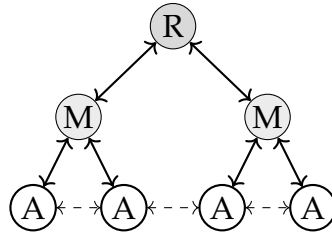


Figure 2.6: Hybrid topology: a hierarchical tree (solid lines) overlaid with a distributed mesh among the leaf agents (dashed lines).

- **Description:** A hybrid topology combines two or more distinct architectural patterns within one framework. Figure 2.6 illustrates a highly common MAS implementation:
 - *Vertical Hierarchy (Solid Lines):* High-level task decomposition handled via a tree structure.
 - *Horizontal Distributed Layer (Dashed Lines):* At the leaf level, agents are allowed to collaborate peer-to-peer to solve granular problems before reporting back to their manager.
- **Advantages:** By combining different topologies, a hybrid system aims to achieve global efficiency while preserving local responsiveness, thereby capturing the advantages of each component topology [Moore 2025].
- **Disadvantages:** The combination of two coordination paradigms introduces significant complexity, making routing and shared state management substantially harder to implement correctly [Moore 2025].

2.2.2.2 Dynamic Topologies

- **Description:** Dynamic topologies depart from the fixed communication graphs of all preceding structures: rather than determining agent connectivity at design time, the system restructures itself at runtime in response

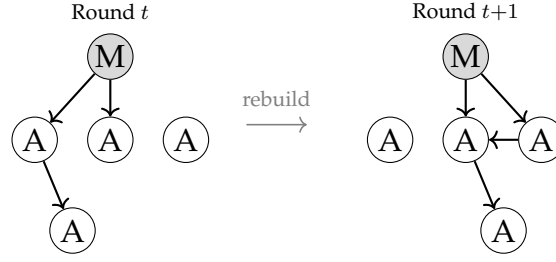


Figure 2.7: Dynamic topology: the communication graph (shown at two consecutive rounds) is rebuilt at each step, with only the edges relevant to the current sub-problem activated.

to the task’s evolving demands [Tran et al. 2025]. Connections form when a sub-task demands specific expertise, dissolve once the relevant exchange concludes, and new specialist agents may be spawned. As illustrated in Figure 2.7, the active graph differs from one step to the next: only a sparse subset of edges is active at any given round. DyTopo is a manager-guided multi-agent framework in which each agent continuously generates a *need* descriptor and a *key* descriptor that are matched semantically each round to determine the next active edge set [Lu et al. 2026].

- **Advantages:** Because connectivity is rebuilt per step, communication is routed only where the current sub-problem demands it, reducing unnecessary inter-agent traffic compared to a fully connected static graph [Tran et al. 2025]. Dissolving connections once a sub-task concludes also prevents context-window accumulation across the system. This adaptability makes dynamic topologies well-suited to tasks such as APR, where the required sequence of expertise cannot be determined before execution begins [Lu et al. 2026].
- **Disadvantages:** Without a fixed communication graph, the system must determine the active topology at every step, adding coordination overhead that any static topology avoids [Tran et al. 2025]. Maintaining coherent shared state across connections that form and dissolve at runtime is substantially harder to implement correctly, and the absence of a predictable routing structure complicates debugging and reproducibility [Lu et al. 2026].

2.3 Automated Program Repair (APR)

APR is the task of automatically producing a semantically correct patch given evidence of a fault, without requiring the programmer to specify the repair. Program repair is among the most actively investigated software engineering tasks for LLMs [Hou et al. 2024]. As introduced in Chapter 1, the repair workflow decomposes into three sub-tasks: **fault localization**, which identifies the code element most likely responsible for the failure; **patch generation**, which synthesizes one or more candidate fixes for a specific issue; and **patch validation**, which determines whether a candidate patch makes all failing tests pass without introducing regressions [Xia et al. 2024].

The practical difficulty of APR on real-world repositories arises from scale and context. A real-world codebase may span hundreds of thousands of lines distributed across different modules, so the true fault site may be several function calls away from the code that visibly fails, and a correct patch must preserve the semantic correctness of the codebase, not merely pass the tests. The gap between generating isolated code completions and performing genuine repository-level repair remains large. The *SWE-bench* benchmark, which evaluates systems on 2,294 real GitHub issues across Python repositories averaging 438,000 lines of code, demonstrated this gap directly: the best model at the benchmark’s introduction, Claude 2, resolved only 1.96% of instances [Jimenez et al. 2024].

This limitation has motivated the adoption of MAS pipelines in which multiple LLMs collaborate under a structured topology. Each agent carries a focused system prompt and a specialized tool set, and together they execute the fault-localization, patch-generation, and patch-validation workflow. The open-source repositories submitted to the *SWE-bench* leaderboard [SWE-bench Team 2024a] document a wide variety of such architectures, from agentless pipelines to fully autonomous multi-agent frameworks. These systems are examined in detail in Chapter 3.

2.4 Frameworks for LLM Applications

Several software frameworks are available to simplify the construction of LLM-based applications and MAS. These frameworks encapsulate repetitive infrastructure concerns such as prompt formatting, tool registration, state management, and inter-agent communication, and expose higher-level abstractions suited to build-

ing complex multi-step workflows. The following are the frameworks most relevant to the APR systems examined in this thesis.

- **LangChain and LangGraph.** LangChain ⁷ is a modular framework for composing LLM-powered applications. It provides abstractions for prompt templates, tool integrations, memory modules, and agent executors that can be composed into reusable pipelines. LangGraph ⁸ extends LangChain with a stateful, graph-based execution model suited for multi-agent workflows. A `StateGraph` defines a shared typed state that all nodes can read and modify; edges encode control flow, enabling cycles, branches, and parallel execution within a single coordinated workflow. This combination makes LangGraph the primary orchestration layer for the framework developed in this thesis.
- **LlamaIndex.** LlamaIndex ⁹ is a data framework that connects LLM applications to external knowledge sources. Its core abstraction is **retrieval-augmented generation (RAG)**: rather than compressing an entire repository into a prompt, an index is built over the document collection and only the fragments most relevant to the query are retrieved at inference time. In the APR context, LlamaIndex is used in several leaderboard systems to narrow the search space during fault localization by selectively surfacing the most relevant source files and functions.
- **AutoGen.** AutoGen [Wu et al. 2023] is a multi-agent conversation framework developed by Microsoft Research. Its central abstraction is the **conversable agent**: an entity that can send and receive messages, invoke tools, and optionally execute code. Agents are composed into workflows through predefined or dynamic conversation patterns including two-agent dialogues, group chats, and nested chats. AutoGen’s code-execution agents are particularly relevant to APR, where iterative test-and-fix loops between a coding agent and a verification agent are a common design pattern.
- **CrewAI.** CrewAI ¹⁰ is a framework for orchestrating role-playing autonomous agents. Each agent is defined by a role, a goal, and a backstory that shape its system prompt, and agents are grouped into crews that execute a shared task list. Crews can operate in sequential mode (tasks

⁷[langchain-ai/langchain](https://github.com/langchain-ai/langchain)

⁸[langchain-ai/langgraph](https://github.com/langchain-ai/langgraph)

⁹[run-llama/llama_index](https://github.com/run-llama/llama_index)

¹⁰[crewAIInc/crewAI](https://github.com/crewAIInc/crewAI)

completed one after another) or hierarchical mode (a manager agent delegates and reviews sub-tasks). The role-centric design maps naturally to the fault-localization, patch-generation, and review workflow common in APR.

- **MetaGPT.** MetaGPT [Hong et al. 2024] assigns standard software engineering roles (product manager, architect, engineer, and quality assurance) to LLM-backed agents organized under a structured operating procedure. Agents communicate through standardized artifacts such as requirements documents, design specifications, and code review reports rather than free-form messages, enforcing rigorous inter-agent handoffs. The explicit role hierarchy and document-based communication distinguish MetaGPT from more flexible frameworks and make it well suited to workflows where structured development process compliance is a priority.

2.5 SWE-bench Leaderboard Problems Subset and Evaluation Metrics

SWE-bench [Jimenez et al. 2024] is the primary evaluation framework for LLM-based APR. Given a repository and a natural-language description of a GitHub issue, the model must produce a patch that resolves the issue. A patch is counted as correct if, after being applied to the repository, all designated **fail-to-pass** tests (those that were failing before the fix) now pass, and no **pass-to-pass** tests (those already passing before the fix) regress. The **resolution rate**, the percentage of correctly resolved instances, is the primary metric reported on the leaderboard [SWE-bench Team 2024a]. The *SWE-bench* leaderboard catalogs submissions across five scoring categories: Verified [Chowdhury et al. 2024], Lite, Full, Bash Only, and Multimodal, though the repositories examined in this empirical analysis come exclusively from Verified, Lite, and Full.

2.5.1 Verified

SWE-bench Verified [Chowdhury et al. 2024] is a 500-problem subset produced through a human validation campaign carried out in collaboration between Princeton University and OpenAI Preparedness, released on August 13, 2024. Professional software engineers reviewed 1,699 problems from the original *SWE-bench* test set against several criteria: clarity of the problem statement, correctness of the

unit tests, and solvability without external context. Of the 1,699 reviewed problems, 1,199 (70.6%) were filtered out, leaving 500. Among the discarded instances, 38.3% were removed for underspecified problem descriptions and 61.1% for unit tests that could unfairly penalize valid solutions.

2.5.2 Lite

SWE-bench Lite is a 300-problem subset drawn from 11 of the 12 original repositories, designed to reduce evaluation costs while maintaining benchmark quality [SWE-bench Team 2024b]. Instance selection applied several strict filters: problem statements must contain at least 40 words; patches must modify at most one file with at most three edit hunks; instances must not reference external URLs, commit SHAs, or other pull requests; and test suites must not rely on error-message-content checks.

2.5.3 Full

The full *SWE-bench* [SWE-bench Team 2024c] contains 2,294 problems collected from 12 popular Python repositories: *astropy*, *Django*, *Flask*, *matplotlib*, *NumPy*, *pandas*, *Pillow*, *pytest*, *Requests*, *scikit-learn*, *seaborn*, and *SymPy* [Jimenez et al. 2024]. Problems were constructed from merged pull requests: each instance pairs a repository snapshot (before the fix) with the natural-language issue description, and the evaluation oracle is derived from the test suite added or modified by the pull request. The full set provides the most complete evaluation but is computationally expensive. Single-LLM approaches without scaffolding originally resolved fewer than 2% of instances (Claude 2: 1.96%), establishing the need for multi-agent and retrieval-augmented frameworks [Jimenez et al. 2024]. The full set remains the ground truth for measuring the upper bound of system capability, while Lite and Verified serve as the practical evaluation subset for faster iteration.

2.6 Similar Products and Empirical Studies

This section positions *EvoMas* relative to existing visual workflow builders and reviews prior empirical work on MAS topology comparison.

2.6.1 Similar Existing Products: Agent Workflow Builder

EvoMas is the framework developed as the second contribution of this thesis. It is an open-source, APR-oriented platform that compiles a JSON topology specification into an executable LangGraph workflow, in which each node corresponds to a specialist agent (locator, patcher, reviewer, helper/proxy, orchestrator, bug-reproducer, environment-setup) and each edge represents the unidirectional communication between two agents. Each agent issues calls to a local LLM served via Ollama, optionally invokes registered tools, and passes its results to the agents it is connected to through a typed shared state. *EvoMas* is distinguished from alternative builders by an architecture intended to support topology evolution: agents can be added, removed, or rewired between runs by editing the configuration, with full runtime evolution identified as the principal direction for future work (see Section 8.3). A broad family of platforms already targets the construction of LLM-based applications and agent workflows. Table 2.2 positions *EvoMas* against the most relevant of them. The selection spans general LLM workflow builders (Flowise, Langflow, n8n, Dify), commercial SaaS automation and conversational tools (Gumloop, Latenode, Voiceflow), and visual layers over established multi-agent frameworks (AutoGen Studio [Microsoft Research 2024] over Microsoft AutoGen [Wu et al. 2023], LangGraph Studio [LangChain AI 2024] over LangGraph¹¹, CrewAI Studio¹² over CrewAI¹³). All of these platforms and *EvoMas* share a visual builder for an LLM-based workflow, so the visual-builder dimension does not discriminate between them and is omitted from the table. The dimensions that do matter for this thesis are the target domain, whether the platform is open source, whether topology can be reconfigured beyond a designer-defined static graph, whether the platform integrates with local self-hosted LLM runtimes such as Ollama, and whether an *SWE-bench*-style evaluation harness is integrated.

2.6.2 Similar Empirical Studies: Topology and Agent Organization

The intuition that the structural arrangement of communicating units shapes the systems they produce predates the emergence of LLM-based MAS by several decades. Conway’s Law [Conway 1968] observes that “any organization that

¹¹langchain-ai/langgraph

¹²strnad/CrewAI-Studio

¹³crewAIInc/crewAI

Table 2.2: Comparison of visual LLM-application platforms against the *EvoMas* framework on the dimensions most relevant to this thesis.

Platform	Domain	Open source	Reconfigurable	Local LLM	SWE-bench eval
Flowise	General	Yes	No	Yes	No
Langflow	General	Yes	No	Yes	No
n8n	Automation	Yes	No	Yes	No
Dify	General	Yes	No	Yes	No
Gumloop	General	No	No	No	No
Latenode	Automation	No	No	Partial	No
Voiceflow	Chat / voice	No	No	No	No
AutoGen Studio	MAS	Yes	No	Yes	No
LangGraph Studio	MAS	No	No	Yes	No
CrewAI Studio	MAS	Yes	No	Yes	No
<i>EvoMas</i> (this thesis)	APR	Yes	Planned	Yes	Yes

designs a system [...] will inevitably produce a design whose structure is a copy of the organization’s communication structure”. Although Conway’s Law was originally formulated for human engineering teams, the principle transfers directly to multi-agent systems. A small but illustrative test of this transposition is reported by [Sima \(2024\)](#), who take six real-world corporate structures (Amazon, Google, Facebook, Microsoft, Apple, and Oracle), encode each as an agent topology, and benchmark the resulting systems on a 13-instance subset of *SWE-bench*. The experimental ranking places the topologies modeled on Apple and Microsoft, both built around multiple competing internal teams, in the top two positions, and the topologies modeled on Google, Amazon, and Oracle, dominated by a single chain of command, at the bottom. This ranking matches the order of the same firms by real-world market value: Apple and Microsoft, the two largest companies by capitalization at the time of the experiment, are also the two whose organizational patterns yield the best results when replicated as agent topologies.

The TopoAgent position paper of [Yang et al. \(2025\)](#) argues that the topology of LLM-based MAS (how agents are configured, connected, and coordinated) has been under-studied relative to its impact on task performance, and proposes a three-stage design discipline (agent selection, structure profiling, and topology synthesis) for treating topology as a first-class research artifact. A complementary study by the same research group [[Jin et al. 2023](#)] evaluates how the answers produced by LLMs vary with the language of the prompt by stress-testing models on healthcare queries issued in multiple languages and reporting substantial degradations outside English. Together, the two contributions illustrate that systematic evaluation of LLM-based systems requires explicit attention to structural

variables (agent topology in one case and prompt language in the other) that standard benchmarks tend to ignore.

From a different angle, [Li et al. \(2024\)](#) show that the simplest possible “topology”, a flat ensemble of agents combined through sampling-and-voting, already improves LLM performance proportionally to the agent count, with harder tasks benefiting more. This isolates the contribution of redundancy from that of explicit coordination structure.

The empirical analysis carried out in Chapter 3 of this thesis pursues a related but narrower research question. Rather than benchmarking topology variants on synthetic tasks or transposing prefabricated organizational templates, it examines the topologies actually deployed by open-source *SWE-bench* leaderboard submissions and links them to the prompt, agent-role, and autonomy choices made by their authors. The studies surveyed above therefore serve as methodological reference points for how to operationalize topology comparison, while the present work contributes a domain-specific characterization grounded in APR practice.

Chapter 3

Empirical Study of Architecture and Topologies of MAS for APR

3.1 Research Questions

The *SWE-bench* leaderboard provides a unique empirical window into how the research community designs and deploys LLM-based APR systems at scale. While individual systems have been described in the literature, no systematic and cross-cutting characterization of their framework dependencies, prompt design, agent roles, autonomy levels, and topology choices exists across the full leaderboard. Five research questions structure the empirical analysis conducted in this chapter. Together they characterize the architectural landscape of MAS-based APR systems along the dimensions relevant to the framework developed in Chapter 4.

RQ₁ surveys the infrastructure layer. MAS developers depend on agent frameworks to handle state management, message passing, and tool integration, rather than reimplementing this infrastructure each time from scratch. The common platforms available for this purpose are introduced in Section 2.4.

RQ₁. Agent Framework Adoption

What frameworks exist for building LLM-based MAS, and which are most commonly adopted in SWE-bench APR solutions?

Identifying which of these frameworks each leaderboard submission actually adopts, as reported in Section 3.3.2, yields a short list of platforms already validated on real APR workloads, and that short list is the starting point for the **technology stack** adopted by the framework in Chapter 4.

RQ₂ examines the communication layer between agent and model. Just as

human communication relies on stylistic conventions such as tone to transmit the right intention, LLM prompts encode constraints through specific linguistic choices. Building on the system / proxy / human prompt-type taxonomy already established in Section 2.1.2, identifying the stylistic patterns that developers use in their prompts reveals why certain prompts produce more reliable agent behavior than others.

RQ₂. Prompt Engineering Patterns

What stylistic patterns appear in the prompts of LLM-based APR agents, and how do they influence model behavior?

The analysis in Section 3.3.4 classifies the prompts of each analyzed repository against the eight stylistic patterns introduced in Section 3.2.4. The resulting evidence of which patterns are consistently used across leaderboard submissions is then carried into Chapter 4, where it guides how the agent prompts of the framework are written.

RQ₃ investigates the functional specialization of agents. Just as a software company distributes responsibilities across engineers, testers, and reviewers to manage complexity, a MAS for APR must divide the repair workflow into specialized roles. Not every role present in a system is strictly required, some are architectural choices while others reflect unavoidable sub-tasks of the repair problem itself. Distinguishing essential from optional roles clarifies the minimum viable agent set for APR.

RQ₃. Agent Role Characterization

What agent roles are necessary to solve APR tasks in MAS, and which roles appear consistently across independent SWE-bench submissions?

The role taxonomy derived in Section 3.3.5 reduces the design space for new APR frameworks and directly drives the agent design of the framework in Chapter 4 by establishing which roles must be present in any topology configuration.

RQ₄ examines where control authority actually resides in deployed systems. Autonomy is frequently cited as a defining property of agent-based systems, yet the boundary between an agent deciding its next action through LLM inference and a software pipeline invoking the next function through hardcoded logic is often blurred in practice. Clarifying this boundary is essential for understanding what **agent-based** genuinely means in the context of SWE-bench submissions.

RQ₄. Agent Autonomy

To what degree are LLM-based APR agents truly autonomous, and how much of the execution flow is controlled by the underlying software rather than by agent decisions?

The autonomy classification in Section 3.3.6 reveals whether the APR community has converged on a preferred level of agent control, identifies the trade-offs between deterministic pipelines and fully autonomous agents in terms of reliability and cost, and informs the autonomy model adopted by the framework in Chapter 4.

RQ₅ investigates the structural dimension of inter-agent communication. The arrangement of agents in a topology acts as the organizational pattern of the system. Different organizational patterns yield different repair outcomes, so the same set of agents can succeed or fail on the same bug purely because of how they are wired together.

RQ₅. Topology Distribution

What types of agent communication topologies appear in SWE-bench APR solutions, and what are the advantages and disadvantages of each?

The topology inventory in Section 3.3.7, paired with a structured advantages-disadvantages analysis, gives framework designers a concrete decision guide for topology selection, identifies which structural patterns dominate under competitive evaluation conditions, and exposes gaps in the deployed landscape that the framework in Chapter 4 is designed to fill.

This study adheres to the standards for repository mining as outlined in the *Repository Mining* and *General Standard* categories of the ACM/SIGSOFT Empirical Standards [Ralph et al. 2021].

3.2 Methodology

The empirical study analyzes a set of open-source repositories submitted to the SWE-bench leaderboard across the Verified, Lite, and Full evaluation categories. Three core principles govern the data collection procedure throughout all phases of the analysis.

Reproducibility. Each repository is pinned to an absolute Git commit hash recorded via `git rev-parse HEAD` at the time of analysis. All code references

in this chapter use permanent, commit-anchored GitHub URLs so that ongoing development of the analyzed projects does not invalidate any cited line or function, and the full analysis can be reproduced from the exact same codebase state.

Structured data management. All collected data (repository metadata, agent classifications, prompt texts, and tool inventories) are organized in a Google Sheets workbook. Controlled-vocabulary dropdown menus enforce consistent classification across repositories, reducing transcription errors and enabling cross-sheet aggregation with spreadsheet formulas.

Systematic coverage. The analysis follows an eight-step protocol applied uniformly to each repository: (1) repository selection, (2) framework analysis, (3) agent analysis, (4) prompt analysis, (5) agent role analysis, (6) autonomy analysis, (7) topology analysis, and (8) graph visualization.

Figure 3.1 summarizes the complete research process at a glance, with each phase tagged by the research question it contributes to. A detailed version that additionally lists the per-phase sub-steps is provided as Figure B.3 in Appendix B. The following subsections detail the methodology of each step.

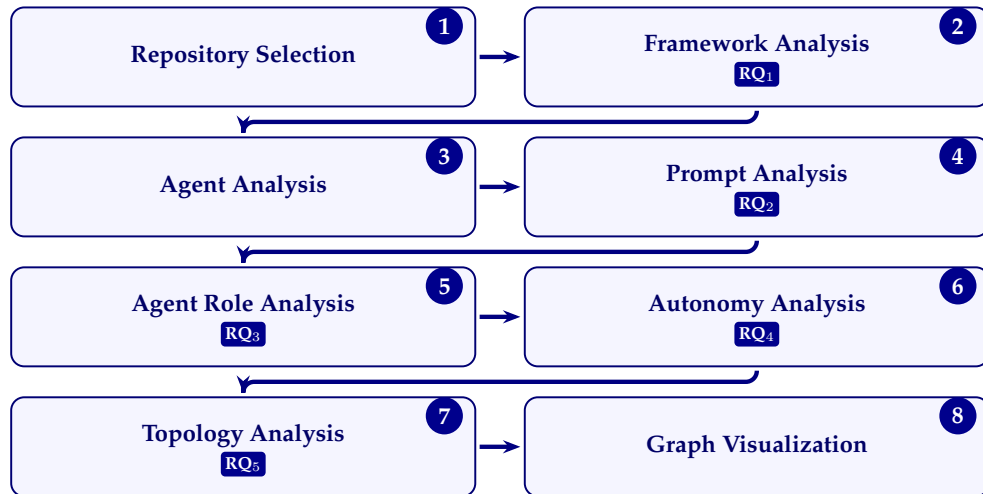


Figure 3.1: Compact overview of the empirical study research process. The eight numbered phases are applied uniformly to each analyzed repository and follow a boustrophedon left-to-right, right-to-left flow. Phases that directly answer a research question are tagged with the corresponding RQ chip. A more detailed version of this diagram, including the per-phase sub-steps applied within each phase, is provided in Figure B.3 in Appendix B.

3.2.1 Repository Selection

Candidate repositories are drawn from the open-source submissions listed on the *SWE-bench* leaderboard across the Verified, Lite, and Full evaluation categories described in Section 2.5, yielding an initial pool of 48 repositories. Analysis priority is assigned first to the architecturally most relevant submissions to build a solid baseline understanding of the APR design space; the remaining repositories are then processed in descending order of GitHub star count.

For each candidate the source code is cloned and `git rev-parse HEAD` is executed to record an absolute commit hash, pinning the **repository URL** to a permanent state independent of any later upstream changes.

Metadata collection. The following fields are extracted for every repository from the GitHub repository page and the project README, if available, regardless of whether full code-level analysis is possible:

- *Commits, releases, and stars.* These three counts, taken directly from the GitHub repository page, serve as recognition indicators confirming that the analyzed submissions represent actively maintained, community-acknowledged projects rather than one-time experiments.
- *Programming language.* The primary language is read from the GitHub language breakdown; repositories not implemented in Python are excluded from code-level analysis.
- *Publication link.* The README is inspected first for a direct reference to a companion paper; if none is present, the repository name and authors are searched on arXiv to locate any associated preprint or technical report.
- *Comments.* An extra column listing each notable implementation feature of the repository that may be integrated into the framework design of this project.

All collected fields are organized in a Google Sheets workbook using the following column schema: *repo, url, commits, releases, stars, Analyzed?, Publication?, Framework used, Dependency graph, Has Agents?, # Agents, Agents Sheet Hyperlink, Topology/Architecture Description, Topology/Architecture File, Topology/Architecture Type, Topology URL, Programming language, Autonomy Rating, Autonomy Description, and Comments*. Columns whose values depend on subsequent analysis steps are

populated as those steps are completed. The resulting master repository table is released as part of the project public companion repository ¹.

3.2.2 Framework Methodology

Following the initial selection and metadata collection, the underlying architecture of each repository is examined to determine its reliance on existing LLM-based orchestration tools. This phase collects the primary data required to answer **RQ₁** regarding agent framework adoption by identifying whether a system builds upon established MAS frameworks or implements its own routing logic.

Dependency graph inspection. The repository’s dependency graph is extracted directly via the GitHub web interface, which provides a comprehensive export of the project’s dependencies in JSON format. This export enumerates the complete list of Python package dependencies (i.e., `pip` imports) declared by the repository.

Regex scan. To confirm and pinpoint the exact usage of these dependencies within the codebase, a regular expression is applied to all Python source files using VS Code’s global search. The list of framework names is built from a web search of established LLM-based orchestration libraries combined with a first pass over the candidate repositories that surfaces additional frameworks in actual use. The resulting expression detects explicit framework import statements:

```
(?:import|from)\s+.*(?:metagpt|langgraph|spade|aiomas|pyMAS|
pettingzoo|magent|langchain|llama.index|crewai|autogen|pydantic-
ai).*
```

Custom architectures and native frameworks. When no established framework import is detected by the scan, the repository typically falls into one of two categories. It either relies entirely on custom, ad hoc orchestration scripts to manage its pipeline, or the repository itself is designed as a general-purpose agent framework (e.g., `OpenHands`, `moatless-tools`). This distinction is recorded to contextualize the system’s topology and autonomy in later phases.

3.2.3 Agent Methodology

Once the repository selection and framework analysis are complete, the agent population of each system is identified and characterized at the implementation level.

¹TFG_public/Open-Source-Proyects-SWE-Bench - Repositories.csv

Agents are located by navigating class hierarchies and symbol definitions using the Pylance static analysis extension for VS Code [Microsoft 2024], combined with targeted regular expressions for specific keywords.

Agent location. The primary strategy applies a global regular expression search for the keyword `prompt`. Because `prompt` is a semantically precise term in LLM-based codebases, matches reliably surface the prompt variable or string literal; all references to that variable are then traced using Pylance’s *Find All References* feature to identify the agent or function that uses the prompt. A complementary search for the keyword `agent` surfaces *classes* whose names incorporate the term.

Prompt classification. To collect the empirical data required to address **RQ₂** regarding prompt engineering, each discovered prompt is assigned a type following the taxonomy defined in Section 2.1.2: *system*, *proxy*, or *human*. The classification draws on two complementary signals: the content of the prompt text itself, and the name of the variable holding the prompt (for example, `system_prompt` or `human_message`), which often disambiguates the type without requiring a full reading of the text.

Tool identification. Once the agent file is located, the keyword `tool` is searched within that file to surface tool registrations, decorator annotations, and tool lists associated with the agent.

For each discovered agent, a dedicated row is added to the repository’s agent sheet in the Google Sheets workbook. The sheet schema comprises: *Name*, *Agent Role* (described in detail in Section 3.2.5), *Short Description*, *Agent in the repo*, three columns for each prompt with *Prompt Type*, *Prompt text*, and *Prompt URL*, one or more *Tool URLs*, and a free-text *Comment* field for extra information.

3.2.4 Prompt Methodology

A Python script ² automates the detection of stylistic patterns in the agent prompts collected during the empirical study. Given the per-agent CSV files in the `agents_csv` directory as input, it scans each prompt text and flags the presence of eight categories:

1. UPPERCASE emphasis words (e.g. IMPORTANT, MUST)
2. directive modals (*should*, *must*, *ensure*)

²TFG_public/First_Part_Analysis/Prompt_Analysis

3. exclamation or question marks
4. XML/HTML markup tags
5. role-definition openers (*You are a . . . , Act as*)
6. negation directives (*do not, never, avoid*)
7. output-format keywords (*JSON, markdown, return*)
8. numbered instruction lists

Results are written to `prompt_analysis.csv` and used to populate the quantitative prompt analysis in Section 3.3.4.

3.2.5 Agent Role Methodology

Building on the per-agent data collected during Section 3.2.3, this phase gathers the data necessary to answer **RQ₃** regarding agent role characterization. Each agent across all analyzed repositories is assigned to one of eight functional role categories. These categories were not predefined but rather derived iteratively from an initial observational analysis of the first processed repositories.

The resulting taxonomy consists of the following eight agent roles:

- **Locator:** Responsible for locating the root cause of the issue within the codebase.
- **Patcher:** Handles the actual code modification. Its duties can include writing a patch, selecting the best patch among multiple generated options, and applying the patch to the file system.
- **Helper/Proxy:** Serves auxiliary functions without modifying code. This includes answering questions about the codebase, providing interactive user support and framework documentation, or acting as a proxy agent to process and format raw LLM responses into structured formats like JSON.
- **Planner/Orchestrator:** Acts as an architectural decision-maker that separates the planning of modifications from their execution. It describes the necessary code changes and delegates the actual file editing to a designated coder agent based on that plan.

- **Base Agent:** A parent class defining common agent behaviors and interfaces, from which other specialized agents inherit.
- **Bug Reproduction:** Attempts to dynamically reproduce reported bugs by writing and executing specific reproduction tests before a fix is attempted.
- **Environment Setup:** Automatically sets up and configures the target repository's development environment, ensuring dependencies are installed for testing and building.
- **Reviewer:** Verifies that the proposed patch is successfully applied and confirms whether the underlying issue has been adequately resolved.

Cross-sheet aggregation and role overlap. To synthesize this categorization, a dynamic Google Sheets formula is used to aggregate agents of each category from the individual repository sheets into a unified master agent-type table. This master table provides the dataset for the role distribution analysis presented later in Section 3.3.5.

The aggregation is driven by the following spreadsheet formula:

```
=TEXTJOIN(", " & CHAR(10), TRUE,
  IFERROR(
    FILTER(
      INDIRECT($A3 & "!$B:$B"),
      ISNUMBER(SEARCH(B$1, INDIRECT($A3 & "!$C:$C")))
    ), "")
)
```

3.2.6 Autonomy Methodology

The **autonomy** of a MAS measures the degree to which agents themselves determine their execution sequence, rather than relying on a software pipeline or human to orchestrate them. See Appendix A for a formal definition.

Each analyzed system is assessed along the autonomy dimension to address **RQ₄** by examining where decision-making authority resides within the execution flow.

By inspecting the orchestrator code and routing mechanisms of each repository, systems are classified into one of three distinct autonomy levels:

- **Orchestrated:** A control program explicitly sequences agent invocations using hardcoded software logic.
- **Semi-autonomous:** Agents decide the sequence subject to external constraints.
- **Autonomous:** Agents collectively determine the full execution sequence without external direction, relying entirely on free-form LLM inference and delegation.

Recording this classification in the structured workbook provides the empirical basis to evaluate whether the open-source APR community favors highly controlled pipelines or fully autonomous multi-agent interactions.

3.2.7 Topology Methodology

After identifying the agents and their URLs in Section 3.2.3, the topology analysis step examines how those agents are composed into a running system, providing the structural evidence needed to answer **RQ₅** regarding topology distribution.

Starting from the individual code locations recorded during the agent analysis, the repository is inspected to locate the main execution runner. From this entry point, the execution flow is systematically traced. By reading call sites, constructor arguments and inter-agent message paths using Pylance extension's *Go to Definition* and *Find All References* capabilities, the communication links between agents are precisely mapped. In addition to this code-level inspection, the project's README or any associated companion paper is reviewed, as these documents frequently provide valuable high-level architectural diagrams and contextual information that clarify the intended system design.

Based on this combined structural and documentary evidence, the overall coordination pattern of each system is classified into one of the topology categories defined in Section 2.2.1.

The findings are recorded in the master Google Sheets workbook using four dedicated columns:

- *Topology/Architecture File:* A reference to any architectural figure extracted from the companion paper or README, if available.
- *Topology/Architecture Description:* A short textual description summarizing the provided architecture figure or system documentation.

- *Topology/Architecture Type*: The actual topology classification assigned to the system.
- *Topology URL*: A permanent, commit-anchored GitHub link pointing to the specific line of code that serves as the structural evidence for the classification.

3.2.8 Graph Visualization Methodology

After conducting the individual repository analysis in the preceding steps, the collected data is consolidated into a single graph artifact suitable for macro-level inspection. To better visualize and explore the complex relationships across this comprehensive dataset, an ETL³ pipeline takes the exported Google Sheets data, specifically the master repository table and the aggregated per-agent CSVs, as input and constructs a directed graph in GEXF format. The implementation is released as an open-source Python tool, `repo_graph`, available at repository⁴.

Five node types are distinguished within the graph: *repository*, *agent*, *agent node*⁵, *prompt*, and *tool*, which are connected by labeled directed edges (`has_agent`, `has_node`, `has_prompt`, `has_tool`).

To facilitate an intuitive, macro-level inspection of the entire analyzed landscape, the resulting graph is loaded into Gephi Lite [Gephi Consortium 2024], a browser-based open-source network visualization tool that runs primarily in the browser as a client-side application. Gephi Lite ingests the GEXF file directly and exposes interactive controls for layout selection, node filtering, and visual encoding.

3.3 Analysis Results

This section reports the findings of the empirical study across the five research questions defined in Section 3.1. The subsections follow the order of the analysis protocol described in Section 3.2.

³*Extract, Transform, Load*: a data integration process that **extracts** raw data from one or more sources, **transforms** it (cleaning, normalizing, or enriching it), and **loads** it into a destination system such as a database or data warehouse.

⁴`TFG-public/First-Part-Analysis/repo_graph-master`

⁵A discrete node within a specific agent subgraph used in Prometheus.

3.3.1 Results of the Repository Selection

The repository selection step yields a pool of 48 candidate APR systems drawn from the *SWE-bench* leaderboard ⁶, summarized in Table 3.1 together with their primary language, commit and release counts, star count, and any companion arXiv publication. Of these, 36 are written primarily in Python and 7 in other languages (Rust, TypeScript, Go, Shell); the remaining 5 do not have a clearly identified primary language. The table also reports the most active projects (*aiders* and *OpenHands*, each with more than 12,000 commits and tens of thousands of stars, with *OpenHands* reaching 67,256 stars and *aiders* 40,160).

Not all 48 candidate repositories are suitable for a full code-level analysis as APR pipeline implementations. Three categories of submissions are either excluded from full analysis or given a reduced methodology: repositories whose public implementations are incomplete, submissions that are fine-tuned LLMs or plug-in components rather than standalone APR pipeline systems, and repositories implemented in languages other than Python. Each category is described next; the remaining repositories proceed through the full protocol.

- **Seven repositories** cannot be fully analyzed due to various reasons, including incomplete public implementations; for these, only the available general data (README, commit history, release notes, star count) is recorded.
- **Four repositories** fall outside the MAS pipeline category: three are open-source large language models specifically designed for software improvement, and one is a plug-in augmentation component rather than a standalone APR system.
- **Seven repositories** are implemented in languages other than Python (TypeScript, Go, Shell, Rust). Code-level analysis is not applicable to these, and only general metadata is recorded.

⁶TFG_public/Open-Source-Proyects-SWE-Bench - Repositories.csv

Table 3.1: *SWE-bench* leaderboard repository candidate pool (48 repositories, ordered by descending star count at the time of analysis). The Language column reports the dominant programming language and its corresponding percentage. “—” denotes missing or not-applicable data; arXiv identifiers in the Publication column are shortened to the numeric portion.

Repository	#Commits	#Releases	#Stars	Language	Publication
OpenHands	12,397	97	67,256	Python (62.0%)	2511.03690
aider	13,003	93	40,160	Python (80.0%)	2509.25873
composio	4,506	542	26,455	Python (21.5%)	—
sunu	2,999	3	19,269	TypeScript (86.3%)	—
SWE-agent	2,787	10	18,349	Python (94.8%)	2405.15793
agentscope	794	27	16,004	Python (100.0%)	2402.14034
trae-agent	320	0	10,685	Python (99.4%)	2507.23370
claude-coder	1,203	0	5,287	TypeScript (98.9%)	—
SWE-bench	637	0	4,200	—	—
Skywork-SWE-32B	52	—	4,200	—	—
refact	7,427	20	3,451	Rust (60.7%)	—
ridges	2,226	—	3,451	Python (99.4%)	—
auto-code-rover	345	2	3,051	Python (98.0%)	2404.05427
aware-swe-agent	11	—	3,051	Python (100.0%)	—
mini-SWE-agent	860	41	2,655	Python (100.0%)	2405.15793
aide	138,645	0	2,190	TypeScript (95.5%)	—
Agentless	32	2	2,007	Python (100.0%)	2407.01489
SuperCoder	108	1	966	Go (62.7%)	—
augment-swebench-agent	13	0	852	Python (99.4%)	—
swe-rl	14	0	672	Python (100.0%)	2502.18449
Prometheus	1,183	0	626	Python (97.4%)	2507.19942
R2E-Gym	143	—	626	Python (68.1%)	GitHub
moatless-tools	480	1	624	Python (97.3%)	2410.20285
CodeFuse-CGM	29	0	518	Python (99.7%)	2505.16901
joycode-agent	10	0	317	Python (100.0%)	—
debug-gym	740	5	291	Python (82.9%)	2503.21557
Lingma-SWE-GPT	19	0	253	Python (95.5%)	2411.00622
RepoGraph	19	0	248	Python (83.5%)	2410.14684
HyperAgent	52	0	234	Python (91.9%)	2409.16299
live-swe-agent	13	1	221	—	2511.13646
Lingxi	25	0	207	Python (100.0%)	2510.11838
CodeR	37	0	159	—	2406.01304
SWE-Fixer	13	0	131	Shell (98.1%)	2501.05040
engine-core	12	0	128	TypeScript (95.2%)	—
ExpeRepair	3	0	106	Python (99.6%)	—
SWE-Dev	28	0	57	Python (97.3%)	2506.07636
programmer	185	0	57	Python (70.0%)	—
Agentless-Lite	25	2	45	Python (100.0%)	2407.01489
OrcaLoca	459	0	33	Python (100.0%)	2502.00350
KGCompass	16	0	22	Python (83.7%)	2503.21710
aegis	11	0	9	Python (33.1%)	—
Co-Patcher	2	1	8	Python (100.0%)	2505.18955
CodeSouler	7	0	3	—	—
AnonymousSWE-GPT	2	0	2	Python (95.5%)	—

(continued on next page)

(continued from previous page)

Repository	#Commits	#Releases	#Stars	Language	Publication
DARS-Agent	89	0	2	Python (70.0%)	2503.14269
navie-editor	35	0	2	Python (100.0%)	—
patchwork	1,320	—	2	Python (99.3%)	—
MCTS-Refine-Codes	1	0	1	Python (100.0%)	—

Each subsequent subsection applies stricter eligibility criteria, progressively reducing the repository sample as illustrated in Figure 3.2.

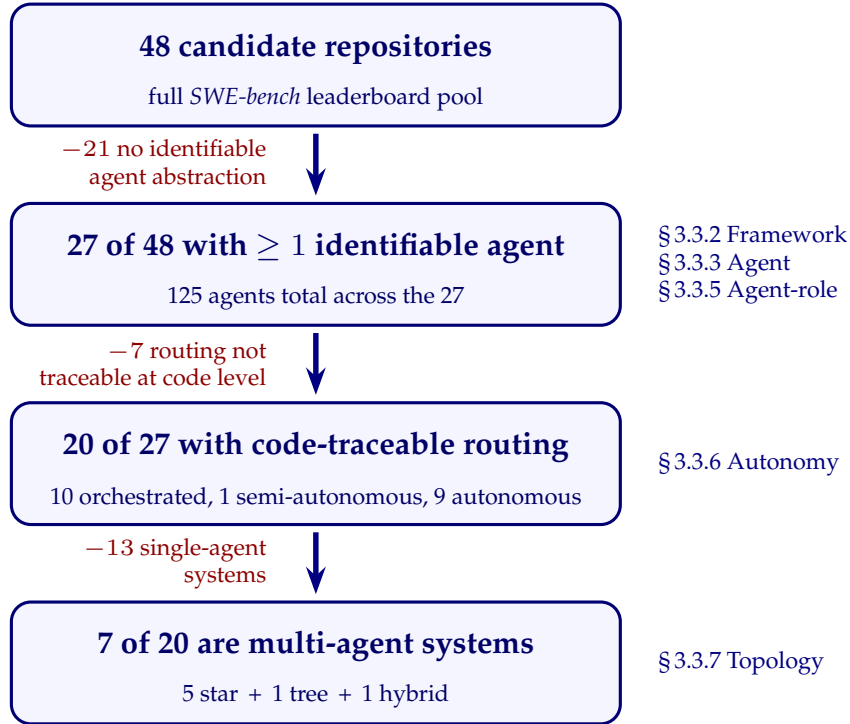


Figure 3.2: Reduction of the candidate repository pool across the four analysis stages of this chapter. Each downward arrow names the filter criterion and reports the resulting drop in sample size. The section labels on the right of each surviving box point to the subsection that uses that subset as its analysis sample.

3.3.2 RQ1: Results of the Framework Analysis

The framework analysis identifies six distinct MAS orchestration libraries across the 48 repositories, with LangChain (7 repositories) and LangGraph (4) the most adopted, followed by LlamaIndex (3), AutoGen and MetaGPT (2 each), and CrewAI (1), as reported in Table 3.2. The same table classifies each repository by its broader infrastructure role, revealing that 20 repositories implement custom or-

chestration scripts rather than relying on an established library, while 13 are themselves general-purpose agent frameworks released as platforms in their own right.

Several of the LLM frameworks introduced in Section 2.4 are well-suited for constructing MAS-based APR pipelines. The following paragraphs report which of those frameworks appear concretely in the open-source repositories analyzed in this study and describe the specific role each plays within the corresponding APR pipeline.

LangChain and LangGraph. In the analyzed pool, Prometheus⁷ uses LangGraph’s StateGraph to route execution across its pipeline of specialized LLM nodes⁸. Composio⁹ illustrates a complementary use case: it extends LangChain’s agent executor with many tool actions that APR agents can invoke without custom integration code.

LlamaIndex. Among the analyzed repositories, moatless-tools [Antoniades et al. 2025]^{10,11} builds its Monte Carlo Tree Search on top of LlamaIndex for repository-level retrieval and file indexing.

CrewAI. Among the analyzed repositories, Composio¹² extends CrewAI’s crew-based multi-agent executor with its library of pre-built tool integrations¹³, enabling role-specialized agents to interact with external services such as version control systems and issue trackers without implementing custom connector code.

AutoGen. Within the analyzed set, HyperAgent¹⁴ uses AutoGen’s GroupChat abstraction to coordinate its five specialized agents: the user_proxy and planner form the conversation hub, and a custom_speaker_selection_func routes execution to the navigator, editor, or executor spoke by matching keyword signals in the planner’s last message¹⁵ [Wu et al. 2023]. Composio¹⁶ illustrates a complementary use case: it augments AutoGen agents with its pre-built

⁷EuniAI/Prometheus

⁸issue_graph.py#L94

⁹ComposioHQ/composio

¹⁰aorwall/moatless-tools

¹¹aorwall/moatless-tree-search

¹²ComposioHQ/composio

¹³crewAIInc/crewAI

¹⁴FSoft-AI4Code/HyperAgent

¹⁵plan_seeking.py#L208

¹⁶ComposioHQ/composio

tool integration library, enabling them to interact with external services without custom connector code.

Table 3.2 aggregates the framework and infrastructure data collected during the dependency-graph and regex scans described in Section 3.2.2. The upper half of the table lists the six MAS orchestration libraries detected in the analyzed repositories; because a single repository may import more than one library, those rows are not mutually exclusive. The lower half classifies each repository by its broader infrastructure role: *custom scripts* covers repositories that implement their own orchestration without any established MAS library; *framework-as-system* identifies repositories whose primary contribution is the agent platform itself; and *not analyzed* marks submissions for which code-level analysis was not possible.

Table 3.2: Framework adoption and infrastructure classification across the 48 candidate repositories. MAS library rows count repositories that import the corresponding package; a repository may appear in more than one row. Infrastructure category rows are not mutually exclusive either, as a single repository can, for example, be a framework-as-system that internally imports a MAS library.

Category	Count
<i>MAS Orchestration Libraries</i>	
LangChain	7
LangGraph	4
LlamaIndex	3
AutoGen	2
MetaGPT	2
CrewAI	1
<i>Infrastructure Category</i>	
Custom orchestration scripts	20
Framework-as-system	13
Plug-in component	1
Not analyzed	7

Answer to RQ₁.

Six MAS orchestration libraries are used to build the LLM-based APR systems analyzed from the *SWE-bench* leaderboard: LangChain, LangGraph, LlamaIndex, AutoGen, MetaGPT, and CrewAI. LangChain and LangGraph emerge as the most commonly adopted libraries, while a sizable share of repositories rely on custom orchestration scripts rather than on any established library.

3.3.3 Results of the Agent Analysis

The agent analysis identifies 125 agents distributed across 27 repositories, with a median of four agents per repository, reported per repository in Table 3.3. The remaining 21 repositories without any identified agent either could not be analyzed at code level or, like `Agentless`, operate through fixed LLM-call pipelines with no discrete agent abstraction.

Two patterns emerge: systems that decompose the repair workflow into fine-grained specialized roles, and systems that consolidate all functionality into a single general-purpose agent. Different repositories also adopt their own notions of what constitutes an agent.

Table 3.3: Number of identified agents per analyzed repository, ordered by descending agent count. The count reflects top-level agents only; sub-components such as LangGraph nodes within a parent agent are not counted as separate agents.

Repository	#Agents
patchwork	15
aider	14
composio	14
OpenHands	8
HyperAgent	7
auto-code-rover	6
agentscope	6
OrcaLoca	6
sunu	6
Lingxi	5
Prometheus ¹⁷	5
debug-gym	4
ExpeRepair	4
Lingma-SWE-GPT	4
programmer	3
SWE-agent	3
aegis	2
DARS-Agent	2
KGCompass	2
mini-SWE-agent	2
augment-swebench-agent	1
claude-coder	1
joycode-agent	1

(continued on next page)

¹⁷Prometheus exposes five top-level agents (Issue Classification, Environment Build, Bug Reproduction, Context Retrieval, and Issue Resolution); each is internally decomposed into multiple LangGraph nodes (26 in total), which are not counted here as separate agents.

(continued from previous page)

Repository	#Agents
moatless-tools	1
R2E-Gym	1
ridges	1
trae-agent	1

3.3.4 RQ2: Results of the Prompt Analysis

The prompt analysis covers the 166 prompts extracted from the 21 repositories that expose at least one analyzable prompt and detects all eight stylistic patterns introduced in Section 3.2.4, with frequencies summarized in Table 3.12. Directive words are the most common pattern (60.2% of prompts), followed by uppercase emphasis (50.0%) and output-format prescriptions (44.0%); negation, markup tags, and numbered lists each appear in roughly a third of the prompts, while punctuation conventions are the rarest (26.5%). The per-repository tables in this subsection show how these patterns concentrate in a small number of prompt-heavy systems such as `aider`.

A single prompt may contain multiple occurrences of the same or of different patterns, and one agent may have more than one prompt, even of the same type.

Uppercase emphasis. Critical rules are written in fully uppercase words to force the model’s attention. The per-repository counts are reported in Table 3.4. The `agent.locate_file_class_func` of `Lingma-SWE-GPT`¹⁸ uses this pattern to impose a hard quantitative constraint on the model’s output:

```
OUTPUT AT MOST 10 LOCATIONS that need to be modified in their
workspace and sort them by modification priority.
```

Directive word patterns. Obligations are made explicit with modal verbs such as *should*, *must*, *always*, and *ensure*. The per-repository counts are reported in Table 3.5. The `general_agent` of `augment-swebench-agent`¹⁹ uses *should* to make testing a required step:

```
You should run relevant tests to verify that your changes work.
```

¹⁸`agent.locate_file_class_func.py#L7`

¹⁹`system.prompt.py#L3`

Table 3.4: Repositories in which the **uppercase emphasis** pattern was detected. The #pattern column reports the count of the pattern found in all agent prompts of a repository (83 prompts across 18 repositories).

Repository	#pattern	Repository	#pattern
aider	23	KGCompass	1
augment-swebench-agent	1	Lingma-SWE-GPT	3
auto-code-rover	9	Lingxi	2
claude-coder	3	OpenHands	3
composio	4	OrcaLoca	4
DARS-Agent	1	patchwork	10
debug-gym	1	R2E-Gym	3
ExpeRepair	6	SWE-agent	6
HyperAgent	2	trae-agent	1

The TestAgent of ExpeRepair²⁰ uses *must* to enforce a structural constraint on the generated script:

```
The script must be self-contained.
```

The planner of HyperAgent²¹ uses *should* to prevent the model from repeating prior reasoning steps:

```
Each consequent steps should be strictly based on the previous steps.
```

Table 3.5: Repositories in which the **directive words** pattern was detected. The #pattern column reports the count of the pattern found in all agent prompts of a repository (100 prompts across 20 repositories).

Repository	#pattern	Repository	#pattern
aider	22	KGCompass	1
augment-swebench-agent	1	Lingma-SWE-GPT	4
auto-code-rover	8	Lingxi	5
claude-coder	3	OpenHands	3
composio	2	OrcaLoca	6
DARS-Agent	1	patchwork	13
debug-gym	1	programmer	3
ExpeRepair	9	R2E-Gym	3
HyperAgent	4	SWE-agent	9
joycode-agent	1	trae-agent	1

²⁰agent_reproducer.py#L23

²¹planner.py#L1

Punctuation conventions. Rhetorical questions trigger self-evaluation, while exclamation marks reinforce urgency. The per-repository counts are reported in Table 3.6. The `Proxy` agent of `auto-code-rover`²² uses question marks to prompt the model to assess completeness:

```
do we need more context?
where are bug locations?
```

The navigator of `HyperAgent`²³ uses an exclamation mark to mark a rule as critical:

```
Do not repeat your actions (IMPORTANT!)
```

Table 3.6: Repositories in which the **punctuation** pattern was detected. The `#pattern` column reports the count of the pattern found in all agent prompts of a repository (44 prompts across 13 repositories).

Repository	#pattern	Repository	#pattern
aider	18	Lingma-SWE-GPT	2
auto-code-rover	4	Lingxi	1
composio	2	mini-SWE-agent	1
DARS-Agent	1	patchwork	1
debug-gym	1	R2E-Gym	3
HyperAgent	3	SWE-agent	6
KGCompass	1		

Markup tags. XML-style tags delimit input sections or prescribe the structure of the model’s output. The per-repository counts are reported in Table 3.7. The `problem_solver_agent` of `Lingxi`²⁴ uses tags to prescribe both the plan and observation:

```
Please edit the file in the codebase according to the following
code change plan wrapped in the <code_change_plan> tags.
```

```
Wrap your observation and next step reasoning in the
<observation> tags.
```

²²`agent_proxy.py#L16`

²³`navigator.py#L2`

²⁴`problem_solver.py#L2`

The ShellAgent of SWE-agent²⁵ wraps its entire role context in a `<setting>` tag:

```
<setting>You are an expert software engineer overseeing junior
developers. They suggest actions to take to solve a problem.
You must choose the best action to take.</setting>
```

Table 3.7: Repositories in which the **markup tags** pattern was detected. The `#pattern` column reports the count of the pattern found in all agent prompts of a repository (56 prompts across 14 repositories).

Repository	#pattern	Repository	#pattern
aider	14	Lingma-SWE-GPT	2
auto-code-rover	3	Lingxi	1
claude-coder	3	OpenHands	2
DARS-Agent	1	OrcaLoca	3
debug-gym	1	patchwork	6
ExpeRepair	7	R2E-Gym	3
HyperAgent	1	SWE-agent	9

Role definition. Many prompts open with a persona sentence that anchors the model to a specific identity. The per-repository counts are reported in Table 3.8. The navigator of HyperAgent²⁶ establishes its role as:

```
You are an expert in finding all relevant information inside a
code repository to answer the query from a planner agent.
```

Table 3.8: Repositories in which the **role definition** pattern was detected. The `#pattern` column reports the count of the pattern found in all agent prompts of a repository (65 prompts across 15 repositories).

Repository	#pattern	Repository	#pattern
aider	10	Lingma-SWE-GPT	4
augment-swebench-agent	1	Lingxi	5
auto-code-rover	6	OrcaLoca	4
composio	8	patchwork	12
DARS-Agent	1	R2E-Gym	2
debug-gym	1	SWE-agent	3
ExpeRepair	3	trae-agent	1
HyperAgent	4		

²⁵action_sampler.py#L105

²⁶navigator.py#L2

Negation. Explicit prohibitions prevent common failure modes by specifying what the model must not do. The per-repository counts are reported in Table 3.9. The navigator of HyperAgent²⁷ includes:

```
Do not care about any Pull Request or Existing Issue in the repository.
```

Table 3.9: Repositories in which the **negation** pattern was detected. The #pattern column reports the count of the pattern found in all agent prompts of a repository (55 prompts across 16 repositories).

Repository	#pattern	Repository	#pattern
aider	14	joycode-agent	1
augment-swebench-agent	1	Lingma-SWE-GPT	3
auto-code-rover	4	Lingxi	2
claude-coder	2	OpenHands	2
composio	1	OrcaLoca	3
DARS-Agent	1	patchwork	5
ExpeRepair	9	R2E-Gym	3
HyperAgent	3	trae-agent	1

Output format. Prompts frequently prescribe the exact structure of the model’s response to reduce parsing errors in downstream agents. The per-repository counts are reported in Table 3.10. The navigator of HyperAgent²⁸ specifies:

```
When you write Python code for action, put the code in a Markdown code block with the language set to Python.
```

Numbered lists. Instructions are enumerated to impose an explicit execution order on the model. The per-repository counts are reported in Table 3.11. The navigator of HyperAgent²⁹ opens its workflow with:

1. Understanding the query, think step-by-step carefully before decision to propose actions to collect necessary information to resolve the query.
2. Do not repeat your actions. Only generate 1 block of code at one time.

²⁷navigator.py#L2

²⁸navigator.py#L2

²⁹navigator.py#L2

Table 3.10: Repositories in which the **output format** pattern was detected. The #pattern column reports the count of the pattern found in all agent prompts of a repository (73 prompts across 17 repositories).

Repository	#pattern	Repository	#pattern
aider	20	Lingma-SWE-GPT	4
auto-code-rover	2	Lingxi	2
claude-coder	3	OpenHands	1
composio	1	OrcaLoca	4
DARS-Agent	1	patchwork	11
debug-gym	1	R2E-Gym	2
ExpeRepair	9	SWE-agent	6
HyperAgent	4	trae-agent	1
KGCompass	1		

Table 3.11: Repositories in which the **numbered list** pattern was detected. The #pattern column reports the count of the pattern found in all agent prompts of a repository (48 prompts across 14 repositories).

Repository	#pattern	Repository	#pattern
aider	11	Lingma-SWE-GPT	2
auto-code-rover	1	Lingxi	3
claude-coder	3	OpenHands	2
DARS-Agent	1	OrcaLoca	3
ExpeRepair	9	patchwork	4
HyperAgent	4	R2E-Gym	3
KGCompass	1	trae-agent	1

Table 3.12 ranks all eight patterns by their aggregate occurrence across the full population of 166 prompts (one per agent-prompt pair extracted from the 21 repositories that expose at least one analyzable prompt), providing a cross-repository frequency baseline for each strategy.

Table 3.12: Aggregate frequency of each prompt stylistic pattern across all 166 prompts analyzed, listed in the same order as the pattern subsections. Percentage computed relative to the total prompt count.

Pattern	#Prompts	%
Uppercase emphasis	83	50.0
Directive words	100	60.2
Punctuation	44	26.5
Markup tags	56	33.7
Role definition	65	39.2
Negation	55	33.1
Output format	73	44.0
Numbered list	48	28.9

Answer to **RQ₂**.

Across the prompts collected from LLM-based APR agents on the *SWE-bench* leaderboard, eight recurring stylistic patterns are observed: directive words, uppercase emphasis, output-format prescriptions, role definition, markup tags, negation, numbered lists, and punctuation conventions. Each pattern influences model behavior through a distinct mechanism: directive words and uppercase emphasis draw the model’s attention to mandatory rules; output-format and markup tags constrain the structure of generated text so that downstream agents can parse it deterministically; role definition stabilizes the persona across turns; negation prohibits known failure modes; numbered lists impose an explicit execution order on multi-step instructions; and punctuation conventions, in particular exclamation marks and rhetorical questions, signal urgency and prompt self-evaluation.

3.3.5 RQ3: Results of the Agent Roles Analysis

The agent roles analysis classifies each of the 125 identified agents into one of eight functional roles, summarized in Table 3.13. The Patcher role is the most frequent, present in 17 of the 27 repositories with at least one identifiable agent (63.0%), followed by Base Agent (51.9%) and Locator (48.1%); the more specialized Bug Reproduction role appears in only four repositories (14.8%).

Agents across the analyzed repositories are manually classified into functional roles³⁰. The classification spans eight recurring roles, each capturing a distinct responsibility within the APR pipeline.

³⁰TFG_public/Open-Source-Proyects-SWE-Bench - AgentType.csv

Table 3.13: Agent role distribution across the 27 analyzed repositories with at least one identifiable agent. Each row reports the number of repositories in which at least one agent fulfills the corresponding role. Percentage computed relative to the 27 repository base. Sorted in descending order of prevalence.

Role	#Repositories	%
Patcher	17	63.0
Base Agent	14	51.9
Locator	13	48.1
Helper/Proxy	10	37.0
Reviewer	10	37.0
Planner/Orchestrator	7	25.9
Environment Setup	7	25.9
Bug Reproduction	4	14.8

Locator. The locator identifies the probable defect within the repository given a failing test or a natural-language issue description, implementing the **fault localization** sub-task. In HyperAgent, the navigator³¹ is explicitly scoped to this responsibility through its system prompt³²:

```
You are an expert in finding all relevant information inside a
code repository to answer the query from a planner agent.
```

Patcher. The patcher proposes one or more candidate fixes for the localized defect, performing **patch generation**. In joycode-agent, the PatchEngine³³ frames this responsibility through its system prompt³⁴:

```
You serve as an intelligent coding assistant, collaborating
with software developers to execute pull request
implementations, equipped with comprehensive tools for codebase
interaction and manipulation.
```

Planner/Orchestrator. The orchestrator decomposes the high-level repair goal into an ordered sequence of sub-tasks and coordinates which specialist agents handle each step. In HyperAgent, the planner³⁵ takes on this responsibility, as es-

³¹plan.seeking.py#L12

³²navigator.py#L2

³³patch.agent.py#L355

³⁴system.prompt.py#L3

³⁵plan.seeking.py#L194

tablished by its system prompt³⁶:

```
You are a great developer with expertise in resolving general
software engineering task. You have been assigned a task to
do in a large repository. Devise a detailed plan to delegate
tasks other interns to resolve the ultimate goal.
```

Helper/Proxy. Helper agents are a broad category covering any agent that does not directly perform one of the core APR tasks: fault localization, patch generation, or patch review. This includes agents that translate or relay information between pipeline stages, agents that manage intermediate state, and agents that perform supporting analysis not tied to a specific repair sub-task. In Lingxi, the `solution_mapper_agent`³⁷ is a representative example: it bridges the fault localization output to the patch generation step by mapping the identified problem to a concrete set of files and change locations, as defined in its system prompt³⁸:

```
You are a solution mapper. You should do the follow two tasks:
1. According to the result from problem decoder, your task
is to browse the relevant files and functions related with the
problem. To do this, you should first understand the files and
their semantics, and then understand the proposed solution.
2. And then, from the current status and the expected status,
reasoning and generate the detailed code change plan for each
relevant files/functions.
```

Bug Reproduction. This role generates a reproducible, self-contained test case that reliably triggers the reported defect, providing an automated oracle for subsequent **patch validation**. In ExpeRepair, the `TestAgent`³⁹ enforces this property through an explicit constraint in its system prompt⁴⁰:

```
You are an experienced software engineer responsible for
writing a test script for an issue reported in your GitHub
project {repo_name}.
Do NOT implement any fixes, you are ONLY interested in writing
a test script.
```

³⁶planner.py#L1

³⁷supervisor_graph_demo.py#L202

³⁸solution_mapper.py#L3

³⁹agent_reproducer.py#L179

⁴⁰agent_reproducer.py#L23

Environment Setup. Before any test can be executed, the target repository must be installed and its dependencies resolved. Prometheus isolates this concern into a dedicated `Environment Build Agent`⁴¹, which automatically detects the project type, installs dependencies, configures build tools, and validates the resulting environment.

Reviewer. The reviewer validates candidate patches by checking correctness and potential regressions before submission. In Lingxi, the `reviewer_agent`⁴² performs a joint assessment of the patch and the solution plan, as established by its system prompt⁴³:

```
You are a reviewer. You are provided the latest code changes
generated by the problem_solver.
```

Base Agent. Some systems expose a general-purpose agent that acquires its functional role entirely through prompting, with no hard-coded specialization at the implementation level. In OpenHands, the `CodeActAgent`⁴⁴ serves as the primary runtime agent, capable of executing commands, editing files, and running tests within a single persistent context. Its general identity is established through its system prompt⁴⁵:

```
You are OpenHands agent, a helpful AI assistant that can
interact with a computer to solve tasks.
```

⁴¹`Multi-Agent-Architecture.md#L22`

⁴²`hierarchy_graph_demo.py#L88`

⁴³`reviewer.py#L2`

⁴⁴`codeact_agent.py#L57`

⁴⁵`system_prompt.j2#L1`

Answer to RQ₃.

The set of roles necessary to solve an APR task in a MAS reduces to the three core sub-tasks of the repair pipeline: *Locator* for fault localization, *Patcher* for patch generation, and *Reviewer* for patch validation; the remaining five roles (Planner/Orchestrator, Helper/Proxy, Base Agent, Environment Setup, and Bug Reproduction) are architectural choices rather than strict requirements. The empirical ranking does not exactly mirror this necessary set: Patcher and Locator track the necessity claim and appear most consistently across the analyzed repositories, but Reviewer is frequently absorbed into the Patcher loop or replaced by automated test execution, and Bug Reproduction is rarely isolated as a dedicated agent.

3.3.6 RQ4: Results of the Autonomy Analysis

The autonomy analysis assigns each of the 20 repositories with full code-level traceability to one of three autonomy levels, summarized in Table 3.14. Ten systems are fully orchestrated and nine are fully autonomous, while only one sits at the intermediate semi-autonomous level. This polarization indicates that, in practice, design teams tend to commit either to a deterministic software pipeline or to an unconstrained agent loop, rather than to mixed designs that grant agents a constrained routing decision.

Orchestrated. Prometheus⁴⁶ exemplifies this level: its LangGraph StateGraph⁴⁷ routes execution through a fixed conditional pipeline in which the next node is determined by evaluating `state["issue_type"]` against a static mapping, not by any LLM call. HyperAgent⁴⁸ similarly uses a `custom_speaker_selection_func`⁴⁹ that routes to Navigator, Editor, or Executor by matching keywords in the planner's last message, a deterministic rule defined in code rather than through agent reasoning. ExpeRepair⁵⁰ follows an equally deterministic sequence: bug reproduction, patch generation, and validation are each driven by a dedicated inference script invoked in a fixed order⁵¹, with no agent deciding what comes next. joycode-agent⁵² adopts the same pattern: environment setup, test generation,

⁴⁶EuniAI/Prometheus⁴⁷issue_graph.py#L94⁴⁸FSoft-AI4Code/HyperAgent⁴⁹plan_seeking.py#L208⁵⁰ExpeRepair/ExpeRepair⁵¹reproduce.initial.py#L44⁵²jd-opensource/joycode-agent

patch generation, and validation are executed in a fixed sequence by `run_patch_pipeline.py`⁵³.

Semi-autonomous. Lingxi⁵⁴ is the sole representative of this level: an LLM-based supervisor decides which specialist agent to activate next, but is restricted to a fixed set of options (`problem_decoder`, `solution_mapper`, `problem_solver`, or `finish`)⁵⁵.

Autonomous. OpenHands⁵⁶ is the representative example in the analyzed set: its `CodeActAgent` maintains its own action loop⁵⁷, selecting bash commands, file edits, or web searches and emitting a finish signal entirely through LLM inference, with the event controller acting only as I/O plumbing rather than an orchestrator.

Table 3.14: Autonomy level distribution across the 20 repositories for which full code-level autonomy classification was possible.

Autonomy Level	#Repositories
Orchestrated	10
Semi-autonomous	1
Autonomous	9
Total	20

Answer to RQ₄.

LLM-based APR agents are only partially autonomous in practice. The autonomy distribution is bimodal: design teams commit either to a fully orchestrated pipeline, where the surrounding software sequences agent invocations through hardcoded rules, or to a fully autonomous loop, where the agent itself drives every routing decision through LLM inference; mixed designs that grant agents a constrained routing decision are the exception. LLMs remain too unpredictable to be trusted with end-to-end control under tight context budgets, which is why a comparable share of systems still rely on deterministic orchestration.

⁵³`run_patch_pipeline.py#L140`

⁵⁴`nimasteryang/Lingxi`

⁵⁵`supervisor_graph_demo.py#L93`

⁵⁶`All-Hands-AI/OpenHands`

⁵⁷`codeact_agent.py#L169`

3.3.7 RQ5: Results of the Topology Analysis

The topology analysis classifies seven multi-agent systems into the topology categories defined in Section 2.2.1, summarized in Table 3.15. Five systems (OpenHands, HyperAgent, Lingxi, ExpeRepair, joycode-agent) realize a centralized star topology, one (Prometheus) is hierarchical, and one (agentscope) is hybrid; pure decentralized, distributed, or dynamic topologies do not appear in the analyzed set. Agentless pipelines, single-agent systems, fine-tuned models, and plug-in components are omitted from this classification.

Table 3.15: Topology classification of the multi-agent *SWE-bench* leaderboard repositories in the analyzed set.

Repository	Topology
OpenHands	Centralized (Star)
HyperAgent	Centralized (Star)
Lingxi	Centralized (Star)
ExpeRepair	Centralized (Star)
joycode-agent	Centralized (Star)
Prometheus	Hierarchical (Tree)
agentscope	Hybrid

The dominance of the star pattern is consistent with the practical constraints of competitive benchmark submissions. Decentralized coordination and dynamic graph management impose overhead that conflicts with the latency and token-cost requirements of the *SWE-bench* evaluation harness.

Answer to RQ₅.

Three topology classes are observed in *SWE-bench* APR solutions: centralized (star), hierarchical (tree), and hybrid. No pure decentralized, distributed, or dynamic topology appears. The star topology offers the lowest implementation cost and the most predictable token budget, but concentrates all routing decisions in a single hub that becomes a single point of failure in the reasoning chain. The hierarchical (tree) topology supports stronger task decomposition and reduces hub overload, at the cost of additional coordination latency and the risk of error propagation through mid-level managers. The hybrid topology combines a star hub with a linear subchain and bidirectional back-edges from worker to hub, enabling local iteration cycles at the cost of substantially higher routing complexity. The absence of decentralized and dynamic designs reflects the coordination overhead they impose under competitive benchmark conditions. The trade-offs reported for each topology class are presented as observed alignments with the broader topology survey in Section 2.2.1, since the analyzed sample of seven multi-agent systems is too small to support class-level conclusions on its own.

3.3.8 Results of the Graph Visualization

Applying the `repo_graph` ETL pipeline described in Section 3.2.8 to the master repository table and per-agent CSVs yields a single GEXF artifact that consolidates the empirical evidence collected across the preceding subsections into one inspectable graph. Loading this artifact into Gephi Lite [Gephi Consortium 2024] produces the renderings shown in the figures that follow. The figures shown here are illustrative snapshots of the interactive layouts; the underlying GEXF file and the export script are released as part of the project public companion repository⁵⁸, and the same graph is also deployed as a fully client-side interactive site⁵⁹ that accepts the optional URL parameter `?repo=<repo_id>` to restrict the view to a single repository subgraph. The complete list of deployment URLs, covering the full graph and a per-repository subgraph link for each analyzed repository, is collected in the `DEPLOYMENTS.md` file.⁶⁰

These renderings are collected in Figure 3.3 (the full repository graph) together with three representative repository subgraphs: Figure 3.4, Figure 3.5, and Figure 3.6.

⁵⁸[TFG_public/First_Part_Analysis/repo_graph-master](https://github.com/TFG_public/First_Part_Analysis/repo_graph-master)

⁵⁹xiangfengye.github.io/TFG_public/repo-graph/

⁶⁰[repo_graph-master/DEPLOYMENTS.md](https://github.com/TFG_public/First_Part_Analysis/repo_graph-master/blob/main/DEPLOYMENTS.md)

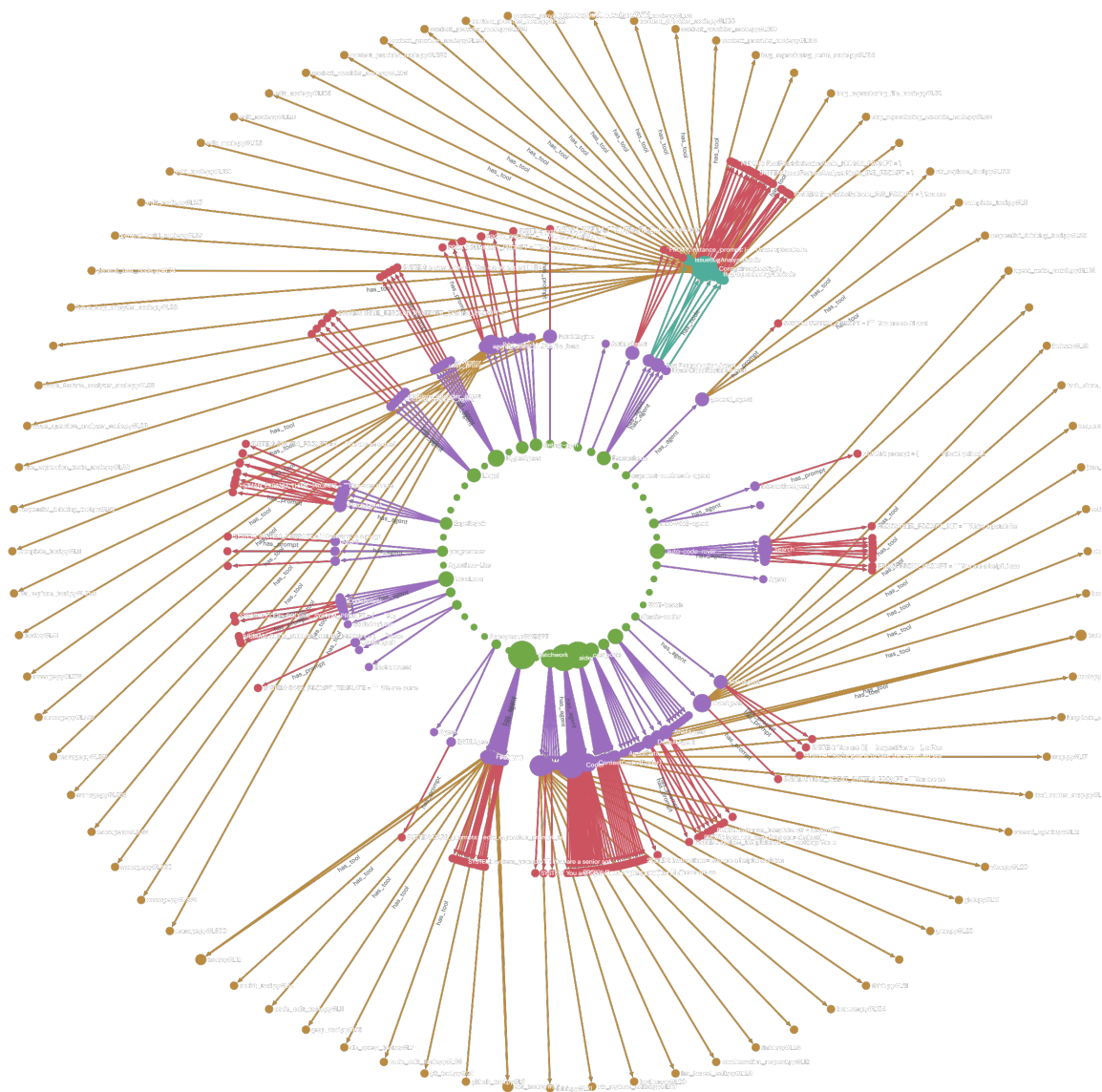


Figure 3.3: Radial layout of the complete GEXF graph rendered in Gephi Lite. *Node colors*: repository (green), agent (purple), agent_node (cyan), prompt (red), tool (orange). The interactive version for a full visualization of the labels is deployed at: xiangfengyepan.github.io/TFG_public/repo-graph/.

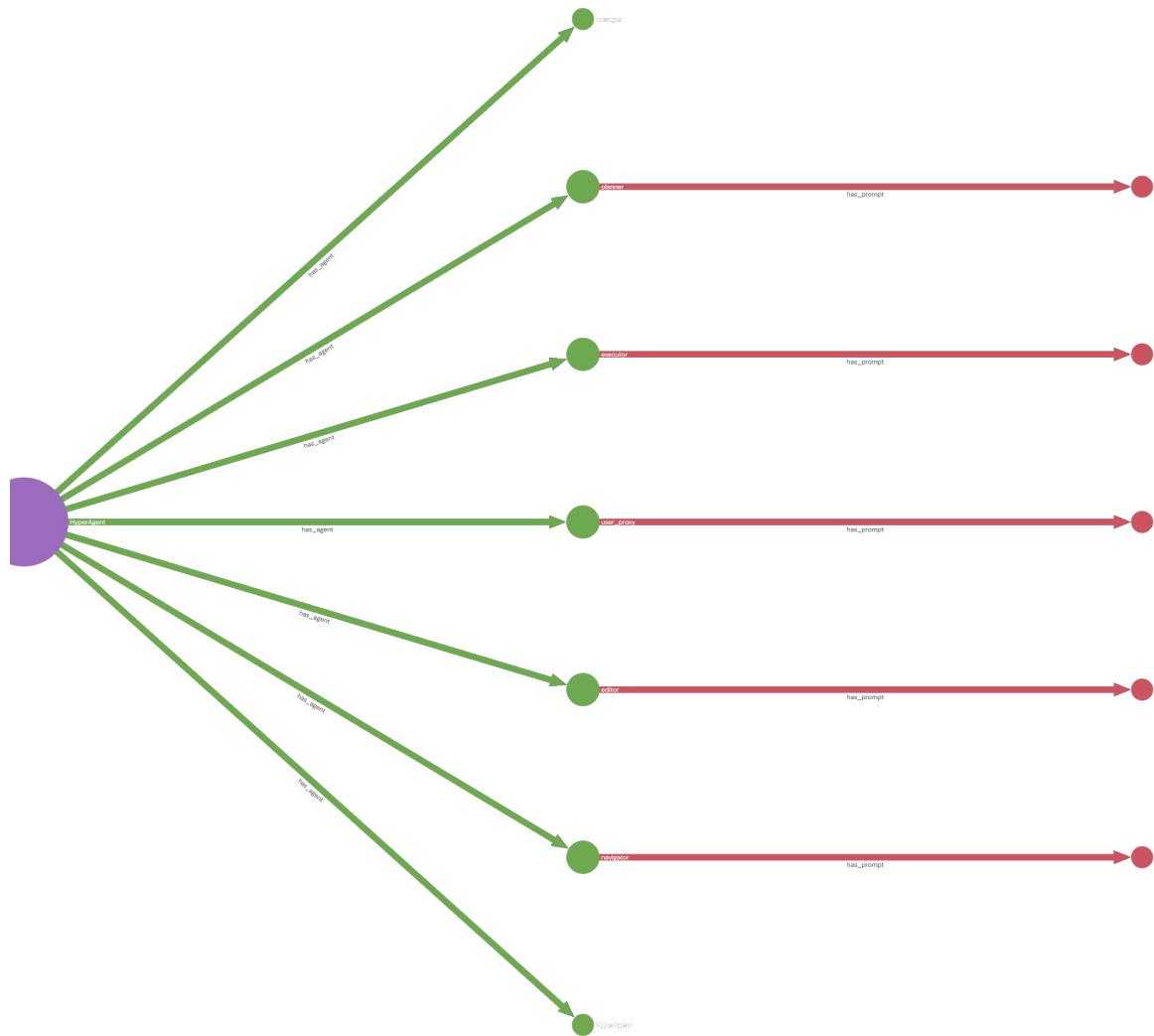


Figure 3.4: Local subgraph of HyperAgent. *Node colors:* repository (purple), agent (green), prompt (red). The interactive version for a full visualization of the labels is deployed at: xiangfengyepan.github.io/TFG_public/repo-graph/?repo=repo_HyperAgent.

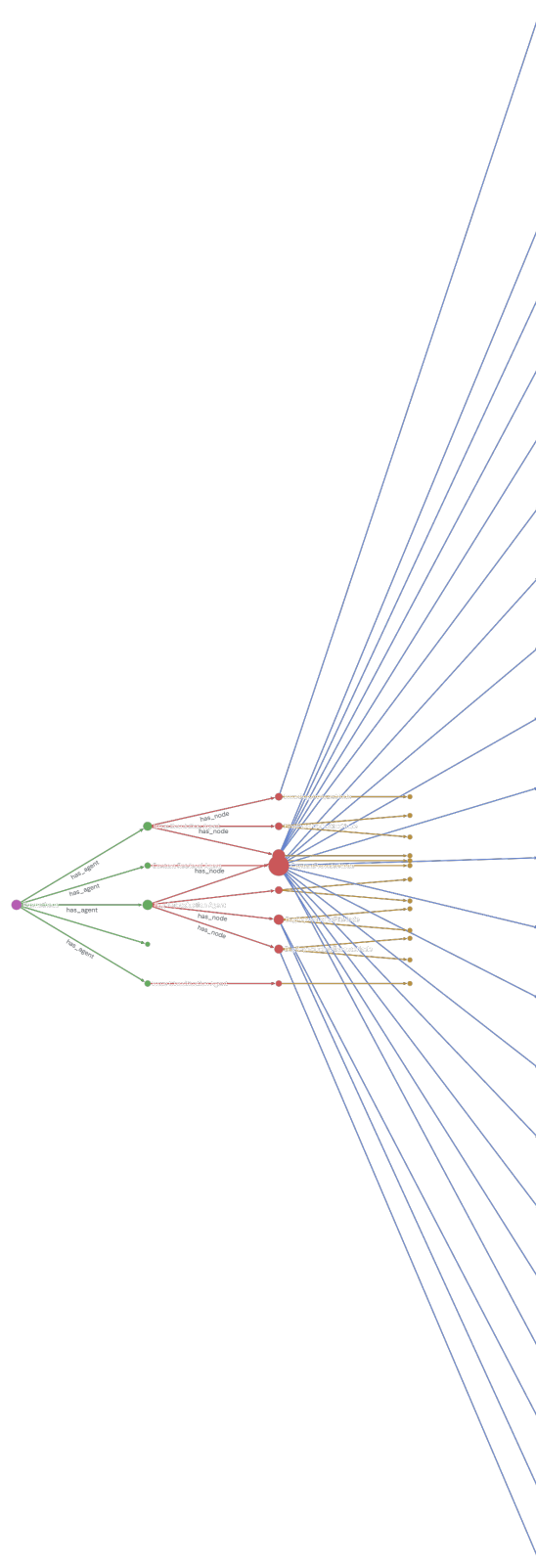


Figure 3.5: Local subgraph of Prometheus. *Node colors:* repository (purple), agent (green), agent_node (red), prompt (orange), tool (blue). The interactive version for a full visualization of the labels is deployed at: xiangfengyepan.github.io/TFG_public/repo-graph/?repo=repo_Prometheus.

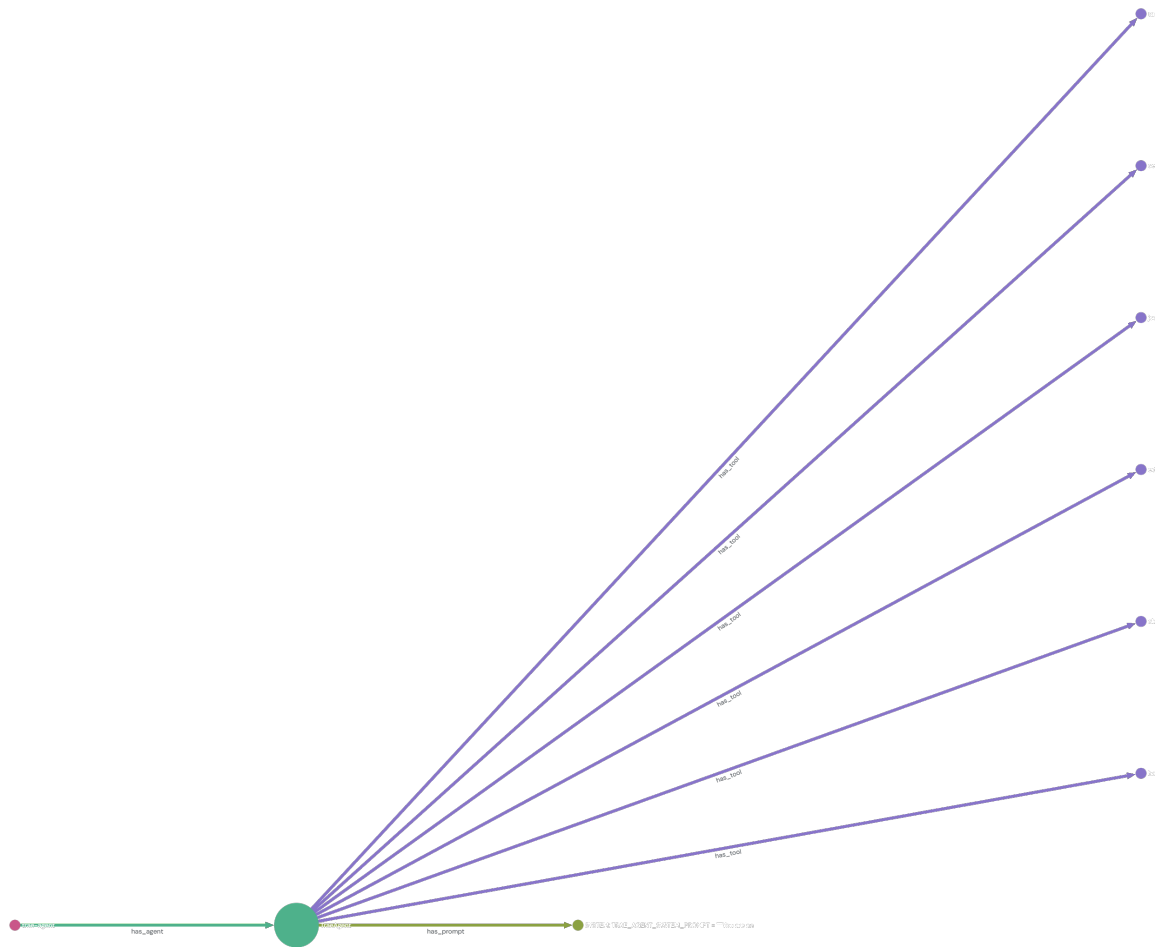


Figure 3.6: Local subgraph of `trae-agent`. *Node colors:* repository (red), agent (green), prompt (yellow), tool (purple). The interactive version for a full visualization of the labels is deployed at: xiangfengyepan.github.io/TFG_public/repo-graph/?repo=repo_trae_agent.

Chapter 4

Specification and Design of the Solution

4.1 Methodology

4.1.1 Development Process

The development of this project follows an Agile methodology, specifically inspired by the Scrum framework, as defined in the initial GEP deliverables [Sabaté 2023]. The project is structured into short sprints of approximately one week, each focused on an objective agreed during the weekly review meeting with the supervisors. The roles of Product Owner and Scrum Master are assumed jointly by the student and the supervisors, while the development team consists of the student alone.

This project is divided into two major phases, empirical analysis and framework construction. An early framework prototype¹ was developed in parallel with the repository analysis to validate the feasibility of the design assumptions on a small local LLM. Once the probe succeeded, framework construction was paused so that the empirical analysis could be completed in full, and resumed only afterwards. This pause-and-resume pattern is reflected in the revised Gantt chart of Figure B.1.

4.1.2 Monitoring Tools

Several monitoring tools are used to keep tasks visible and progress traceable. Task management relies on Trello, organized as a Kanban-style board with the states

¹[TFG-public/framework-master](https://github.com/TFG-public/framework-master)

TO DO, *DOING*, and *DONE*, following the agile-project-management practice described in the GEP course material [Sabaté 2023]. Version control and code archival use Git, with GitHub as the remote host for both the *EvoMas* framework and the empirical-analysis artifacts. Long-term scheduling and timeline management are supported by a Gantt chart produced with the `gantt` package, which is updated at each milestone (Figure B.1).

Communication with the supervisors is sustained through weekly synchronous meetings and through asynchronous progress reports sent by email after each sprint.

4.2 System Requirements

This section specifies the requirements of the *EvoMas* framework, the software solution produced in this thesis. These requirements were derived from the empirical analysis of the *SWE-bench* leaderboard repositories presented in Chapter 3 and refined through the weekly review meetings with the supervisors. The requirements are scoped exclusively to the framework’s APR capabilities and its architectural design goals. Each requirement is accompanied by a *satisfaction criterion* that defines the condition under which the requirement is considered met.

4.2.1 Functional Requirements

Functional requirements specify the behaviors the *EvoMas* framework must exhibit during a repair run.

Bug Ingestion. The framework must accept a GitHub issue description and the corresponding source repository as input, and make the issue context available to all agents.

Satisfaction criterion: Given a valid issue description and a local repository path, the system extracts the issue text, identifies the repository file structure, and delivers this context to the first agent without manual intervention.

Fault Localization. The framework must automatically identify the source files and code regions most likely responsible for the reported issue.

Satisfaction criterion: The localization agent returns a ranked list of candidate locations for the fault.

Patch Generation. The framework must produce a concrete code patch addressing the localized fault candidates.

Satisfaction criterion: The generated patch applies cleanly to the repository via `git apply` and is syntactically valid according to the Python interpreter.

Patch Validation. The framework must verify that a generated patch resolves the reported issue without introducing regressions in the existing test suite.

Satisfaction criterion: The validation agent executes the repository's tests after applying the patch and correctly reports the pass or fail outcome for each evaluated instance.

Workflow Construction. The repair pipeline must be expressed as a directed, typed workflow using LangGraph's `StateGraph`, connecting agents through labeled edges that carry a shared typed state.

Satisfaction criterion: The complete pipeline executes end-to-end from bug ingestion to patch validation on at least one *SWE-bench* instance without manual intervention between stages.

Flexible Topology Configuration. The agent topology must be reconfigurable without altering the implementation of any individual agent.

Satisfaction criterion: At least two structurally distinct topologies are deployable from the same codebase by changing only the topology configuration.

Flexible Agent Management. Individual agents must be replaceable, addable, or removable from the pipeline without modifying other agents or the orchestration layer.

Satisfaction criterion: Swapping one agent implementation for another requires changes only to that agent's class and its configuration entry, with no edits to any other agent or to the orchestration layer.

4.2.2 Non-Functional Requirements

Non-functional requirements define quality attributes that constrain how the framework must be built and operated.

Modularity. Each agent must be encapsulated in an independent Python module with a clearly defined interface, so that components can be developed, tested, and replaced in isolation.

Satisfaction criterion: No agent module imports code directly from another agent module; all inter-agent communication passes exclusively through the shared `StateGraph` state object.

Scalability. Introducing a new agent into the pipeline must require no changes to existing agents or to the framework core.

Satisfaction criterion: A new agent is integrated by registering it and adding it to the workflow graph. No pre-existing agent or orchestration file is modified.

Performance. The framework must demonstrate practical effectiveness on real-world APR tasks by achieving a measurable resolved rate on the *SWE-bench* harness.

Satisfaction criterion: At least one topology configuration achieves a non-zero resolved rate on a representative *SWE-bench* Lite subset evaluated under the local harness. Leaderboard submission with a frontier-class LLM is left to future work (see Section 8.3).

Usability. The agent topology and all agent parameters must be fully specifiable through a configuration file, without requiring edits to core framework source code.

Satisfaction criterion: A new topology is deployable by a user with no prior knowledge of the framework internals, using only the configuration file and the documentation.

Budget. The framework must operate with a locally hosted LLM, so that there are no commercial LLM API costs.

Satisfaction criterion: Every component of the framework is available under an open-source license and runs without any internal costs.

Privacy and Security. All LLM inference during local development and testing must be performed in a way that prevents source code and prompt data from being transmitted to external servers.

Satisfaction criterion: Local experimental runs use Ollama LLM exclusively, no repository content or agent prompt is sent to any remote endpoint during local execution.

Hardware Compatibility. The framework must execute correctly on the local machine without requiring dedicated GPU acceleration or any cloud computing services.

Satisfaction criterion: The complete repair pipeline for at least one *SWE-bench* instance completes successfully on the specified hardware using a locally hosted LLM via Ollama.

4.3 System Specification

4.3.1 Use Case Diagram

Figure C.1 (Appendix C) presents the use case diagram of the *EvoMas* framework, capturing the principal actors and the functional scope of the system within the objectives defined in Section 1.3.1.

4.3.2 Conceptual Schema

Figure C.2 (Appendix C) captures the principal model domain entities of the *EvoMas* framework and their relationships.

4.3.3 Data Persistence

The framework avoids databases and persists all the data directly on the local file system. The choice is justified by three of the non-functional requirements introduced in Section 4.2.2.

Modularity. Each entity is a self-contained JSON file. No database schema migrations are needed when the framework evolves, and any entity can be inspected or hand-edited with a plain text editor.

Privacy and Security. Because the whole framework runs on the local machine (Section 4.5.3), there is no need to host, authenticate against, or secure a managed database service.

Usability. In a research framework whose primary workflow is launching, inspecting, and comparing experiments, plain JSON configuration files and JSONL log files are easier to author, diff, and share than database rows. The researcher can edit a topology in any text editor, attach a run directory to an

email, and reproduce a result on a different machine without provisioning or restoring a database.

A managed database would have offered no gain and would have added complexity that the single-user research workflow does not need.

4.3.4 Taxonomy of MAS Designs and Analysis Dimensions

Figure B.2 in Appendix B presents a nine-branch taxonomy covering the complete design space of the empirical analysis and the framework development. All branches are covered across Chapter 2 and Chapter 3. Together they define the feature space used in this thesis.

4.4 System Design

This section presents the static and dynamic views of the *EvoMas* framework through three subsections. Section 4.4.1 and Section 4.4.2 cover the static structure of the Python backend and the Angular frontend respectively. Each sub-system is presented as a level-1 package diagram that gives the overall structure, then broken down into level-2 diagrams that document each package in detail. Section 4.4.3 then covers the dynamic behavior of the framework through sequence and activity diagrams. All diagrams are in Appendix C.

4.4.1 Backend

The Python backend is organized as nine top-level packages whose dependency structure is summarized in Figure C.3. Three level-2 sub-systems are documented separately: the agent hierarchy (Figure C.4), the FastAPI application (Figure C.5), and the MCP tool layer (Figure C.6).

4.4.2 Frontend

The Angular single-page application is organized as thirteen packages spread across four layers (Figure C.7). Four page components are documented separately: the Topology editor (Figure C.8), the Inference run-control (Figure C.9), the Evaluation verdict surface (Figure C.10), and the Results audit surface (Figure C.11).

4.4.3 Behavioral Views

The dynamic behavior of the framework is captured by two sequence diagrams (the Run Inference flow in Figure C.12 and the Run Evaluation flow in Figure C.13) and two activity diagrams (the backend repair pipeline in Figure C.14 and the evaluation pipeline in Figure C.15).

4.5 Architectural Design

This section covers the design ideas adopted from the empirical analysis, the analysis of alternatives, and the physical architecture.

4.5.1 Design Ideas Adopted from the Empirical Analysis

The architectural design of the *EvoMas* framework synthesizes a set of recurring patterns observed in the leaderboard repositories analyzed in Chapter 3. Table 4.1 lists the concrete ideas that have been transposed into the framework, grouped by architectural concern, with the source repositories from which each one was drawn.

Table 4.1: Design ideas adopted from the empirical analysis (Chapter 3), grouped by architectural concern. Every row corresponds to a concrete element of the *EvoMas* codebase; rows that did not have a matching implementation have been removed.

Description	Source
SWE-bench I/O contract. Cloned repository at the pinned base commit as input; unified <code>git diff</code> as output; revision via re-applying the patch and running the hidden test suite.	[Jimenez et al. 2024]
Three-phase workflow. locator , patcher , and reviewer stages chained in sequence inside the topology graph.	[Xia et al. 2024]
Agentless-style control. The framework drives a fixed sequence of steps rather than letting agents choose actions autonomously.	[Xia et al. 2024] ²
Graph / sub-graph / node. Graph for the multi-agent system, sub-graph per agent, node per prompt-plus-tool pair; maps onto a <code>LangGraph StateGraph</code> .	³
Manager pattern. Explicit orchestrator for specialized agents.	⁴

continued on next page

²codefuse-ai/CodeFuse-CGM

³EuniAI/Prometheus

⁴LingmaTongyi/Lingma-SWE-GPT; AutoCodeRoverSG/auto-code-rover

Description	Source
BaseAgent contract. Abstract base class shared across all agents (common-agent file pattern).	⁵
LLMToolAgent skeleton. Constructor plus <code>run()</code> loop wrapping the function-calling cycle.	⁶
Configurable per-agent instances. Per-agent configuration keyed by an identifier (<code>agents_config.py</code> pattern).	⁷
ReAct cycle. Reasoning-and-acting loop inside every <code>LLMToolAgent</code> .	[Yao et al. 2023]
Proxy agent. Normalizes the raw LLM output into a structured JSON payload.	⁸
One prompt per file. Assembled at runtime from named constants.	⁹
System / proxy / human classification. Explicit prompt-role categories.	¹⁰
XML-style scoping tags. e.g. <code><plan>...</plan></code> inside long prompts.	¹¹
ToolRegistry class. Catalog of available tools and their schemas, exposed to agents via a per-agent whitelist.	¹²
Static syntax linting. Flake8 pre-validation without executing the code.	¹³
BM25 retrieval. Lexical retrieval over the repository class index.	¹⁴
Token counter. Per-agent and per-turn token budgeting and telemetry.	¹⁵
Weave instrumentation. Traces every LLM and tool call to support debugging.	¹⁶
Pluggable LLM back-ends. Unified contract enabling local Ollama or commercial APIs, with APR-specialized models fine-tuned for software engineering tasks as illustrative back-ends.	[Skywork 2025] ¹⁷

4.5.2 Analysis of Alternatives

Beyond the high-level architectural choice (evolutionary MAS framework for APR), three concrete design decisions had to be taken during the framework-construction phase. Each was evaluated against two or more alternatives, and the alternatives together with the chosen option are discussed below.

⁵wandb/programmer; SWE-agent/SWE-agent; LingmaTongyi/Lingma-SWE-GPT; AutoCodeRoverSG/auto-code-rover

⁶patched-codes/patchwork; ComposioHQ/composio

⁷wandb/programmer

⁸LingmaTongyi/Lingma-SWE-GPT

⁹Aider-AI/aider

¹⁰EuniAI/Prometheus

¹¹SWE-agent/SWE-agent

¹²agentscope-ai/agentscope

¹³evandiewald/aegis

¹⁴LingmaTongyi/Lingma-SWE-GPT

¹⁵fishmingyu/OrcaLoca

¹⁶wandb/programmer

¹⁷LingmaTongyi/Lingma-SWE-GPT; LingmaTongyi/AnonymousSWE-GPT

LLM backend. Two alternatives were considered: a free-tier proprietary API (Gemini or OpenAI) and a local-only Ollama instance. A free-tier API would have removed any local hardware requirement and given access to large frontier models out of the box, at the cost of rate limits, recurring fees beyond the free tier, and the need to upload codebases to third parties. A local-only setup, by contrast, gives full control over the model and zero per-token cost but constrains the experiments to relatively small open-weight models. The project runs experiments exclusively on Ollama with the small open-weight model `qwen3.5:9b`, which keeps cost at zero, avoids uploading source code, and preserves the privacy and reproducibility claims made in Chapter 7. The framework still exposes Gemini and OpenAI as pluggable backends for users who already have access to them, so the choice does not exclude future comparisons.

User interface. The alternatives were a command-line interface only or a web frontend. A CLI is the lightest option and is sufficient for scripted benchmarks and *SWE-bench* harnesses, but it makes interactive inspection of the topology and the agent interactions tedious, which would hurt the qualitative analysis. A web frontend requires more implementation effort but allows the user to visualize the topology, configure agents from a browser, and inspect runs without having to read raw logs. A web frontend was therefore included; the extra effort is justified by the fact that one of the primary research artifacts is the topology itself, which is much easier to communicate and debug visually than from a CLI log.

Evaluation corpus. Three options were on the table: synthetic small instances only, *SWE-bench* instances only, or both together with user-chosen custom instances. Synthetic-only is easy to pass and useful as a smoke test but is not directly comparable to the state of the art. *SWE-bench*-only is aligned with the state of the art but is expensive on a small local model and offers no cheap way to exercise edge cases. Supporting both, with user-chosen custom instances, combines fast iteration on hand-picked cases with reproducible results on *SWE-bench*, and additionally enables end users to point the framework at their own codebase and try to repair real bugs in it. This last option is the one adopted by the framework.

4.5.3 Physical Architecture

Figure 4.1 shows the deployment of the *EvoMas* framework across three physical nodes connected over a local network. None of the nodes is exposed to the public internet beyond read-only access to public GitHub repositories. The deployment-side reproduction instructions are documented in Section 5.3.

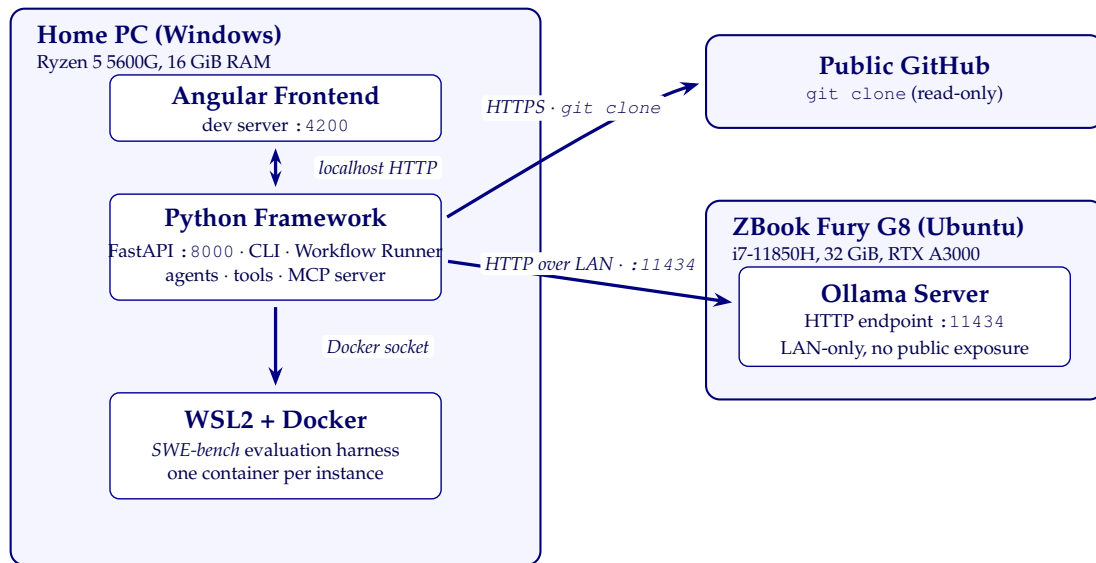


Figure 4.1: Deployment diagram of the *EvoMas* framework. Three nodes (home PC, ZBook Ollama server, public GitHub) communicate over four protocols: localhost HTTP between the Angular frontend and the FastAPI server, the Docker socket between the framework and the WSL Docker daemon, HTTP over the LAN between the framework and the Ollama server, and HTTPS *git clone* against GitHub.

Chapter 5

Development of the Proposal

5.1 Environment Setup

Figure 5.1 summarizes the runtime and development stack on which the *EvoMas* framework was built. The whole stack ran entirely on the local workstation, requiring no cloud service or paid third-party account.



Figure 5.1: Technology stack of the *EvoMas* framework.

The Python backend pinned its dependencies in `pyproject.toml`¹ and the Angular single-page application pinned its dependencies in `app/package.json`².

¹EvoMas-master/pyproject.toml

²EvoMas-master/app/package.json

External services. External services accessed from the backend are limited to a locally hosted Ollama instance for LLM inference (Section 4.5.3), a WSL2-hosted Docker daemon for the *SWE-bench* harness, and a read-only `git clone` against GitHub for per-instance workspace materialization.

Per-sub-project `.env` files held the environment variables that the framework read at startup through `python-dotenv`, so every credential and machine-specific path stayed out of the repository. The `evomas/.env.example` file documented every variable consumed by the backend (shown in Listing 1):

```
1 # Ollama HTTP endpoint used by every `ollama/*` agent.
2 OLLAMA_BASE_URL="http://localhost:11434"
3
4 # Optional API keys for remote providers (only if the agent
5 # config targets `gemini/*` or `openai/*`).
6 # GOOGLE_API_KEY=""
7 # OPENAI_API_KEY=""
8
9 # Predictions + evaluations are written here. Default: <repo>/results.
10 # RESULTS_DIR="results"
11
12 # Per-node revisit budget for the LangGraph runtime. Default: 2.
13 # EVOMAS_GRAPH_MAX_REVISITS="2"
14
15 # Required for `evomas run evaluation --remote` only.
16 # SWEBENCH_API_KEY=""
```

Listing 1: Runtime configuration (excerpt of `evomas/.env.example`).

All remote-API keys are optional and their providers offer a free tier.

The `api/.env.example` file documented the FastAPI server’s bind address (`API_HOST`) and listening port (`API_PORT`), together with an optional `RESULTS_DIR` override that redirected every prediction and evaluation to a folder of choice (shown in Listing 2):

```
1 # Bind address. Use 0.0.0.0 to expose on the LAN, 127.0.0.1 for
2 ↪ local-only.
3 API_HOST=0.0.0.0
4
5 # TCP port the FastAPI server listens on. If you change this, also
6 # update the frontend's BASE URL in
7 ↪ app/src/app/services/api.service.ts
8 # (currently hardcoded to :8000).
9 API_PORT=8000
10
11 # Optional override of the results root. Default: <repo>/results.
12 # RESULTS_DIR="results"
```

Listing 2: FastAPI server configuration (full content of `api/.env.example`). The framework was installed with the scripts `setup.sh` for Linux / macOS and `setup.ps1` for Windows (shown in Listing 3). The setup script created a virtual environment, installed the dependencies declared in `pyproject.toml` and `app/package.json`, and registered the `evomas` console command.

```
1 $ git clone https://github.com/xiangfengyepan/TFG_public
2 $ cd TFG_public/EvoMas-master
3 $ bash setup.sh           # Linux / macOS
4 PS> .\setup.ps1          # Windows PowerShell
5 $ cp evomas/.env.example evomas/.env # fill in OLLAMA_BASE_URL
6 $ cp api/.env.example    api/.env    # optional host/port override
```

Listing 3: Installation commands for the *EvoMas* framework on Linux/macOS and Windows.

The framework was developed in Visual Studio Code, version-controlled with Git, and hosted on GitHub. Project-management tooling, which supports the thesis lifecycle rather than the framework runtime, is documented separately in Section 4.1.2.

5.2 Testing

The `evomas test` console command ran the automated unit tests for both backend and frontend. The backend tests were exercised with `pytest` [pytest contributors 2025] under `evomas/tests/`, one test module per Python module; the frontend tests were exercised with `vitest` under `app/src/` as `*.spec.ts` files, covering each Angular service, page, and component in isolation from the backend. The same command with the `--integration` flag (`evomas test --integration`) additionally ran the slower integration tests under `app/src/integration/`, which exercised each predefined topology end-to-end through the API surface and executed the complete APR cycle against a curated set of instances.

The captured output of a green local run is reproduced in Figure 5.2 (backend) and Figure 5.3 (frontend).

The acceptance criterion for this requirement states that *EvoMas* must solve APR problems end-to-end. A full end-to-end run loaded a topology graph with its agent configuration, executed the inference stage over a program repair task to produce a

```

1 $ PS> evomas test
2 >>> Backend (pytest)
3 $ python -m pytest evomas/tests -q
4 ..... [ 31%]
5 ..... [ 62%]
6 ..... [100%]
7 236 passed, 1 warning in 83.40s (0:01:23)

```

Figure 5.2: Captured output of the backend unit suite (`evomas test` dispatching to `pytest` over `evomas/tests/`). 236 tests pass in roughly 83 seconds on the workstation described in Section 4.5.3.

```

1 >>> Frontend (ng test)
2 $ npx ng test --watch=false
3 Application bundle generation complete. [9.232 seconds]
4
5 RUN v4.1.5 app/
6
7 [OK] src/app/services/inference-state.service.spec.ts (4 tests)
8 [OK] src/app/services/evaluation-run.service.spec.ts (7 tests)
9 [--] src/integration/predefined-configs.spec.ts (23 tests | 23
   ↪ skipped)
10 [OK] src/app/services/results-state.service.spec.ts (2 tests)
11 [OK] src/app/services/topology-state.service.spec.ts (2 tests)
12 [OK] src/app/models/types.spec.ts (5 tests)
13 [OK] src/app/utils/cycles.spec.ts (7 tests)
14 [OK] src/app/guards/unsaved-config.guard.spec.ts (4 tests)
15 [OK] src/app/app.spec.ts (8 tests)
16 [OK] src/app/pages/results/results.component.spec.ts (7 tests)
17 [OK] src/app/pages/topology/topology.component.spec.ts (8 tests)
18
19 Test Files 10 passed | 1 skipped (11)
20 Tests 54 passed | 23 skipped (77)
21 Duration 7.10s

```

Figure 5.3: Captured output of the frontend unit suite (`evomas test` dispatching to `ng test` driven by `vitest` over `app/src/`). 54 tests pass in roughly 7 seconds; the 23 skipped tests belong to the `predefined-configs.spec.ts` integration matrix, which is only run by `evomas test --integration`.

patch, and then evaluated that patch by applying it with `git apply` and running the issue repository's test suite. A green result, in which the previously failing tests passed and no previously passing test regressed, confirmed that the requirement was met.

5.3 Deployment

The *EvoMas* framework is published on GitHub at `xiangfengyepan/TFG-public/EvoMas-master`, which is the public release of the private development repository.

Continuous Integration and Delivery

Figure 5.4 summarizes the CI/CD pipeline that gates every change to the framework. The CI side is a GitHub Actions workflow defined in `.github/workflows/ci.yml` that runs on every push to master/main, on every pull request, and on manual workflow dispatch. Two jobs run in parallel: backend (pyright + pytest) and frontend (vitest + production ng build). The complete workflow file is published in the public release repository.³

The CD side is handled by a mirror step⁴ that automates the source release by downloading the `.zip` archive of the development repository and pushing it into the public release repository `TFG_public/EvoMas-master`.

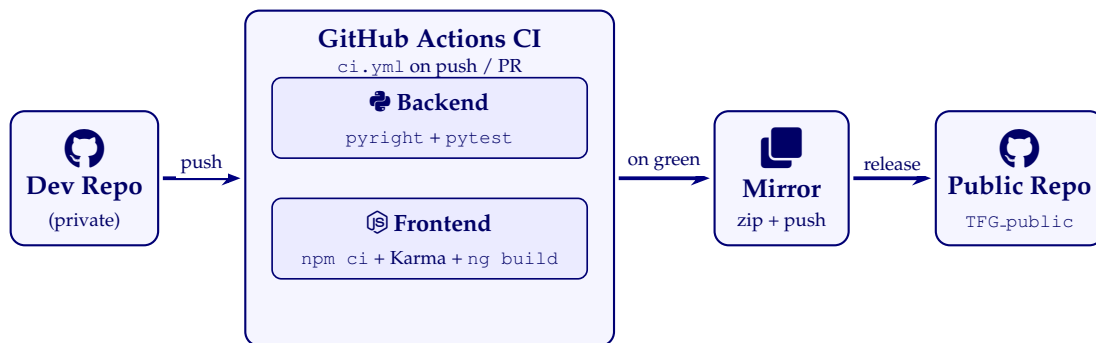


Figure 5.4: CI/CD pipeline of the *EvoMas* framework.

³`EvoMas-master/.github/workflows/ci.yml`

⁴`EvoMas-master/.github/workflows/mirror.yml`

Chapter 6

Experimentation and Evaluation

6.1 Evaluation Metrics

For every run of *EvoMas* over a set of instances, the experimental pipeline records three primary metrics:

- **Resolved rate.** Instances resolved (those that pass the evaluation tests) divided by the total number of instances. This is the primary measure of APR effectiveness.
- **Wall-clock time.** The active wall-clock of each run, measured as the sum of inter-log-line gaps in the inference log with each gap capped at 5 minutes to avoid counting idle time in which the inference server was asleep or unresponsive.
- **Token usage.** Total tokens consumed by the run, split into input and output, extracted from the attribute exposed by the LangChain-Ollama reply.

6.2 Experimental Setup

The full experimental setup, including the per-run notebooks, the prediction logs, and the auto-generated reports, is released under the `EvoMas-master/experiments/` and `EvoMas-master/examples/` directories of the project public repository.

6.2.1 Instances

The experiments draw from four instance sets:

- 5 synthetic APR instances designed as part of this thesis across five difficulty tiers (trivial, easy, medium, hard, expert), each a small Python module shipped with a failing pytest case that the framework must repair, released under the EvoMas GitHub organization¹.
- the 23-instance `swe-bench-lite-dev` subset of *SWE-bench Lite* [SWE-bench Team 2024b], taken from the published `princeton-nlp/SWE-bench_Lite` dataset on Hugging Face without any modification.
- a 77-instance subset of the *SWE-bench Lite test* split, obtained by filtering the full 300-instance test split to those whose gold patch is shorter than 600 characters, chosen to match the size of both resolved patches in the dev subset (see Section 6.3.4).
- two non-APR demos (`translate-intro-en-es` and `websearch-py-latest`) built alongside the framework to test the translation and web-search evaluators, shipped under `EvoMas-master/examples`/².

The full list of instances used in the experiments is given in Table D.1.

6.2.2 Topology

The experiments exercise ten topologies organized into three groups. The first group contains seven repo topologies, each an approximation of the original system rather than an exact reconstruction. Each approximation is assembled from the evidence collected during the Part I analysis, detailed in Section 3.2.7, with the resulting classifications in Section 3.3.7. The second group is the `chain` topology, which implements the three-phase workflow design idea of Table 4.1 by chaining the three essential APR agents identified in Section 3.3.5 (`locator`, `patcher`, `reviewer`) plus a helper `finalizer` agent that wraps the final state into the patch the runner emits. The third group consists of the two topologies used by the non-APR demonstrations. All ten are summarized in Table 6.1, and their visual agent graphs are in Appendix D.

¹github.com/EvoMas

²[TFGpublic/EvoMas-master/examples](https://github.com/TFGpublic/EvoMas-master/examples)

Table 6.1: Topologies exercised in the experiments. The upper half lists the eight APR topologies and the lower half the two non-APR sample topologies. The Class column gives the structural pattern, and the Modeled after column points to the original repository that inspired each entry.

Topology	Class	Modeled after
<i>APR topologies</i>		
chain (baseline)	linear pipeline	n/a
agentscope_hybrid	hub + linear subchain	AgentScope
experrepair_star	centralized (star)	ExpeRepair
hyperagent_star	centralized (star)	HyperAgent
joycode_star	centralized (star)	joycode-agent
lingxi_star	centralized (star)	Lingxi
openhands_star	centralized (star)	OpenHands
prometheus_tree	hierarchical (tree)	Prometheus
<i>Non-APR demo topologies</i>		
translate (demo)	linear chain	n/a
websearch (demo)	single-agent	n/a

6.2.3 Language Models

Every LLM used in this chapter belongs to the qwen family, served locally by Ollama. The two sub-families used are the general-purpose qwen3.5 (2B / 4B / 9B) and the code-finetuned qwen2.5-coder (3B / 7B). The 9B variant is the default for the topology and real-world experiments because it is the largest model that fits in the inference server’s 6 GB VRAM (Section 6.2.6). Table 6.2 lists every model invoked across the experiments.

Table 6.2: Language models used. All five variants and their metadata are taken from the Ollama library pages ollama.com/library/qwen3.5 and ollama.com/library/qwen2.5-coder.

Model	Updated	Pulls	Thinking	Tools
qwen3.5:2b	3 months ago	13.3 M	yes	yes
qwen3.5:4b	3 months ago	13.3 M	yes	yes
qwen3.5:9b	3 months ago	13.3 M	yes	yes
qwen2.5-coder:3b	1 year ago	16.6 M	no	yes
qwen2.5-coder:7b	1 year ago	16.6 M	no	yes

6.2.4 Agent Configuration

Each agent in a topology graph is parameterized by the values listed in Table 6.3. The first four parameters (model, temperature, seed, think) are the experimental variables varied across the reported runs. The remaining parameters are per-role defaults that depend on the agent’s class (locator, patcher, reviewer, etc.). Every Ollama parameter not listed here uses the default value introduced in Section 2.1.1.

Table 6.3: Agent-configuration parameters used across the chapter.

Parameter	Description
<i>Experimental variables (varied across runs)</i>	
model	qwen3.5:{2b, 4b, 9b}, qwen2.5-coder:{3b, 7b}
temperature	0, 0.2, 0.3, 1.0
seed	0 (reproducible), -1 (random seed)
think	true / false (for thinking models)
<i>Per-role defaults (structural, not varied)</i>	
class	per-topology agent role (Planner/Orchestrator, Patcher, Locator, Helper/Proxy, Bug Reproduction, ...)
num_ctx	2048 / 4096 / 8192 / 16384 – per-role context window
num_predict	512 / 1024 / 2048 – per-role output budget
stop	</files>, </patch>, </review>, [] – per-role markers
max_iters	1 / 4 / 6 / 12 – per-role iteration cap

6.2.5 Evaluation Scripts

Each instance set is paired with a dedicated evaluator that decides whether a prediction is resolved. The four scripts used in this chapter live under `EvoMas-master/scripts/evaluation/` and are summarized in Table 6.4.

6.2.6 Hardware

All runs reported in this chapter were executed on the dual-host configuration described in Section 4.5.3 and summarized in Table 6.5. To prevent Windows from suspending either host during long runs, both machines were configured to disable sleep before launching the run.

6.2.7 Reporting Pipeline

The experiment results are auto-generated into a markdown file by `generate_report.py` from the logs, as shown in Figure 6.1. The APR runs and their gener-

Table 6.4: Evaluation scripts for each instance set.

Script	Instance set	Resolution criterion
<code>apply_and_test.py</code>	5 synthetic APR	git apply the predicted patch and run the repository’s pytest suite; resolved iff every test passes
<code>run_swebench_evaluation.py</code>	23 <i>SWE-bench</i> Lite dev	local <i>SWE-bench</i> Docker harness (wrapped in WSL on Windows); resolved iff the hidden test suite passes same harness on the 77-instance test subset
<code>run_swebench_evaluation.py</code>	77 <i>SWE-bench</i> Lite test	
<code>translate_eval.py</code>	translate-intro-en-es	BLEU score against the .gold reference translation; resolved iff the score exceeds a configurable threshold
<code>websearch_eval.py</code>	websearch-py-latest	resolved iff an <code>answer*.md</code> file is produced inside the workspace

Table 6.5: Hardware configuration. The workstation runs *EvoMas* and the *SWE-bench* harness, the ZBook hosts Ollama serving `qwen3.5` and the only GPU (RTX A3000, 6 GB VRAM) large enough to keep the 9B model resident between calls.

Machine	CPU	RAM
Desktop workstation (orchestrator)	Ryzen 5 5600G	16 GB
HP ZBook Fury G8 (inference server)	i7-11850H	32 GB

ated reports are released under `EvoMas-master/experiments/`,³ and the non-APR demonstrations under `EvoMas-master/examples/`.⁴

6.3 Results Analysis

6.3.1 Different LLMs on the Chain Topology

Objective. Measure how repair effectiveness, wall-clock, and token usage scale with model capacity across two `qwen` sub-families (general-purpose `qwen3.5` and code-finetuned `qwen2.5-coder`) on a single fixed topology. The five runs are summarized in Table 6.6.

Results. Table 6.7 reports the per-run evaluation metrics for each model, and Figure 6.2 plots resolved rate against total wall-clock for a visual summary.

Observation.

³`TFG_public/EvoMas-master/experiments`

⁴`TFG_public/EvoMas-master/examples`

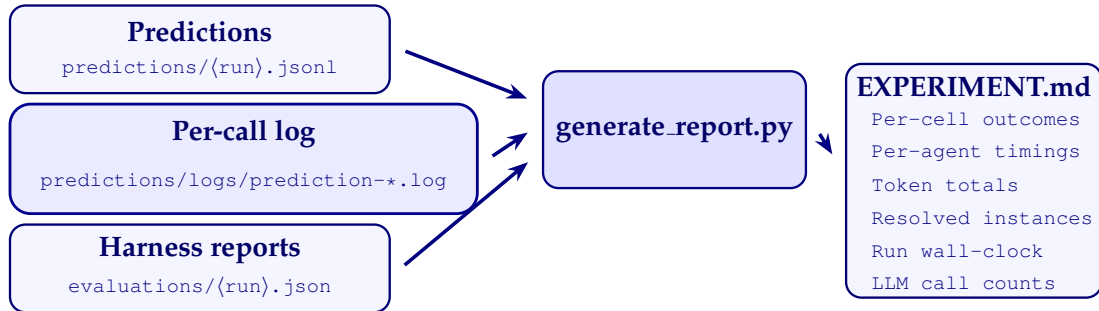


Figure 6.1: Reporting pipeline. Each run drops three artifacts (predictions, per-call log, harness reports) that `generate_report.py` aggregates into a per-run markdown block in `EXPERIMENT.md`; the per-call log is the primary source of token and call-count metrics.

Table 6.6: Configuration of the five chain-topology runs that compare LLM variants on the synthetic APR set.

Instances	Topology	Model	Agent config	Eval script
5 synthetic APR	chain	qwen3.5:2b	default	synthetic
5 synthetic APR	chain	qwen2.5-coder:3b	default, think=false	synthetic
5 synthetic APR	chain	qwen3.5:4b	default	synthetic
5 synthetic APR	chain	qwen2.5-coder:7b	default, think=false	synthetic
5 synthetic APR	chain	qwen3.5:9b	default	synthetic

Table 6.7: Chain-topology results across five `qwen` models. Each **Resolved** cell shows five boxes ordered trivial, easy, medium, hard, and expert, green marks instances the run resolved.

Model	Wall-clock	Resolved	LLM calls	Total tokens (in + out)
qwen3.5:2b	7 min 45 s	□□□□□	118	408,879 (381,539 + 27,340)
qwen2.5-coder:3b	1 min 9 s	□□□□□	25	34,134 (32,618 + 1,516)
qwen3.5:4b	9 min 53 s	■ ■ □ ■ ■	64	186,606 (174,718 + 11,888)
qwen2.5-coder:7b	4 min 12 s	□ □ ■ □ □	24	33,121 (31,471 + 1,650)
qwen3.5:9b	29 min 2 s	■ ■ □ ■ ■	79	226,605 (212,885 + 13,720)

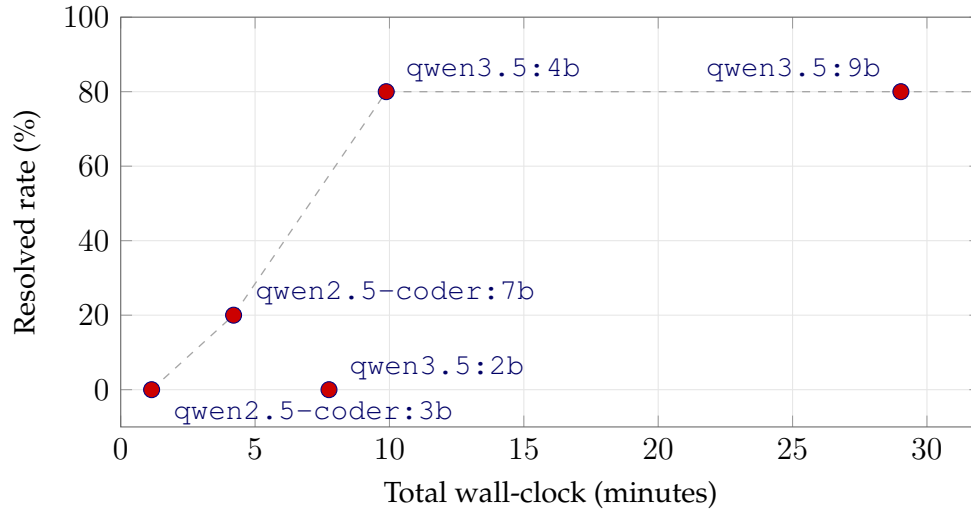


Figure 6.2: Resolved rate (%) vs. total wall-clock (minutes) for five qwen-family model variants on the chain topology.

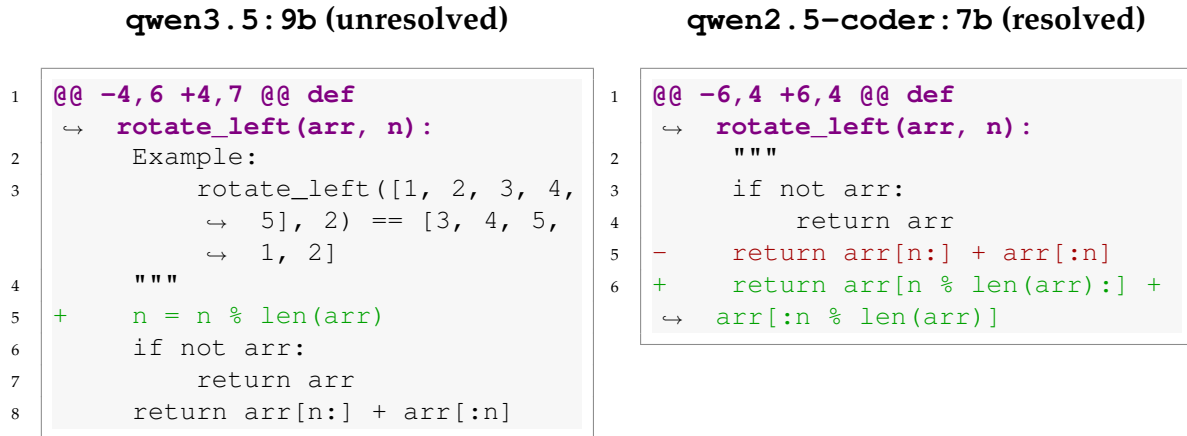


Figure 6.3: Side-by-side patches for the *medium* instance. The qwen3.5:9b patch (left) inserts the modulo normalization *before* the empty-array guard, so an empty `arr` now raises `ZeroDivisionError` and the hidden test fails. The qwen2.5-coder:7b patch (right) applies the same modulo idea inline at the slice indices, which is correct both for non-empty and empty inputs.

- **qwen3.5:2b is too small for APR.** Despite emitting 118 LLM calls and the highest token usage of any model, it cannot return a well-formatted patch diff on any instance.
- **The qwen2.5-coder variants are code-specialized but lack a reasoning step.** As a consequence of that missing reasoning, they run an order of magnitude faster (1–4 minutes, 24–25 calls) but underperform the general-purpose qwen3.5 family (0/5 for coder-3B and 1/5 for coder-7B). The smaller parameter counts (3B and 7B) also contribute to the gap.
- **Coder-7B solves the one instance that 9B cannot.** The single synthetic instance qwen2.5-coder:7b resolves is *medium*, which is also the only instance qwen3.5:9b fails on. The two patches are shown side by side in Figure 6.3. The <think> step of qwen3.5:9b leads it to over-think a separate normalization statement and place it in the wrong position, while qwen2.5-coder:7b, a code-specialized model, emits the minimal correct edit directly.
- **Bigger model does not mean slower pipeline.** The intuition “a bigger model takes more wall-clock” only holds on a single model call. At the pipeline level it breaks because smaller models make more LLM calls to resolve the problem. The reasoning-enabled qwen3.5 family is also consistently slower per call than the non-reasoning qwen2.5-coder, as expected.

Takeaway. Within the qwen3.5 family, reasoning-enabled and larger models reach a higher resolved rate, at the cost of longer wall-clock time. However, the reasoning step can also tip into overthinking, as seen on the *medium* instance where the non-reasoning qwen2.5-coder:7b hits a patch that qwen3.5:9b misses. Hardware limitations (Section 6.2.6) cap the experiments at qwen3.5:9b, which is still much smaller than frontier cloud models. The conclusions reported here therefore apply within the local-deployment regime.

6.3.2 Temperature and Seed Sensitivity

Objective. Test the variability of high sampling temperature (which increases output diversity) against the consistency of low sampling temperature (which concentrates probability mass on the top token) on the same chain pipeline. The two variants are summarized in Table 6.8.

Table 6.8: Configuration of the two chain-topology variants used to probe temperature and seed sensitivity. Each variant is repeated five times with `seed=-1` (random seed per run).

Instances	Topology	Model	Agent config	Eval script
5 synthetic APR (×5 reps)	chain	qwen3.5:9b	temperature=0, seed=-1	synthetic
5 synthetic APR (×5 reps)	chain	qwen3.5:9b	temperature=1, seed=-1	synthetic

Results. Table 6.9 reports the per-run metrics for every repetition. Figure 6.4 visualizes the rep-to-rep behavior of the three primary metrics side by side.

Table 6.9: Run-level metrics for the temperature and seed-sensitivity experiments, all at qwen3.5:9b on the chain topology. The *temp-lo* variant uses temperature=0 and the *temp-hi* variant uses temperature=1, each repeated five times with a fresh random seed (`seed=-1`). Each **Resolved** cell shows five boxes ordered trivial, easy, medium, hard, and expert, green marks instances the run resolved.

Variant	Wall-clock	Resolved	LLM calls	Total tokens (in + out)
temp-lo=0, rep 1	31 min 9 s	■ □ □ ■ ■	76	206,082 (191,392 + 14,690)
temp-lo=0, rep 2	31 min 48 s	■ □ □ ■ ■	76	206,523 (191,424 + 15,099)
temp-lo=0, rep 3	31 min 0 s	■ □ □ ■ ■	76	206,007 (191,370 + 14,637)
temp-lo=0, rep 4	31 min 43 s	■ □ □ ■ ■	76	206,419 (191,438 + 14,981)
temp-lo=0, rep 5	31 min 30 s	■ □ □ ■ ■	76	206,342 (191,450 + 14,892)
temp-hi=1, rep 1	29 min 25 s	■ ■ □ □ □	80	231,711 (217,700 + 14,011)
temp-hi=1, rep 2	25 min 19 s	■ ■ □ ■ ■	76	220,260 (208,432 + 11,828)
temp-hi=1, rep 3	31 min 28 s	■ ■ □ ■ ■	79	234,406 (219,425 + 14,981)
temp-hi=1, rep 4	34 min 28 s	■ ■ □ □ ■	85	276,788 (260,561 + 16,227)
temp-hi=1, rep 5	32 min 47 s	■ ■ ■ ■ □	73	198,720 (183,029 + 15,691)

Observation.

- **Low temperature is effectively deterministic.** Even with a fresh random seed per repetition, all five low-temperature runs converge to the same resolved set and emit nearly identical token counts. At very low temperature, the next-token distribution is concentrated on the top-probability token, so the variation is minimal regardless of the seed.
- **High temperature exposes sampling variance.** The resolved rate ranges from 2 to 4 out of 5, and *medium* (which the fixed-seed baseline in Table 6.11 does not resolve) passes in rep 5.

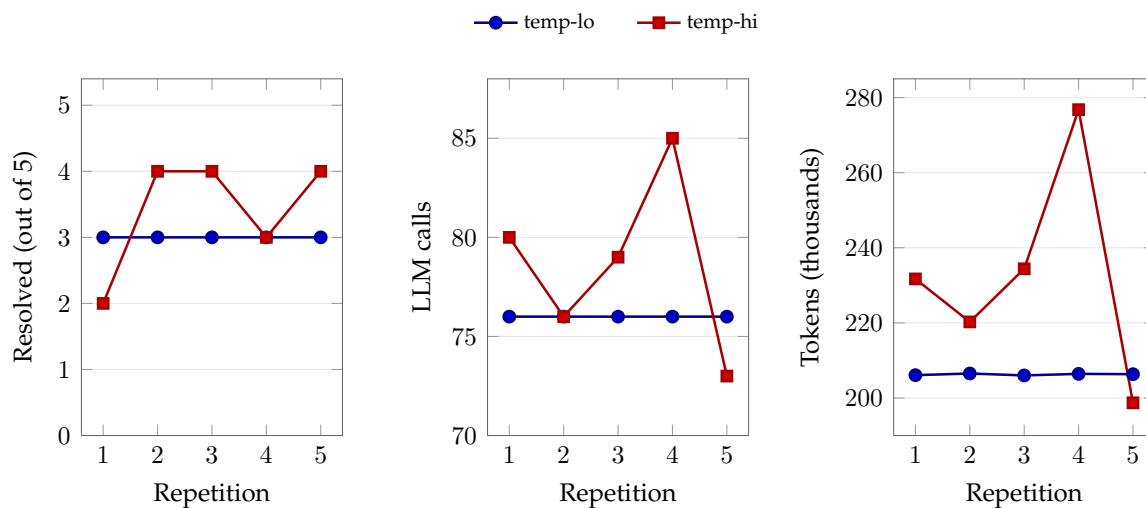


Figure 6.4: Rep-to-rep behavior of the chain topology at `qwen3.5:9b` with a fresh random seed per repetition (5 reps per variant).

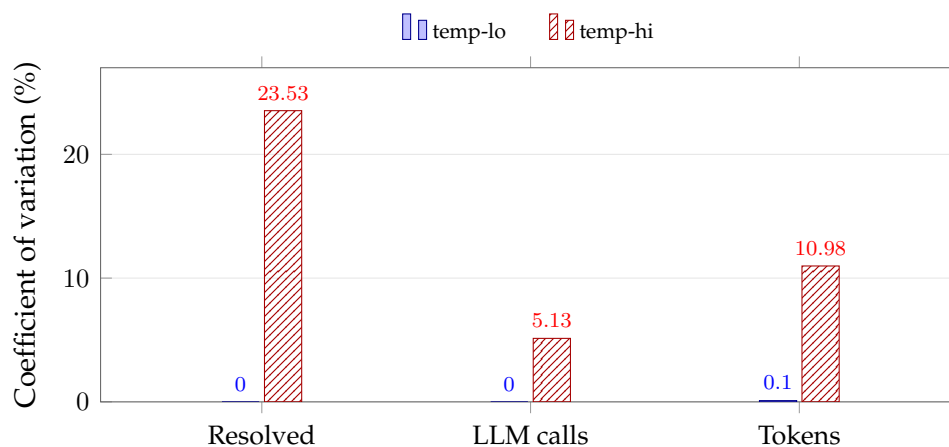


Figure 6.5: Coefficient of variation of the three primary metrics across the five random-seed repetitions per variant.

- **Quantifying the spread.** Figure 6.5 reports the population coefficient of variation (CV, σ/μ) of each metric across the five reps. The temp-lo CV is negligible on calls, tokens and resolved rate, while the temp-hi CV is 5.13 %, 10.98 %, and 23.53 % for LLM calls, tokens, and resolved rate respectively. This matches the definition of the sampling-temperature hyperparameter.

Takeaway. At low temperature the chain is effectively deterministic, so variation across runs is negligible. At high temperature the same configuration produces a range of outcomes, either better or worse. LLM output variability can therefore be modulated through this temperature parameter.

6.3.3 Topology Comparison at qwen3.5:9b

Objective. Compare the eight predefined topologies on the synthetic set under a fixed model (qwen3.5:9b) to see how structural choice affects the APR pipeline. The eight runs are summarized in Table 6.10.

Table 6.10: Configuration of the eight topology runs compared on the synthetic APR set at qwen3.5:9b.

Instances	Topology	Model	Agent config	Eval script
5 synthetic APR	chain (baseline)	qwen3.5:9b	default	synthetic
5 synthetic APR	agentscope_hybrid	qwen3.5:9b	default	synthetic
5 synthetic APR	experepair_star	qwen3.5:9b	default	synthetic
5 synthetic APR	hyperagent_star	qwen3.5:9b	default	synthetic
5 synthetic APR	joycode_star	qwen3.5:9b	default	synthetic
5 synthetic APR	lingxi_star	qwen3.5:9b	default	synthetic
5 synthetic APR	openhands_star	qwen3.5:9b	default	synthetic
5 synthetic APR	prometheus_tree	qwen3.5:9b	default	synthetic

Results. Table 6.11 reports the aggregated run-level metrics for each topology, and Figure 6.6 ranks the eight topologies by resolved rate with wall-clock.

Observation. The ranking exposes both resolved rate and wall-clock cost.

- **openhands_star resolves all five instances with the lowest wall-clock time and fewest tokens.** It resolves all 5 instances in 17 min 32 s with about 83,000 tokens. Against the top-2 topology hyperagent_star (29 min 35 s, 229,188 tokens), openhands_star runs 40.73 % faster and uses 63.86 % fewer tokens. Structurally, the controller role carries no tools and

Table 6.11: Topology comparison at qwen3.5:9b. Each **Resolved** cell shows five boxes ordered trivial, easy, medium, hard, and expert, green marks instances the run resolved.

Topology	Wall-clock	Resolved	LLM calls	Total tokens (in + out)
chain (baseline)	29 min 2 s	■ ■ □ ■ ■	79	226,605 (212,885 + 13,720)
agentscope_hybrid	35 min 21 s	■ ■ □ ■ ■	91	254,316 (237,712 + 16,604)
experepair_star	41 min 18 s	■ □ □ ■ ■	66	197,781 (177,994 + 19,787)
hyperagent_star	29 min 35 s	■ ■ ■ ■ ■	69	229,188 (215,895 + 13,293)
joycode_star	39 min 15 s	■ ■ ■ ■ ■	116	256,908 (238,489 + 18,419)
lingxi_star	42 min 22 s	■ ■ ■ ■ ■	106	300,563 (280,282 + 20,281)
openhands_star	17 min 32 s	■ ■ ■ ■ ■	52	82,840 (74,523 + 8,317)
prometheus_tree	31 min 41 s	■ ■ □ ■ □	56	172,084 (156,756 + 15,328)

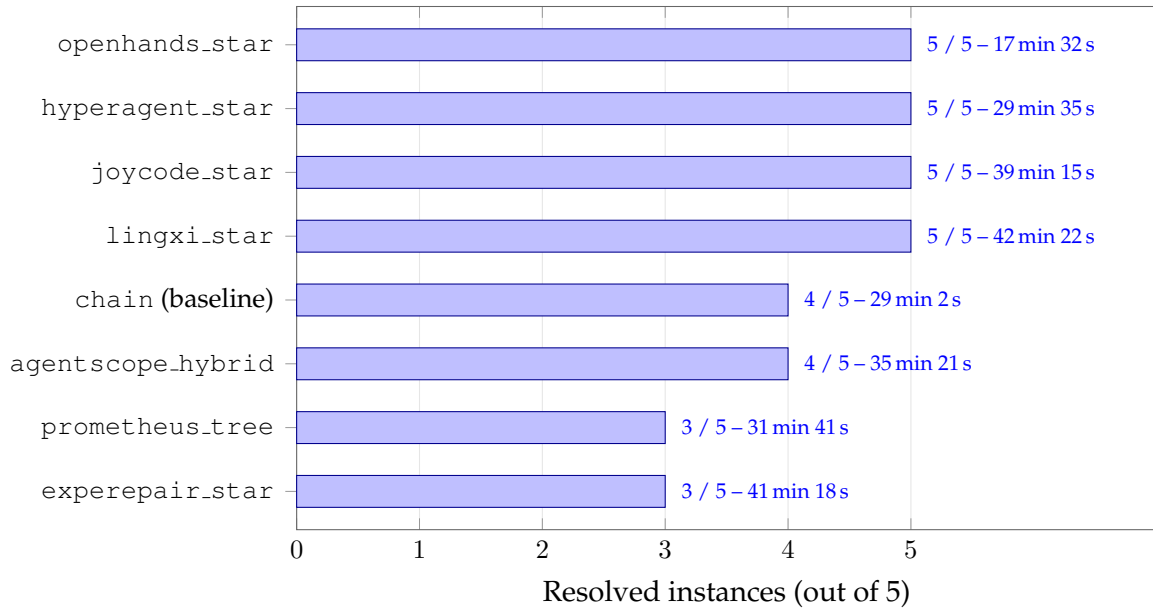


Figure 6.6: Ranking of the eight topologies by resolved rate, with ties broken by ascending wall-clock.

emits a one-word dispatch per cycle, fanning out to `locator`, `coder`, and `finalizer` (the `reader` and `browser` nodes are wired but never selected by the controller prompt). The `locator` and `coder` agents average about 1.4k input tokens per call, and the controller fires only ~ 3 times per instance, so it is rarely re-prompted with full file diffs. However, these results do not establish that `openhands_star` is uniformly superior to `hyperagent_star`. `openhands_star` drops to 0/23 on the *SWE-bench* Lite subset while `hyperagent_star` and the `chain` baseline both retain 2/23 for the fixed seed used here (Section 6.3.4).

- **`prometheus_tree` and `experepair_star` underperform.** Both stop at 3/5 despite competitive token budgets. `prometheus_tree` is a one-way fan-out that picks a single branch and commits to it, so a wrong early decision or a near-miss patch has nowhere to be corrected. `experepair_star` cycles a central hub with its workers, but the verification stage is a terminal node with no outgoing edges to the workers and therefore cannot propagate the verification result back to the repair workers. In both cases the structure is missing the feedback loop between detection and revision to recover from a failure.

Takeaway. Topology choice does affect the APR pipeline, but the hardware constraint caps the model size, which prevents distinguishing whether topology or model capacity is the dominant factor.

6.3.4 Real-World Benchmark

Objective. Validate the effectiveness of the *EvoMas* framework at solving APR problems from real GitHub issues, by running on the 23-instance *SWE-bench* Lite dev subset and a 77-instance subset of the *SWE-bench* Lite test split. The runs are summarized in Table 6.12.

Results. Table 6.13 reports the aggregated run-level metrics, and Figure 6.7 ranks the three topologies by resolved rate with wall-clock as the tiebreaker.

Observation.

Table 6.12: Configuration of the real-world runs at `qwen3.5:9b`. The top three runs cover the 23-instance dev subset; the bottom row covers the 77-instance Lite test subset (gold patch < 600 characters).

Instances	Topology	Model	Agent config	Eval script
23 <i>SWE-bench</i> Lite dev	chain (baseline)	<code>qwen3.5:9b</code>	default	<i>SWE-bench</i> harness
23 <i>SWE-bench</i> Lite dev	<code>hyperagent_star</code>	<code>qwen3.5:9b</code>	default	<i>SWE-bench</i> harness
23 <i>SWE-bench</i> Lite dev	<code>openhands_star</code>	<code>qwen3.5:9b</code>	default	<i>SWE-bench</i> harness
77 <i>SWE-bench</i> Lite test subset	chain	<code>qwen3.5:9b</code>	default	<i>SWE-bench</i> harness

Table 6.13: Topology comparison on *SWE-bench* Lite at `qwen3.5:9b`.

Topology	Wall-clock	Resolved	LLM calls	Total tokens (in + out)
chain (baseline)	4 h 37 min	2 / 23	366	2,178,309 (2,056,725 + 121,584)
<code>hyperagent_star</code>	7 h 40 min	2 / 23	558	4,134,877 (3,937,137 + 197,740)
<code>openhands_star</code>	3 h 15 min	0 / 23	378	1,348,773 (1,256,329 + 92,444)
<i>Subset of SWE-bench Lite test split (77 of 300, gold patch < 600 chars)</i>				
chain (9b)	16 h 4 min	12 / 77	1,173	6,344,990 (5,942,743 + 402,247)

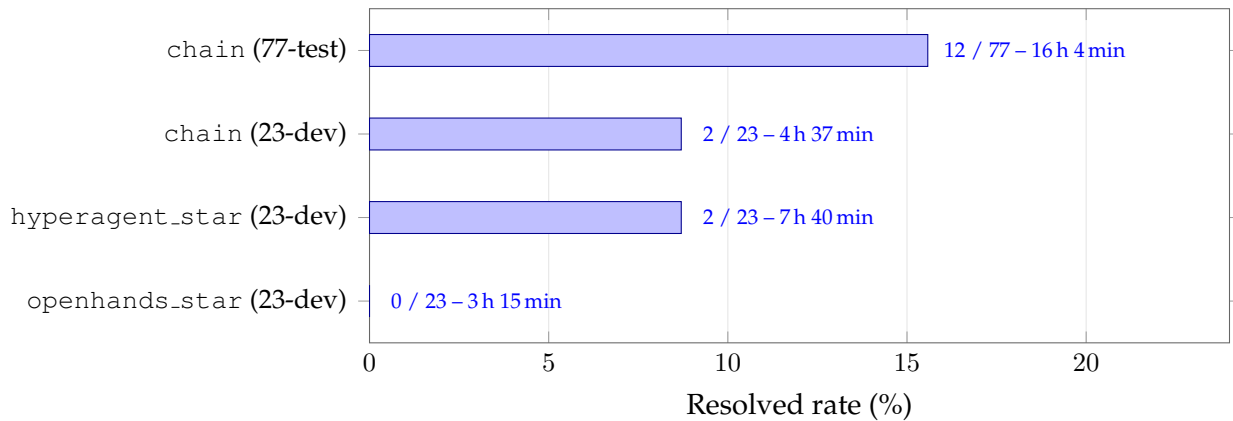


Figure 6.7: Ranking of the topologies tested on *SWE-bench* Lite at `qwen3.5:9b`.

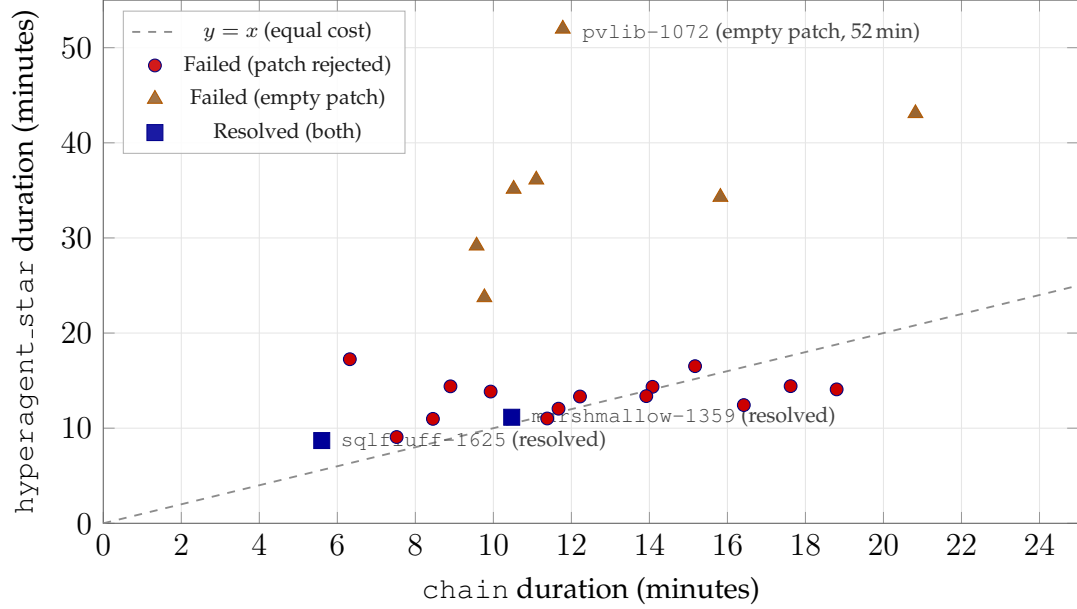


Figure 6.8: Per-instance duration: chain vs hyperagent_star on *SWE-bench Lite*. Each point is one of the 23 instances; the $y = x$ reference line shows that hyperagent_star is uniformly slower, and the seven empty-patch failures (orange triangles) inflate its budget to 24-52 min while chain handles the same instances in 10-20 min.

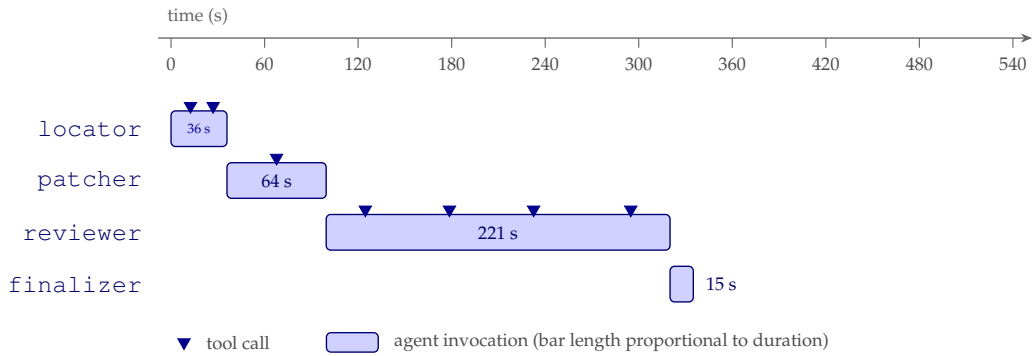


Figure 6.9: Execution trace: chain on sqlfluff-1625. Each row is one agent at qwen3.5:9b, bar length is proportional to in-agent wall-clock, and triangles mark tool calls; the reviewer dominates the 335 s trace at 221 s and 4 tool calls.

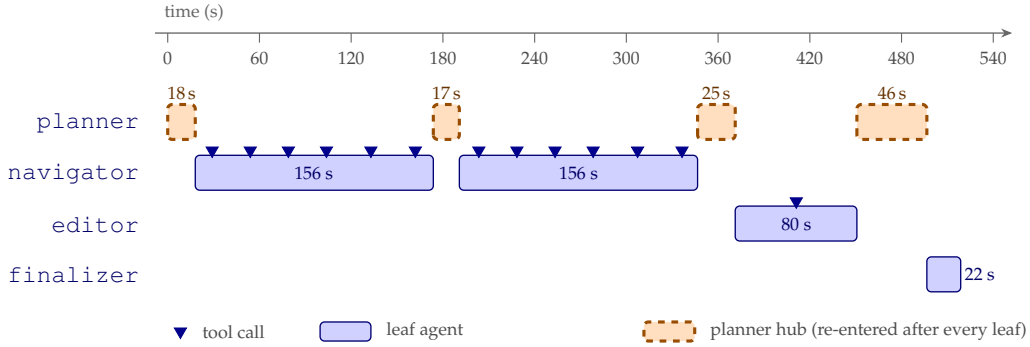


Figure 6.10: Execution trace: `hyperagent_star` on `sqlfluff-1625`. Same instance and time scale as Figure 6.9; the planner hub fires four times, routing successively to navigator, navigator (second pass), editor, and finalizer, for a total of 521 s (35 % longer than chain for the same resolved patch).

- **Same two instances resolved by both 2/23 topologies.** `chain` and `hyperagent_star` resolve on the *same* pair, `marshmallow-1359` and `sqlfluff-1625`. Both share three properties that the other 21 instances do not: a small gold patch⁵ (< 600 characters), a small, focused library, and a bug that is fixable using general Python knowledge without domain-specific expertise. The other 21 instances either need a patch larger than one localized edit or require domain knowledge the model lacks.
- **chain performs comparably to hyperagent_star despite being forward-only.** Topologically, `chain` is forward-only, while `hyperagent_star` wires bidirectional planner-navigator and planner-editor edges and its dispatcher re-routes to the navigator on empty output. At a frontier-scale model that extra recovery loop would be expected to convert into a higher resolved rate, but at the 9B scale used here the two topologies tie at 2/23 while `chain` runs 39.78 % faster and uses 47.32 % fewer tokens than `hyperagent_star`. The per-agent traces on `sqlfluff-1625` make the difference concrete: Figure 6.9 shows that each `chain` agent runs exactly once, whereas Figure 6.10 shows `hyperagent_star`’s planner re-entering the navigator for a second 6-tool pass after the first pass returns empty. The per-instance breakdown in Figure 6.8 shows that on the five `pvl` instances (the five points farthest above the dashed $y = x$ reference line in Figure 6.8) `hyperagent_star` in particular burns its budget on the locator and still emits an empty patch.

⁵The *gold patch* of a *SWE-bench* instance is the human-authored reference diff included in the benchmark as the ground-truth fix.

With a small LLM, every additional agent and dispatcher hop introduces another point of failure, so running a more complex topology does not clearly improve the resolved rate.

- **A subset of SWE-bench Lite test: 12 of 77 resolved.** The full *SWE-bench* Lite test split contains 300 instances, so running all of them at the chain-9b pace of 12.52 min/instance would have taken 62.60 h of inference alone. Therefore, we restricted the run to the 77 instances whose gold patch is shorter than 600 characters, matching the size of both resolved gold patches in the 23-instance dev subset. On this subset, `chain` resolves 12 of 77 (15.58 %) at `qwen3.5:9b`. These results show that the APR pipeline works on real benchmark instances and not only synthetic ones.

Takeaway. Under a locally hosted LLM, the disadvantages of more complex topologies discussed in Section 2.2.1 outweigh their advantages, since each additional agent becomes one more step at which a weak model can hallucinate and emit an empty patch. Accordingly, the simpler chain topology stays competitive on resolved rate while consuming fewer tokens and less wall-clock on this hardware. Whether a complex topology pays off on harder instances should be re-tested with a frontier cloud model, where the per-step error rate is low enough for the coordination benefits to compensate for these risks.

6.3.5 Token Efficiency Across Resolved Instances

Objective. Find the relationship between tokens, wall-clock, and LLM-call count across every APR prediction collected in this chapter, and measure the input/output token ratio across the resolved and unresolved predictions. The aggregated set is summarized in Table 6.14.

Table 6.14: Configuration of the token-efficiency analysis. The 256 `<run, instance>` pairs are the union of every APR prediction produced by the previous four sub-sections.

Instances	Topology	Model	Agent config	Eval script
256	all 8 APR topologies	5 qwen variants	as above	synthetic + SWE-bench harness

Results. Three scatter plots pair the first-order cost dimensions (wall-clock minutes, LLM calls, total tokens). Each plot overlays an ordinary-least-squares regression line on all 256 predictions; markers are color-coded by harness verdict (resolved versus unresolved).

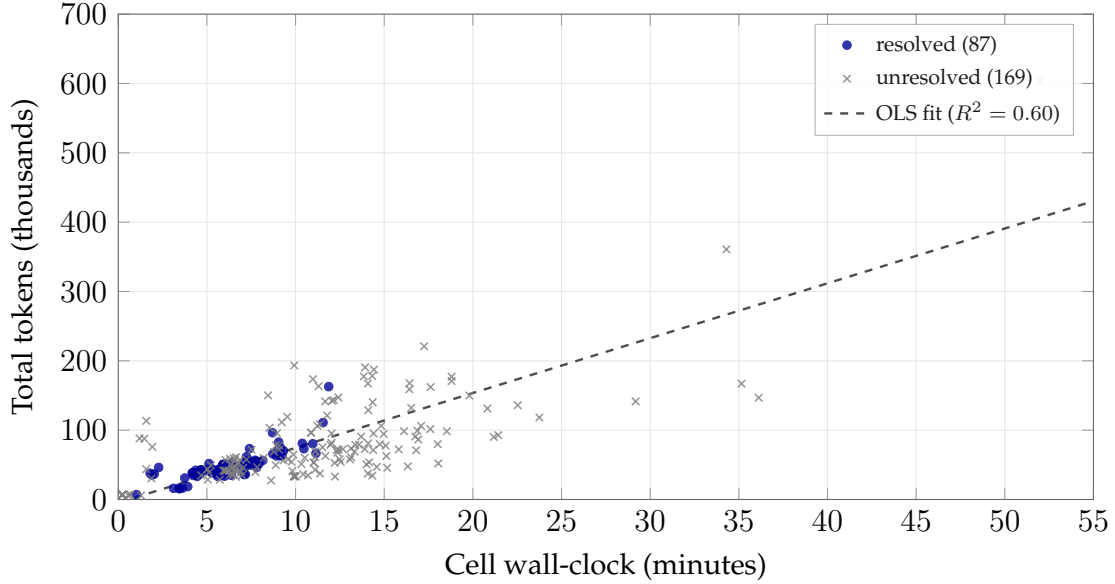


Figure 6.11: Total tokens vs cell wall-clock. Total tokens (thousands) against cell wall-clock (minutes) for every $\langle \text{run}, \text{instance} \rangle$ prediction. The dashed line is the ordinary-least-squares fit on all 256 points ($R^2 = 0.60$).

Observation.

- **Tokens scale roughly linearly with both wall-clock time and LLM call count across all predictions.** Both Figure 6.11 and Figure 6.12 fit a slope of about 7.9k tokens per minute of wall-clock and 8.4k tokens per LLM call respectively. Resolved instances cluster in the bottom left of each plot, while unresolved runs sit in the upper-right, which shows that unresolved instances waste more time and calls without producing a valid patch. The unresolved points are also less consistent in their per-minute and per-call cost, sitting farther from each regression line.
- **Almost all tokens are input context.** Figure 6.13 shows that 79.66-97.90 % of every run is input context (mean 93.00 %) and the model produces at most about 20 % of a run's tokens, because the input on every call accumulates the agent's system prompt, the tool outputs returned so far, and the responses of

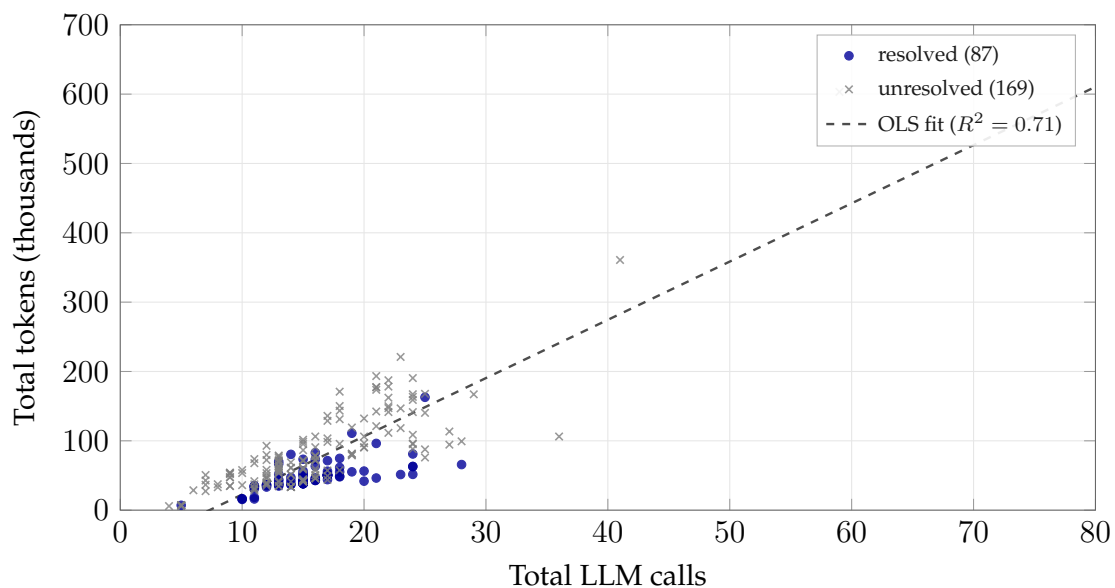


Figure 6.12: Total tokens vs total LLM calls. Total tokens (thousands) against total LLM calls for every $\langle \text{run}, \text{instance} \rangle$ prediction. The dashed line is the ordinary-least-squares fit on all 256 points ($R^2 = 0.71$).

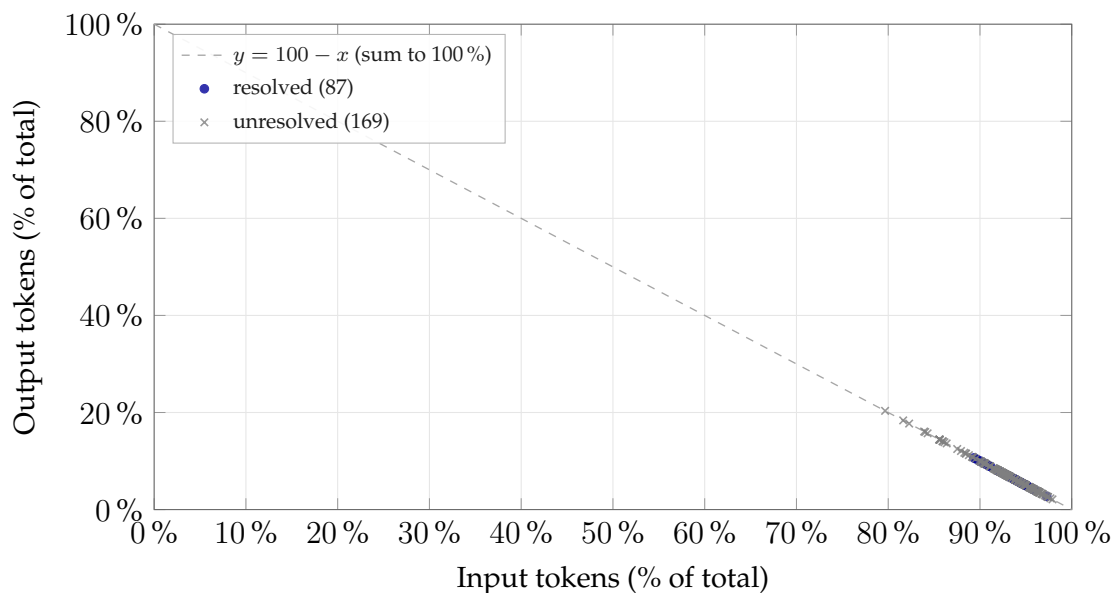


Figure 6.13: Input versus output token share per prediction ($n = 256$; 87 resolved + 169 unresolved).

previous nodes in the topology, while the output is only the agent’s immediate reply.

Takeaway. First, because tokens, wall-clock, and LLM-call count are approximately linearly related across the 256 predictions, the token total of a prediction can be used to estimate its wall-clock and its number of LLM calls with moderate correlation ($R^2 = 0.60$ and $R^2 = 0.71$, respectively). Second, since 79.66-97.90 % of every run is input context, token usage can be reduced substantially without touching the model itself.

6.4 Framework Extensibility Beyond APR

To validate the modularity claim of Section 4.2.2, two end-to-end demonstrations are shipped under `EvoMas-master/examples/` that solve non-APR tasks.

6.4.1 Translation Demo

The `translate_demo` translates the prose of every markdown file in a repository. The demo instance includes a `.gold` file (the human reference translation) and tests English-to-Spanish translation using a BLEU-score evaluator. The evaluator computes the BLEU score⁶ of the output against the gold reference and marks the instance resolved when that score exceeds a configurable threshold. Figure B.4 in Appendix B shows the unified diff produced.

6.4.2 Web-Research Demo

The `websearch_demo` uses web search tools to save the answer to a question in a file. The demo instance `websearch-py-latest` asks “What is the latest stable Python release? Write the answer to `./answer-py.md`”. The evaluator marks the instance resolved when an `answer*.md` is generated. The `answer-py.md` produced by the demo run is shown in Figure 6.14.

⁶*BLEU* (Bilingual Evaluation Understudy) is a machine-translation metric based on n-gram overlap between the candidate output and one or more human reference translations, scored in $[0, 1]$. Higher values indicate closer agreement with the reference.

```
1 The latest stable Python release is Python 3.14.5. This version is
  ↳ highlighted on the official Python downloads page, which notes
  ↳ that Python 3.13.13 is no longer supported on Windows 7 or earlier
  ↳ systems. For more details, see the release notes.
2
3 Sources: https://www.python.org/downloads/windows/
```

Figure 6.14: Websearch-demo output: `answer-py.md`. Verbatim contents written by the researcher agent for the `websearch-py-latest` instance after one `websearch` call and one `save_text` call.

6.5 Limitations

Hardware variance. The Ollama HTTP server runs on a single HP ZBook. GPU temperature and memory overload both contribute to wall-clock variance, especially for long experiment runs. Given the 6 GB VRAM constraint, every experiment in this chapter is executed sequentially rather than in parallel, so that VRAM is dedicated to a single inference at a time.

LLM non-determinism. Every run reported in this chapter fixes the sampling seed to 0, except for the ten temperature/seed-sensitivity runs in Section 6.3.2, so the results are reproducible.

Optimistic synthetic set and small sample size. The five synthetic instances are micro-repositories designed to produce faster, more abundant results on small codebases than the *SWE-bench* Lite subset allows.

Model ceiling. The low resolved rate on *SWE-bench* Lite (at most 2/23 under the present hardware) suggests that `qwen3.5:9b` lacks the reasoning capacity required for the majority of real-world repair tasks. Any follow-up work that aims to climb the public leaderboard will have to swap the LLM for a frontier closed-source model.

6.6 Discussion and Implications

Taken together, the experiments in Section 6.3 converge on the following question.

Question. *Does the choice of agent topology change APR effectiveness, and under what conditions does a more complex topology pay off?*

Answer. Topology does affect APR outcomes, but the small, locally hosted LLM used in these experiments limits how much the topology can contribute.

The topology comparison (Section 6.3.3), the real-world benchmark (Section 6.3.4), and the model comparison (Section 6.3.1) together show that topology affects the outcome while the underlying model sets the ceiling on how much it can help.

These results carry a clear implication for future work. Since topology does affect APR outcomes but the small, locally hosted model caps how much that structure can contribute, the natural next step is to re-run the same topologies under a frontier-scale model. This is left as future work in Section 8.3.

Chapter 7

Sustainability and Ethical Implications

7.1 Self-Assessment

At the start of the project, the sustainability survey provided by the GEP course was completed. The results indicated that while the author's technological grounding in software engineering was solid, the awareness of the economic constraints of deploying AI tools, of the environmental footprint of LLM inference, and of the long-term social implications of automated code generation was considerably narrower. This chapter records what the project has taught about each of those three dimensions.

From an economic perspective, the author's academic background had primarily focused on technical implementation rather than financial viability. Calculating the staff costs, hardware amortization, and indirect expenses for the 540-hour project budget made the financial constraints of academic AI research concrete for the first time (Section 7.2).

Regarding the environmental dimension, the project sharpened my understanding of where the ecological footprint of an APR stack actually lives. Cloud-served models run on large data-center GPU clusters and carry a correspondingly higher energy footprint, whereas serving a small open-weight model locally via Ollama substantially reduces that cost. The choice of local inference was therefore both a privacy and an environmental decision (Section 7.3).

Finally, in the social dimension, the survey raised my awareness of the ethical considerations about code reliability under autonomous patching, the long-term effect on junior developer careers, and the privacy of source code shared with LLMs (Section 7.4).

7.2 Economic Impact

Project Put into Production (PPP): Reflection on the cost you have estimated for the completion of the project.

The total cost of the project, including staff costs, resource costs, contingency margins, and incidentals, is estimated at 22,565.18 €. At roughly the annual salary of a junior software engineer in Spain [Payscale 2025], this figure situates a half-year research project within normal academic research expenditure. The full breakdown is given in Table 7.1.

As anticipated for a software engineering thesis, the vast majority of the budget is allocated to human resources rather than material ones. Because the framework relies entirely on open-source tooling and local execution of the MAS, material costs total 81.68 €, or less than 0.4% of the total budget.

Table 7.1: Detailed project budget, reproducing the figures estimated during the GEP phase. Staff costs already include social-security contributions; material costs combine hardware amortization, electricity, and internet access. The hardware-amortization line covers the home workstation, the HP ZBook Fury G8 laptop added after the GEP estimate as a dedicated Ollama inference server (Section 5.1) contributes an incremental amortization of about 67.50 €, which is absorbed within the contingency margin. The contingency margin is computed as 15% of the staff-plus-materials subtotal.

Item	Hours	Cost/h (€)	Total (€)
<i>Staff</i>			
Project Manager	100	52.00	5,200.00
Researcher	210	39.00	8,190.00
Junior Software Engineer	200	26.00	5,200.00
Software Tester	30	19.50	585.00
Total staff	540		19,175.00
<i>Materials</i>			
Hardware amortization			30.38
Electricity			21.60
Internet access			29.70
Total materials			81.68
Subtotal (staff + materials)			19,256.68
Contingency (15%)			2,888.50
Incidentals reserve			420.00
Total			22,565.18

Useful Life: How are currently solved economic issues related to the problem that you want to address?, and How will your solution improve economic issues

with respect to other existing solutions?

Currently, the economic burden of software debugging and maintenance is addressed by allocating massive portions of a company's budget to human resources. Because human capital is the most expensive asset in the technology sector, manual bug-fixing is a costly bottleneck that drains resources that could otherwise be invested in developing new features or scaling the product.

Our APR framework provides a highly cost-effective alternative. Its real-world application will significantly reduce the budget companies must otherwise allocate to manual software testing and technical debt management. Compared to other emerging AI debugging solutions that rely on expensive, proprietary cloud APIs, our approach prioritizes open-source tools and local execution of MAS, ensuring that operational and scaling costs remain close to zero.

Economic Risks: What scenarios could harm the project's economic viability?

A single scenario could undermine the framework's economic case during its useful life: a license change in the underlying technology stack. The framework has many dependencies, and any of those dependencies going commercial or relicensing under more restrictive terms would force either a maintenance fork or a paid replacement, both of which break the zero-operational-cost premise the framework was built on. The mitigation is to keep the dependency surface to projects with long-standing permissive licenses.

7.3 Environmental Impact

Regarding PPP: Have you estimated the environmental impact of the project?

The environmental impact during the project is relatively low because the only material resources utilized are the personal workstation and a dedicated inference laptop. The project architecture intentionally avoids relying on external servers or cloud-based API calls, meaning there is no hidden carbon footprint generated by remote data centers during development. The primary environmental cost is therefore strictly limited to the electricity consumed by the workstation. Taking an average power draw of approximately 200 W for the AMD Ryzen 5 5600G workstation and its peripherals over 540 hours, plus approximately 100 W for the HP ZBook Fury G8 laptop (Intel i7-11850H + NVIDIA RTX A3000 Mobile) hosting the Ollama LLM server during the 70 hours budgeted for the experiments (task TE.2 in Table B.1), the total electricity consumed is on the order of $540 \times 0.20 + 70 \times 0.10 \approx 115$ kWh. Applying a representative Spanish grid emission

factor of approximately $0.19 \text{ kg CO}_2/\text{kWh}$ for 2023¹, the corresponding carbon footprint is on the order of $115 \times 0.19 \approx 21.9 \text{ kg CO}_2$ equivalent for the entire project execution. For context, 21.9 kg CO_2 is roughly the emissions of about two hours of driving an average car at 100 km/h ². This is an operational estimate and does not account for every indirect contribution to the project's overall carbon footprint.

Regarding PPP: Did you plan to minimize its impact, for example, by reusing resources?

Yes, minimizing the environmental footprint is a core architectural decision of this project. To achieve this, the framework is specifically designed to operate using small, locally hosted LLMs rather than energy-intensive commercial models. In addition, the code and the experimental test cases will be optimized to minimize the execution time and therefore the power consumption of the hardware.

Regarding Useful Life: How is currently solved the problem that you want to address?, and How will your solution improve the environment with respect to other existing solutions?

Currently, software bugs are primarily addressed through manual debugging. This is a tedious process that requires developers to spend countless hours running tests, analyzing the code and logs. While some modern automated solutions exist, they typically rely on proprietary, cloud-based LLMs, which simply shifts the environmental burden to massive data centers with enormous carbon footprints.

An APR framework along the lines explored here improves the environmental impact in two ways. First, by automating the bug-fixing lifecycle, it significantly reduces the hours developers must spend running their workstations at high capacity just to track down errors. Second, this work demonstrates that complex APR tasks can be orchestrated using locally hosted MAS, without cloud inference.

Environmental Risks: What scenarios could increase the project's ecological footprint?

One concrete scenario could increase the framework's ecological footprint during its useful life: the topology configuration allows cycles, so an LLM agent could theoretically enter an infinite loop and keep consuming tokens indefinitely, producing a runaway carbon footprint. The mitigation is the `EVOMAS_GRAPH_MAX_REVISITS` environment variable (Section 5.1), which caps the number of times any node may

¹ourworldindata.org.

²New EU cars averaged $106 \text{ g CO}_2/\text{km}$ in 2023 (EEA, [eea.europa.eu](https://www.eea.europa.eu)); $21.9/0.106 \approx 207 \text{ km}$.

be revisited and forces the workflow to terminate within a bounded number of super-steps.

7.4 Social Impact and Ethics

Regarding PPP: What do you think you will achieve in terms of personal growth from doing this project?

This project represents a significant step in my professional and personal development. By researching and building an APR framework, I will acquire deep knowledge in cutting-edge AI, specifically in the topologies of MAS. I will also gain practical command of automated bug-fixing techniques and of local LLM deployment. Beyond the technical skills, this project will strengthen my ability to manage a complex project and to reason about experimental results in a disciplined way.

Regarding Useful Life: How is currently solved the problem that you want to address?, and How will your solution improve the quality of life with respect to other existing solutions?

Currently, the problem of software bugs is solved primarily through manual debugging by human developers. This process is notoriously stressful and time-consuming, often leading to mental fatigue. While some AI coding assistants exist, they typically act as autocomplete tools rather than autonomous debuggers, still leaving complex problem-solving to the human.

The APR framework explored here, once matured into a production-grade tool, could significantly improve the quality of life for developers by functioning as a collaborative, intelligent assistant that handles the repetitive and frustrating aspects of bug location and patching. By shifting part of the tedious debugging workload to a MAS, developers could plausibly experience lower cognitive load and stress, freeing time to redirect toward more creative and high-value architectural tasks.

Regarding Useful Life: Is there a real need for the project?

Yes, there is a clear need for this project. Bug fixing is an essential, unavoidable part of software development: no matter how skilled a team is, all code will contain issues. As software systems become increasingly complex, the human capacity to manually maintain them is reaching its limit.

Furthermore, while the concept of using MAS for APR is popular, there is a specific need for a framework that supports flexible MAS topologies. Currently, it is not entirely clear which agent configuration is the most effective for fixing code. This

project addresses that exact problem by providing an adaptable framework that allows researchers to analyze, compare, and discover the best possible topology for APR.

Beneficiaries and affected groups: Who benefits from the project, and which collectives could be negatively affected?

The framework's beneficiaries are detailed in Section 1.1.4. Junior developers could be negatively affected, as their learning has traditionally relied on the debugging tasks that an APR pipeline automates. This is a general concern of AI-augmented software development, not specific to this framework. The appropriate response is a change in mindset that treats AI as a tool, not as a replacement for the developer role.

Ethical commitments shaping the framework.

Three ethical commitments shape *EvoMas*. **Attribution:** every borrowed agent prompt and tool carries a commit-pinned source link and is cataloged in the framework's `ACKNOWLEDGEMENTS.md` files. **Local execution:** LLM inference is served by a locally hosted Ollama instance, no user data is collected. **Research scope:** the framework is published as an open-source LLM application for AI researchers.

Social Risks: What scenarios could harm specific collectives or create user dependency?

The principal scenario that could harm the developers who adopt the framework is over-reliance on its output and the gradual loss of debugging skills. The mitigation is the explicit labeling of every output as a patch *suggestion* subject to human review.

7.5 Sustainability Matrix

The three preceding sections can be condensed into the sustainability matrix, which crosses the three impact dimensions (economic, environmental, social) with the three life-cycle stages (project and planning, useful life, and risks) shown in Table 7.2.

7.6 Identification of Laws and Regulations

Several legal and normative aspects are potentially relevant for a project that handles source code, prompts, and the outputs of LLMs. Each is discussed below together with the way the project complies with it.

Table 7.2: UPC sustainability matrix for the *EvoMas* project. Each cell condenses the reflection developed in Section 7.2, Section 7.3, and Section 7.4. The PPP column refers to the project-and-planning stage, the useful-life column to the deployed framework, and the risks column to the scenarios that could degrade each dimension.

	Project / Planning (PPP)	Useful life	Risks
Economic	Human-resource-dominated budget of 22,565.18€; material costs kept low through open-source tooling.	Near-zero operational cost from local execution; cuts the manual-debugging budget for users.	License change in a dependency; mitigated by relying only on permissively licensed projects.
Environmental	Operational footprint of about 21.9 kg CO ₂ ; reuse of existing personal hardware.	Local Ollama inference avoids energy-intensive cloud API calls.	Runaway token loops in cyclic topologies; capped by the <code>EVOMAS_GRAPH_MAX_REVISITS</code> limit.
Social	Personal growth in MAS and APR research; open and attribution-respecting development.	Potential reduction of repetitive debugging workload for developers; fully local, no user data collected.	Over-reliance and loss of debugging skills; mitigated by taking outputs as patch suggestions.

- **Open-source licensing of the analyzed repositories.** The empirical analysis covered repositories from the *SWE-bench* leaderboard, which carry a mix of licenses. Some of these repositories are released under restrictive terms, and a few do not include any explicit license file, in which case the default position is "all rights reserved". To avoid any license-compatibility issue when building the framework, no source code, prompt template or tool implementation has been copied from those repositories. Every agent, prompt and tool used inside the framework is a full reimplementaion from scratch, written only on the basis of the high-level description of the behavior observed during the analysis. Citations and bibliographic references to the original repositories are nevertheless given throughout the thesis so that the inspiration is properly attributed.
- **License of the framework produced by this project.** The framework will be released under the MIT license. This choice keeps the project aligned with the permissive-licensing tradition of most academic Python tooling, allows third parties to reuse the framework in both academic and commercial contexts, and imposes only the standard attribution requirement.
- **Privacy and data protection (GDPR / RGPD).** The framework does not process personal data, and because *EvoMas* builds on a pre-trained model served

through Ollama rather than training one, it has no machine-learning data pipeline of its own. Prompts and source code handled by the agents are software artifacts, not personal data in the sense of Regulation (EU) 2016/679 (GDPR). The pipeline runs entirely on the local machine: the LLM back-end is a local Ollama instance reachable only on the local network through its localhost API, and source code never leaves the local environment. The only point at which the system contacts the public internet is when it performs `git clone` of the GitHub repositories used to reproduce *SWE-bench* instances (or user-supplied custom instances). This is a read-only access to publicly available repositories, with no upload of user data and no transmission of personal information, so it does not trigger GDPR obligations beyond the standard ones already met by GitHub.

- **EU AI Act (Regulation 2024/1689).** At the date of this report, the regulation is in force. The framework developed in this thesis is a research artifact, produced in an academic context, not placed on the market and not put into service in the sense of Article 3 of the Regulation. It is therefore covered by the research exception and does not, on its own, qualify as a general-purpose AI system or as a high-risk AI system under the Act. Should the framework later be packaged and distributed as a product, a dedicated classification exercise will have to be carried out by whoever performs that distribution.
- **FIB and UPC academic regulations.** This work is developed under the *Normativa del treball de fi d'estudis* of the Facultat d'Informàtica de Barcelona (Universitat Politècnica de Catalunya), which governs the structure, authorship and defense of the thesis. The project complies with these regulations: it is a single-author work supervised by the directors designated by the Faculty, follows the official calendar, and uses third-party material only with explicit citation. LLM chatbots were used as writing and code-completion aids; all engineering decisions and final wording were those of the author.

Chapter 8

Conclusions

This thesis investigated the intersection of **Multi-Agent Systems** and **Automated Program Repair** in two parts. Part I (Chapter 3) characterized how APR solutions submitted to the *SWE-bench* leaderboard are actually constructed. Part II (Chapter 4 and Chapter 5) used the empirical findings of Part I to design and implement *EvoMas*, an evolutionary MAS framework for APR that compiles a JSON topology configuration into a LangGraph workflow, evaluated on the *SWE-bench* harness in Chapter 6. This chapter summarizes the research outcomes (Section 8.1), the degree of satisfaction of the original sub-objectives and system requirements (Section 8.2), the open directions for future work (Section 8.3), and a short personal reflection on the project (Section 8.4).

8.1 Research Outcomes and Contributions

This thesis advances understanding of how multi-agent APR systems are designed and orchestrated in practice, by analyzing *SWE-bench* leaderboard solutions (Chapter 3). It also delivers *EvoMas*, an open-source framework that serves as a research tool for the same field, which allows other researchers to reconfigure agents, swap models, and replay the experiments of Chapter 6 without modifying the core source code.

The contributions are the following:

1. A structured analysis of the 48 candidate APR solutions on the *SWE-bench* leaderboard, recorded as CSV files under `TFG_public/First_Part--Analysis/`.
2. The open-source code of *EvoMas*, a MAS framework for APR, released at `xiangfengyepan/TFG_public/EvoMas-master` under the MIT license.

3. A collection of experimental results obtained with locally hosted LLMs, with auto-generated per-run reports released under `EvoMas-master/experiments/` and `EvoMas-master/examples/`.

8.2 Degree of Achievement of the Objectives

The seven sub-objectives defined in Section 1.3.1 are revisited in Table 8.1. The satisfaction of every functional and non-functional requirement defined in Section 4.2 is summarized separately in Table 8.2.

Table 8.1: Degree of achievement of the four empirical-analysis and three framework-development sub-objectives defined in Section 1.3.1.

ID	Sub-objective	Status
<i>Empirical analysis</i>		
EA-1	Analyze agent-based APR systems in selected <i>SWE-bench</i> leaderboard repositories.	Accomplished with limitations
EA-2	Identify and classify the agents involved in these systems by role and responsibility.	Accomplished with limitations
EA-3	Evaluate the level of autonomy and coordination within the analyzed agent systems.	Accomplished with limitations
EA-4	Discover topologies and compare advantages and disadvantages of each.	Accomplished with limitations
<i>Framework development</i>		
FD-1	Design a MAS framework supporting topological evolution and flexible agent orchestration.	Partially accomplished
FD-2	Ensure the framework can solve the problem of APR.	Accomplished
FD-3	Test and compare different topologies and agent configurations on the <i>SWE-bench</i> harness.	Accomplished

Even though the analyzed repositories are open-source, several of them publish only part of their original code. This affects EA-1, where a complete trace of the agents cannot be reconstructed, and EA-3, where the routing logic that drives autonomy is not always fully visible. The limitation for EA-2 lies in the ambiguity of the concept of **agent**. EA-4 is bounded by the limitations of the three previous EA-x sub-objectives.

Turning to framework development, FD-1 is partially accomplished. Flexible orchestration is delivered through the JSON topology configuration, while runtime topological evolution is left as future work in Section 8.3. FD-2 is fully accomplished, since the framework resolves real-world *SWE-bench* instances on local

models (Section 6.3.4). FD-3 is also fully accomplished, as demonstrated by the five experimental subsections of Chapter 6.

Table 8.2: Satisfaction status of the seven functional and seven non-functional requirements defined in Section 4.2.

ID	Requirement	Status
<i>Functional</i>		
FR-1	Bug Ingestion	Satisfied
FR-2	Fault Localization	Satisfied
FR-3	Patch Generation	Satisfied
FR-4	Patch Validation	Satisfied
FR-5	Workflow Construction	Satisfied
FR-6	Flexible Topology Configuration	Satisfied
FR-7	Flexible Agent Management	Satisfied
<i>Non-functional</i>		
NFR-1	Modularity	Satisfied
NFR-2	Scalability	Satisfied
NFR-3	Performance	Satisfied
NFR-4	Usability	Satisfied
NFR-5	Budget	Satisfied
NFR-6	Privacy and Security	Satisfied
NFR-7	Hardware Compatibility	Satisfied

On the requirements side, the *EvoMas* framework covers all seven functional requirements, as proved in Chapter 6. For the non-functional ones, Modularity and Scalability are demonstrated by the non-APR extensibility experiments of Section 6.4, where the same framework is reconfigured for tasks outside program repair without any internal code change. Performance is demonstrated by the experimental results of Chapter 6. Usability is partially satisfied through the JSON-driven configuration and the frontend UI. A proper validation requires a user-experience survey that is left as future work in Section 8.3. Budget is discussed in Section 7.2, Privacy and Security in Section 7.6, and Hardware Compatibility is documented in Table 6.5.

8.3 Future Work

Four directions remain open for future work, each addressing a concrete limitation identified during development.

Runtime topology evolution. The framework’s name reflects an intent that the current implementation fulfills only in part. The JSON-driven configuration of

EvoMas already provides the data-side abstraction. The missing piece is the ability to reconfigure the topology at runtime, which LangGraph does not natively support. Achieving runtime evolution would require switching to a different graph library, for example AutoGen or a custom dispatcher built on `asyncio` and `NetworkX`.

Leaderboard submission with a frontier LLM. The Performance non-functional requirement was kept deliberately non-strict because of hardware limitations, which capped the largest locally deployable model to `qwen3.5:9b`. Better performance is expected from frontier-class LLMs, and re-running the experiments of Chapter 6 under a model such as Claude Opus, GPT-4.x, or Gemini Pro would allow comparing *EvoMas* against the other *SWE-bench* leaderboard solutions on equal terms.

Scope beyond APR. The two non-APR demonstrations included in Chapter 6 show that the framework is not tied to program repair. A natural extension is therefore to extend *EvoMas* into a general-purpose MAS platform.

User-experience study. Usability has not yet been empirically validated. A study in which representative users configure and run the framework without prior codebase familiarity, followed by a structured survey, would provide concrete feedback for future improvement.

8.4 Personal Reflection

This thesis was my first independent end-to-end research project; completing it within the planned schedule required careful scope management. The 540-hour effort and the EUR 22,565.18 budget were both respected; the only schedule change was the framework prototype discussed in Section 1.3.3.

The amount of compute, memory, and time that hosting a local LLM actually consumes came as a surprise to me. That constraint shaped the model size and the evaluation scope of this thesis, and it is a cost that remains hidden when one relies on cloud-hosted models accessed through an API. Going forward, I will carry this knowledge into any future LLM-based project.

If I were to start the thesis again, I would exploit rather than explore the repositories of Part I, going deeper into a selected subset of promising candidates. The

trade-off would be less variability across the leaderboard and more concrete insights per repository.

Appendix A

Glossary

The following terms are used throughout this thesis. Each entry provides a formal definition; the section in which the term first appears in context is cited in parentheses.

Definition (*Agent*): An autonomous computational entity capable of perceiving its environment, reasoning independently, and using tools to achieve a goal. In the context of MAS for software engineering, an agent may invoke LLMs, execute shell commands, read or modify source files, and communicate with other agents (Section 1.1.1).

Definition (*Automated Program Repair (APR)*): A field of software engineering concerned with automatically localizing and repairing defects in source code. APR techniques aim to reduce the time and manual effort required for software maintenance by producing valid patches that make failing tests pass without breaking previously passing ones (Section 1.1.1).

Definition (*Autonomy*): The degree to which agents in a system determine their own execution sequence, rather than relying on external software to orchestrate them. Three levels are distinguished: **orchestrated** (a control program explicitly sequences agent invocations); **semi-autonomous** (agents decide the sequence subject to external constraints); and **autonomous** (agents collectively determine the full execution sequence without external direction) (Section 1.1.2).

Definition (*Framework*): A structured software library that provides reusable agent abstractions, orchestration primitives, and integration points for building LLM-based workflows. In this thesis, the relevant frameworks are those designed for constructing LLM-based agent workflows, such as LangChain and LangGraph (Section 1.1.2).

Definition (*Issue / Bug*): A software problem that causes incorrect or unintended behavior. An *issue* can be a syntax error, a logical error, a performance problem, or any defect affecting the correct functionality of the software. A *bug* specifically denotes a defect that produces a measurable failure, typically captured by one or more failing test cases (Section 1.1.2).

Definition (*Large Language Model (LLM)*): A neural network trained on large corpora of text that can generate human-like responses conditioned on an input prompt. LLMs underpin the reasoning capabilities of agents in modern APR systems (Section 1.1.1).

Definition (*LLM-based System*): A general class of software systems that incorporates at least one LLM as a core reasoning component. The class spans three subcategories: **agent-less** systems, which invoke LLMs directly without an encapsulating agent abstraction; **single-agent** systems, in which a single autonomous entity wraps one or more model calls; and **multi-agent** systems (MAS), in which several agents collaborate. This thesis focuses primarily on the MAS subcategory, although the empirical analysis also encounters agent-less and single-agent designs (Section 1.1.2).

Definition (*Multi-Agent System (MAS)*): A system composed of multiple interacting agents, each with its own decision-making capability, collaborating to accomplish a shared objective. In the APR setting, agents typically divide the repair workflow among specialized roles such as fault localization, patch generation, and validation (Section 1.1.1).

Definition (*Prompt*): The input provided to an LLM in order to elicit a response. Three types are distinguished: **system** prompts establish the model's role and behavioral context; **proxy** prompts relay task-specific instructions on behalf of a tool or calling module; and **human** prompts carry the concrete task description. A fuller treatment is given in Section 2.1.2.

Definition (*Topology*): The connectivity structure of a MAS, describing which agents can communicate directly with which others. Topology is formally represented as a directed graph $G = (V, E)$, where vertices V are agents and edges E are communication channels. Common topology classes include **centralized** (star), **decentralized** (multi-hub), **distributed** (mesh, ring, chain), **hierarchical** (tree), **hybrid**, and **dynamic**; each is treated in detail in Section 2.2.1.

Appendix B

Supplementary Figures and Tables

Table B.1: Project task breakdown across the five role groups with estimated effort in hours, with sub-tasks listed under each role and totals reported per role and in aggregate (540 h).

Code	Task	Hours
PM	Project Management	95
PM.1	Scope Definition	25
PM.2	Planning	15
PM.3	Budget & Sustainability	15
PM.4	Documentation	20
PM.5	Weekly Meetings	20
AR	Analysis of Repositories	115
AR.1	Repository Analysis	50
AR.2	Agent Analysis	50
AR.3	Topology Analysis	15
FC	Framework Construction	160
FC.1	Framework Design	40
FC.2	Framework Implementation	120
TE	Tests and Experiments	100
TE.1	Framework Test Cases	30
TE.2	Experiments	70
FD	Final Documentation	70
FD.1	Thesis Writing	60
FD.2	Presentation	10
Total		540

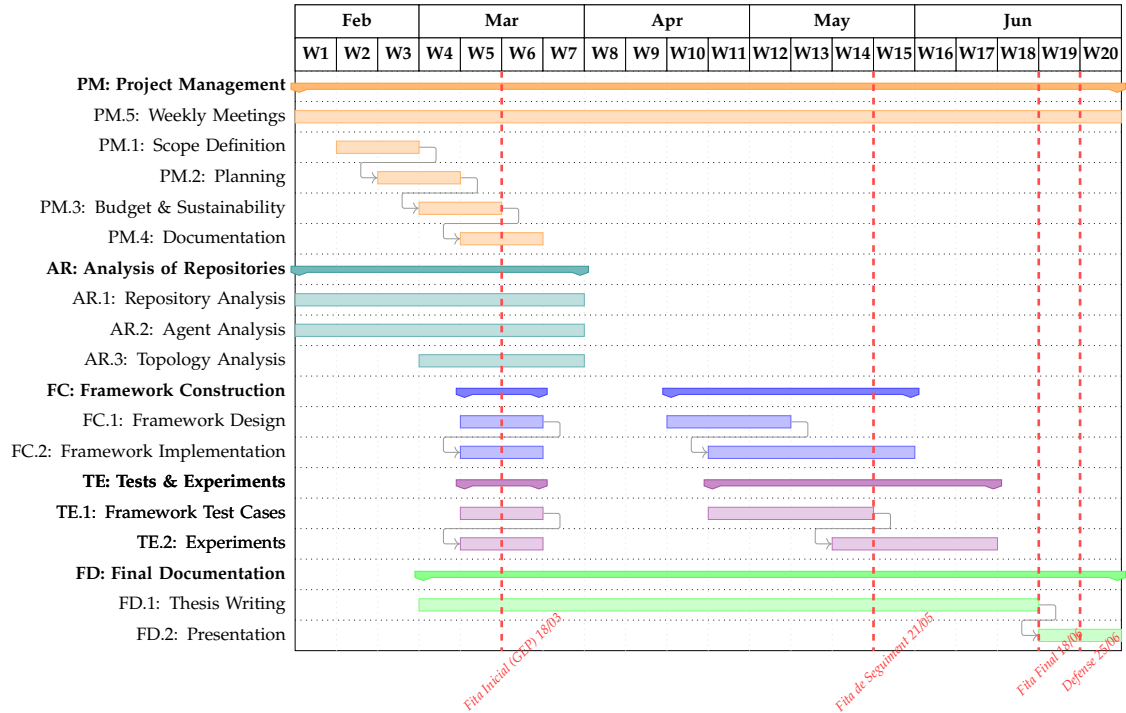


Figure B.1: Gantt chart of the project temporal planning (10 Feb – 23 Jun 2026). Colors identify work packages: orange (PM), teal (AR), blue (FC), violet (TE), green (FD). Arrows indicate finish-to-start dependencies as defined in the project plan. Dashed red lines mark the four formal milestones.

Table B.2: Generative AI tools used during the development of this thesis, grouped by application, with the tools used for each and their overall contribution. All output produced with these tools was reviewed, verified, and edited by the author, who takes full responsibility for the final content. The local `qwen` models served through Ollama are not listed here, as they are the object of study of the experiments in Chapter 6 rather than a development tool.

Application	AI tools	Contribution
Thesis writing	Claude / Claude Code (Anthropic), ChatGPT (OpenAI), Gemini Pro (Google)	Accelerated drafting and revision of chapters, tables, and figures in $\text{\LaTeX}$ format, and improved the clarity and correctness of the English prose.
Framework and script implementation	Claude / Claude Code (Anthropic), GitHub Copilot	Supported faster implementation and refactoring, generated unit tests, and helped with debugging.
Conceptual exploration and research	ChatGPT (OpenAI), Gemini Pro (Google)	Helped to understand complex MAS and APR concepts and locate relevant sources for the bibliography.

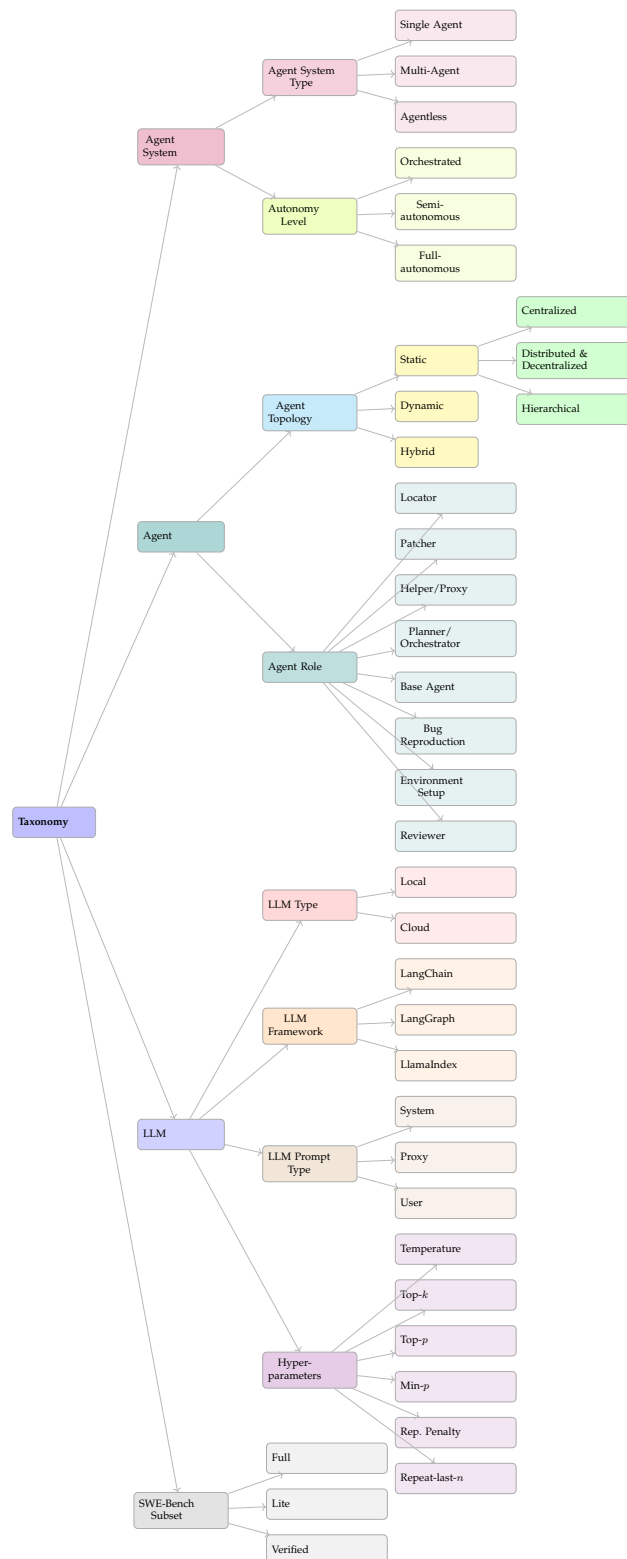


Figure B.2: Nine-branch taxonomy of the MAS analysis dimensions used in this thesis, covering agent roles, topology structures, autonomy levels, prompt stylistic patterns, LLM frameworks, data lifecycle, architecture, deployment, and evaluation. All branches are grounded in the empirical findings of Chapter 3.

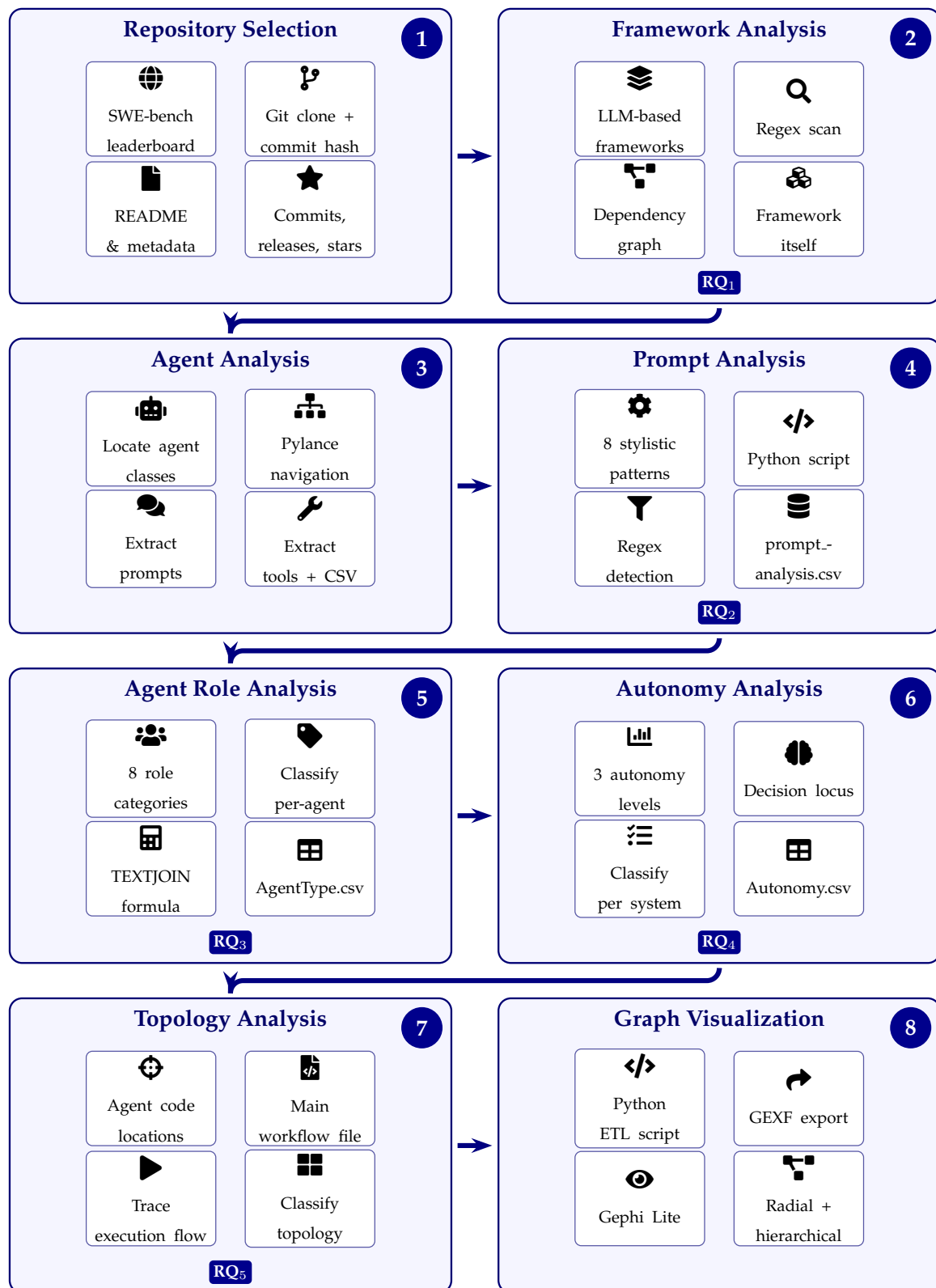


Figure B.3: Overview of the empirical study research process. The eight numbered phases are applied uniformly to each analyzed repository. Phases 4, 5, 6, and 7 all consume the per-agent data produced by Phase 3; Phase 8 (graph visualization) integrates outputs from all preceding phases.

```

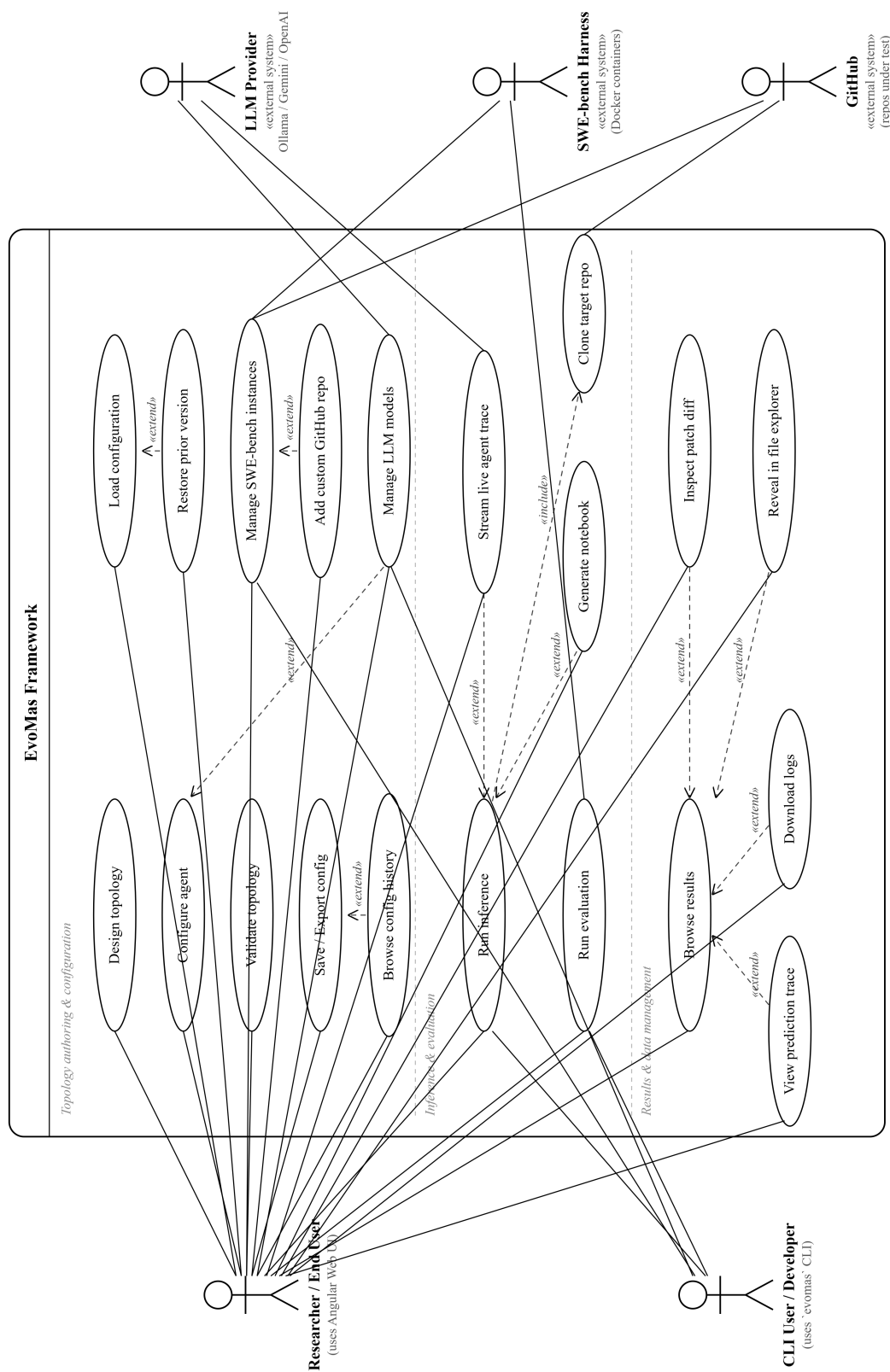
1 diff --git a/intro.md b/intro.md
2 index 516e82c..4857c7d 100644
3 --- a/intro.md
4 +++ b/intro.md
5 @@ -1,18 +1,18 @@
6 -# Welcome to EvoMas
7 +# Bienvenido a EvoMas
8
9 -EvoMas is an evolutionary multi-agent framework for program repair. It composes a small
10 ↪ graph of specialized LLM agents (locator, patcher, reviewer, finalizer) that pass a
11 ↪ problem statement through a chain of refinements until the workspace contains a
12 ↪ patch that fixes the bug.
13 +EvoMas es un marco multiagente evolutivo para reparación de programas. Componen un
14 ↪ pequeño grafo de agentes LLM especializados (locator, patcher, reviewer, finalizer)
15 ↪ que pasan una declaración de problema a través de una cadena de refinamientos hasta
16 ↪ que el espacio de trabajo contiene un parche que arregla el error.
17
18 -## Why a graph
19 +## ¿Por qué un grafo?
20
21 -A single monolithic prompt asking one model to "find the bug and fix it" leaves no room
22 ↪ for the model to specialize. Each agent in the graph has a tighter remit:
23 +Un solo prompt monolítico que pide a un modelo "encontrar el error y arreglarlo" no
24 ↪ deja espacio para que el modelo se especialice. Cada agente en el grafo tiene un
25 ↪ mandato más estrecho:
26
27 -- The locator narrows down the candidate files.
28 -- The patcher proposes a unified diff.
29 -- The reviewer inspects the diff for obvious mistakes.
30 -- The finalizer emits the diff that the harness will evaluate.
31 +- El locator reduce el conjunto de archivos candidatos.
32 +- El patcher propone un unified diff.
33 +- El reviewer inspecciona el diff por errores obvios.
34 +- El finalizer emite el diff que el harness evaluará.
35
36 -Smaller scopes mean smaller prompts, fewer hallucinations, and better calibration of
37 ↪ the model's strengths.
38 +Ámbitos más pequeños significan prompts más pequeños, menos alucinaciones y una mejor
39 ↪ calibración de las fortalezas del modelo.
40
41 -## Running
42 +## Ejecución
43
44 -Install the project, set up a virtual environment, and launch the EvoMas API server.
45 ↪ Then point a notebook or the integration test at a SWE-bench instance, the framework
46 ↪ handles the rest.
47 +Instale el proyecto, configure un entorno virtual y inicie el servidor de la API de
48 ↪ EvoMas. Luego, dirija un cuaderno o la prueba de integración hacia una instancia de
49 ↪ SWE-bench, el marco se encarga del resto.

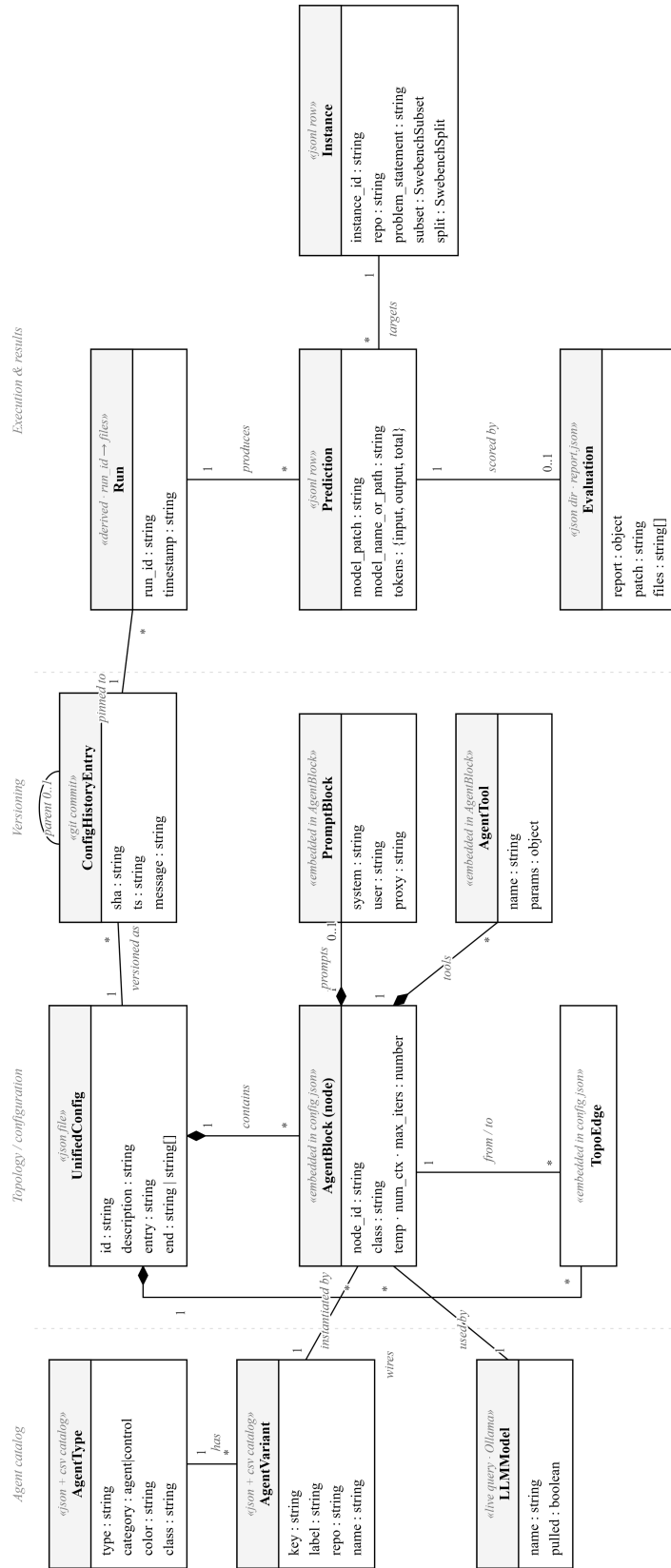
```

Figure B.4: Translation-demo diff: `intro.md`, English to Spanish. Output of the four-agent chain on the `translate-intro-en-es` instance; the headings, list markers, and inline-code tokens (locator, patcher, etc.) are preserved verbatim while the prose is translated.

Appendix C

UML Diagrams of the *EvoMas* Framework

Figure C.1: Use case diagram of the *EvoMas* framework.

Figure C.2: Conceptual data model of the *EvoMas* framework.

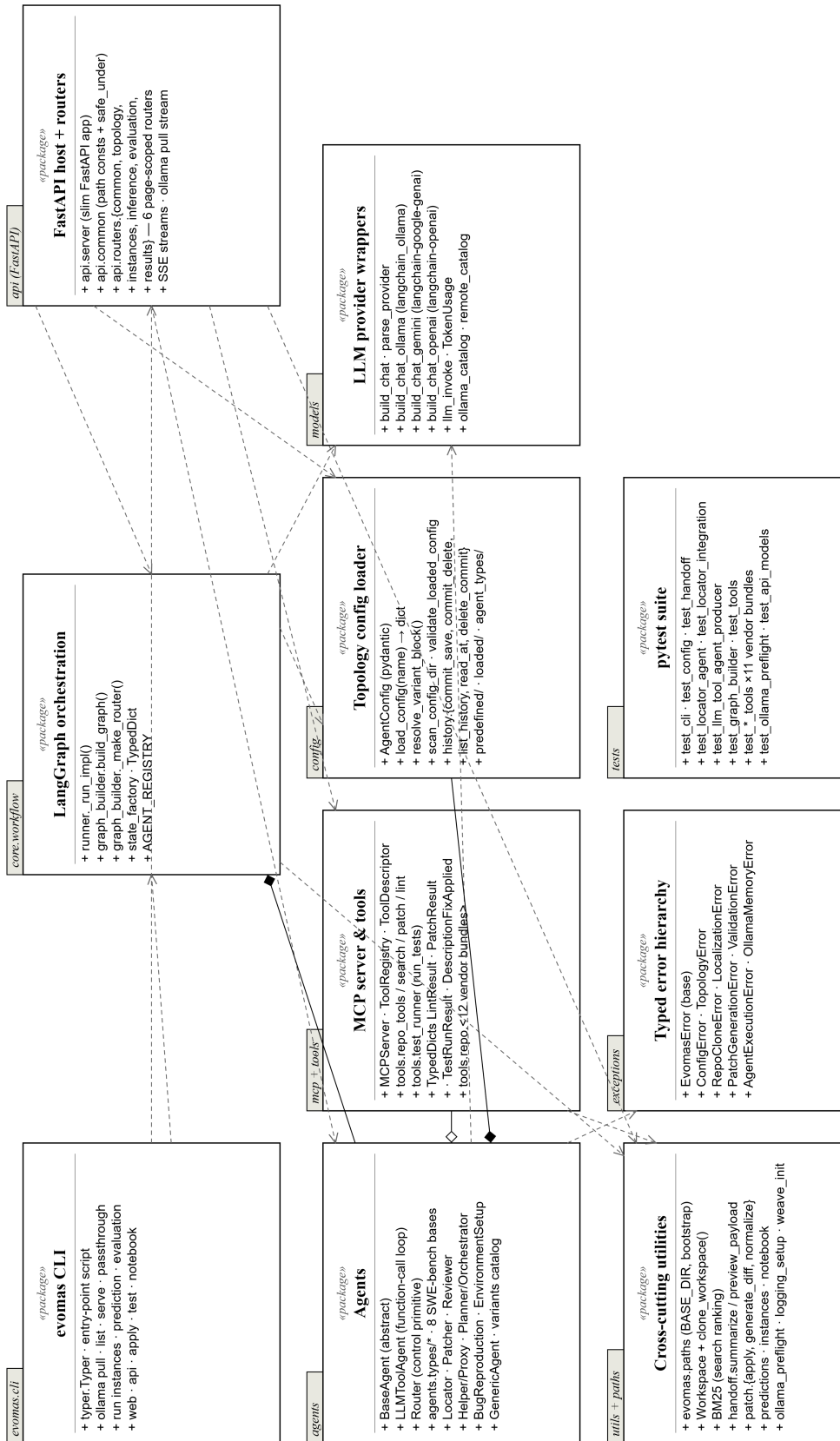
Figure C.3: Backend level-1 architecture of the *EvoMas* framework.



Figure C.4: Backend level-2 view of the agent hierarchy.

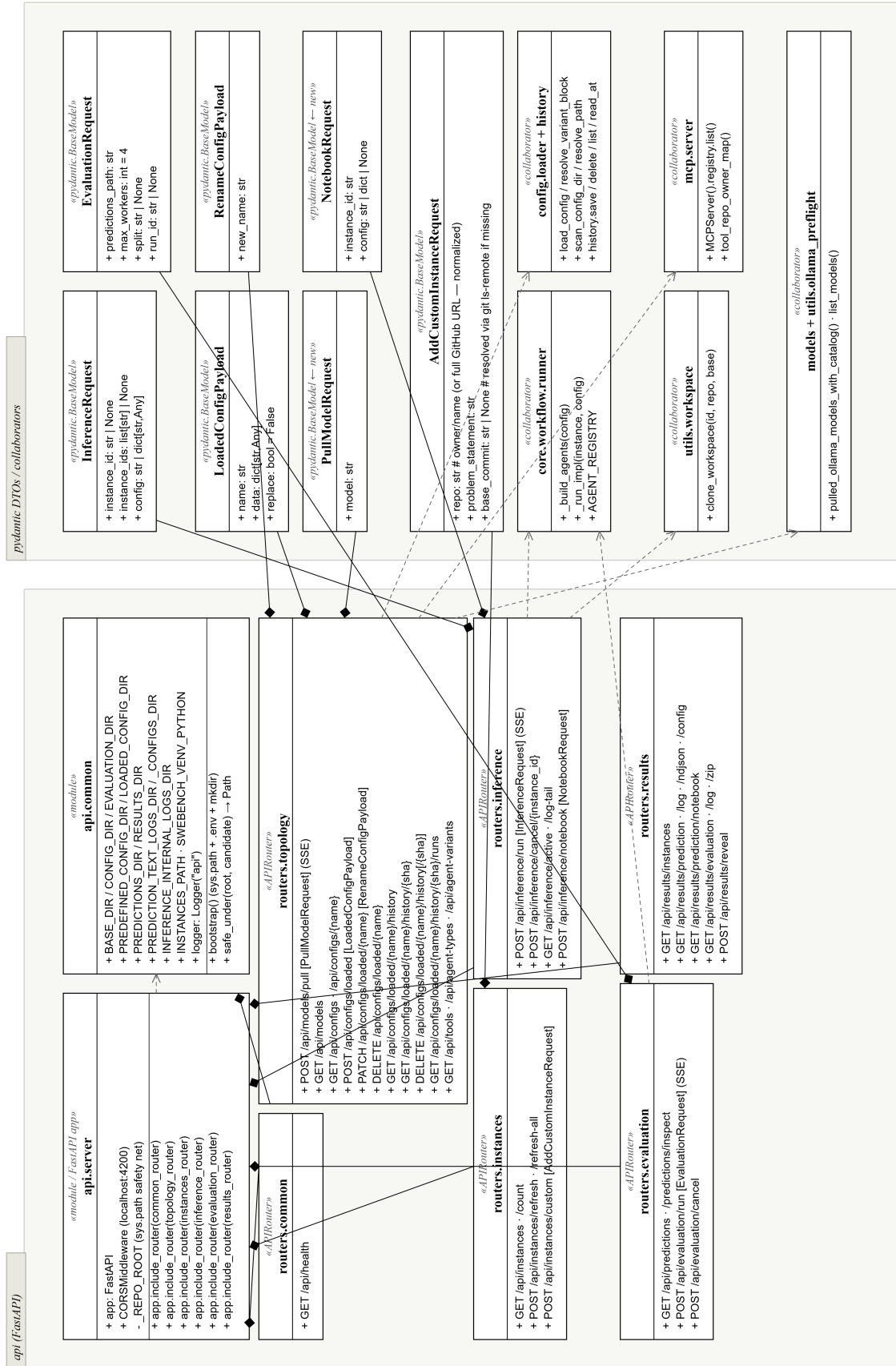


Figure C.5: Backend level-2 view of the FastAPI application.

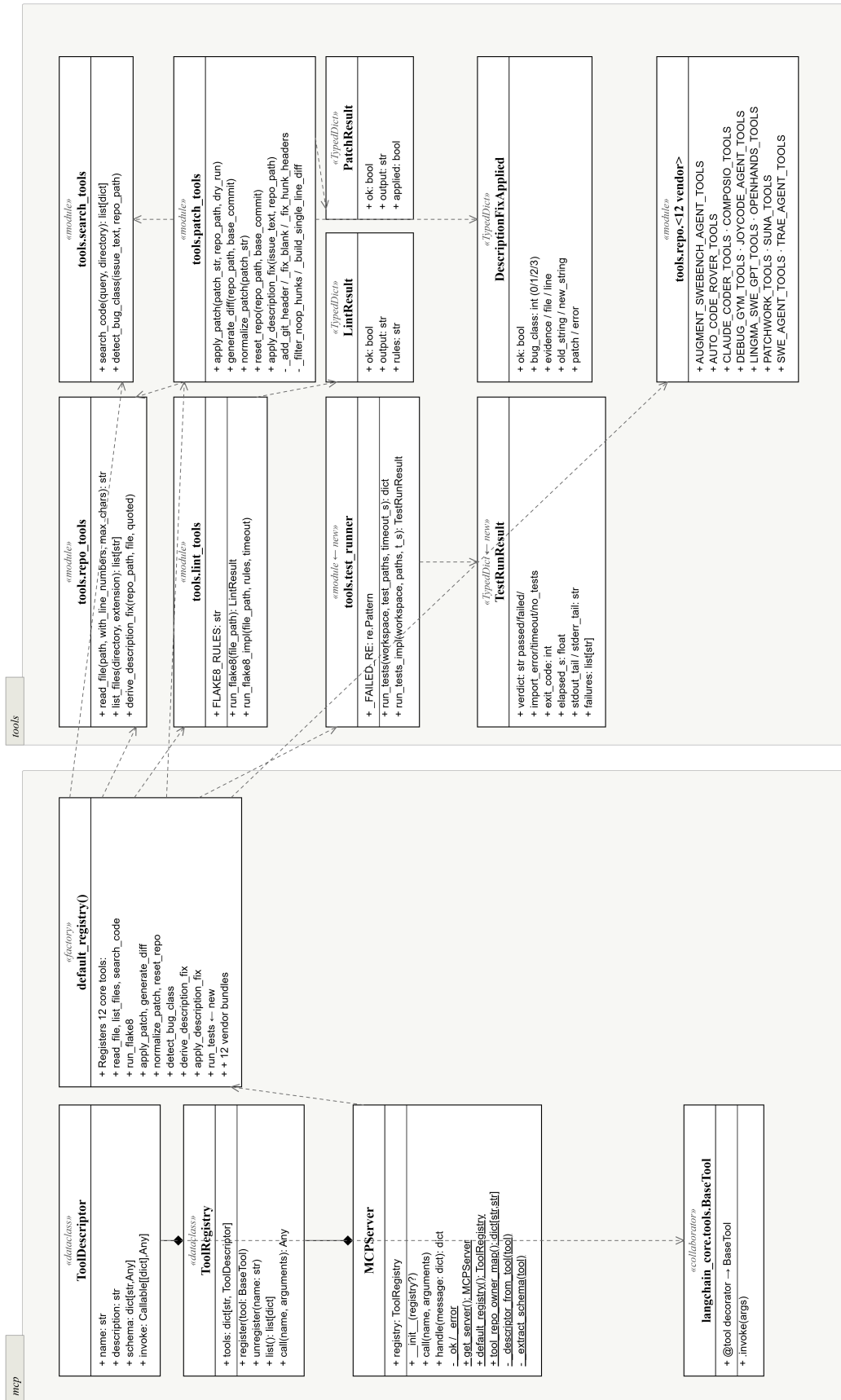
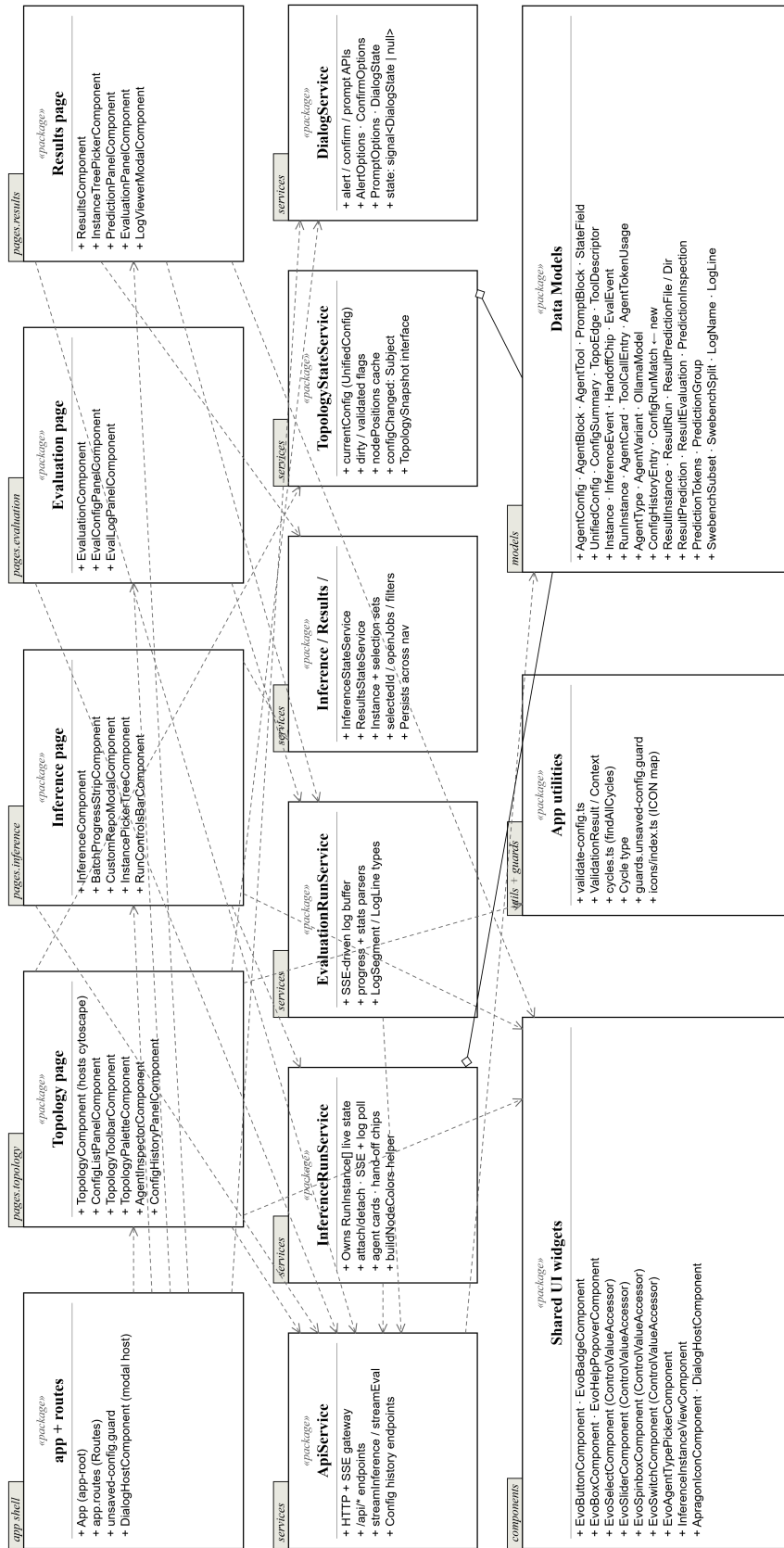


Figure C.6: Backend level-2 view of the MCP server and tool layer.

Figure C.7: Frontend level-1 architecture of the *EvoMas* framework.

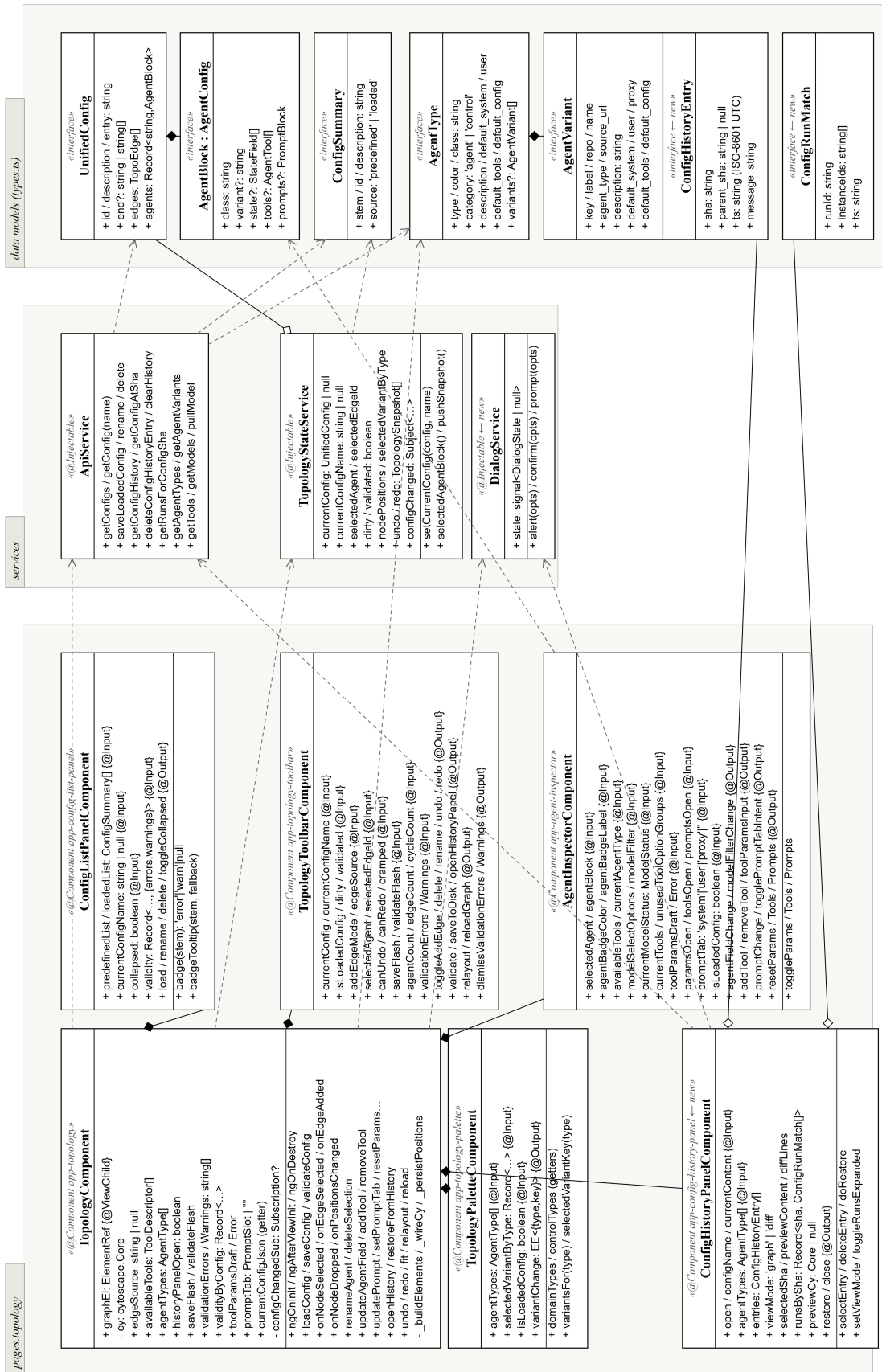


Figure C.8: Frontend level-2 view of the Topology page.

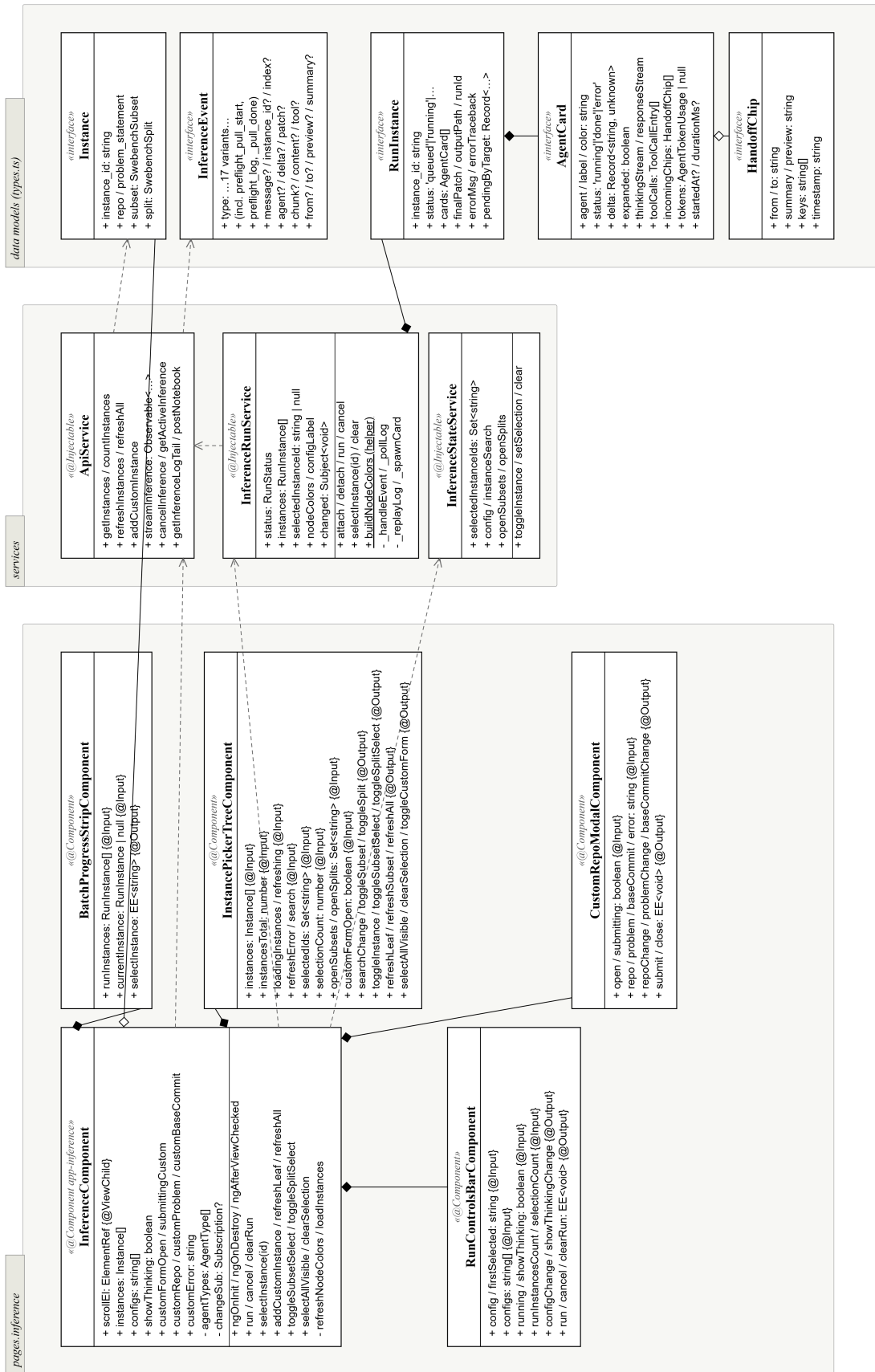


Figure C.9: Frontend level-2 view of the Inference page.

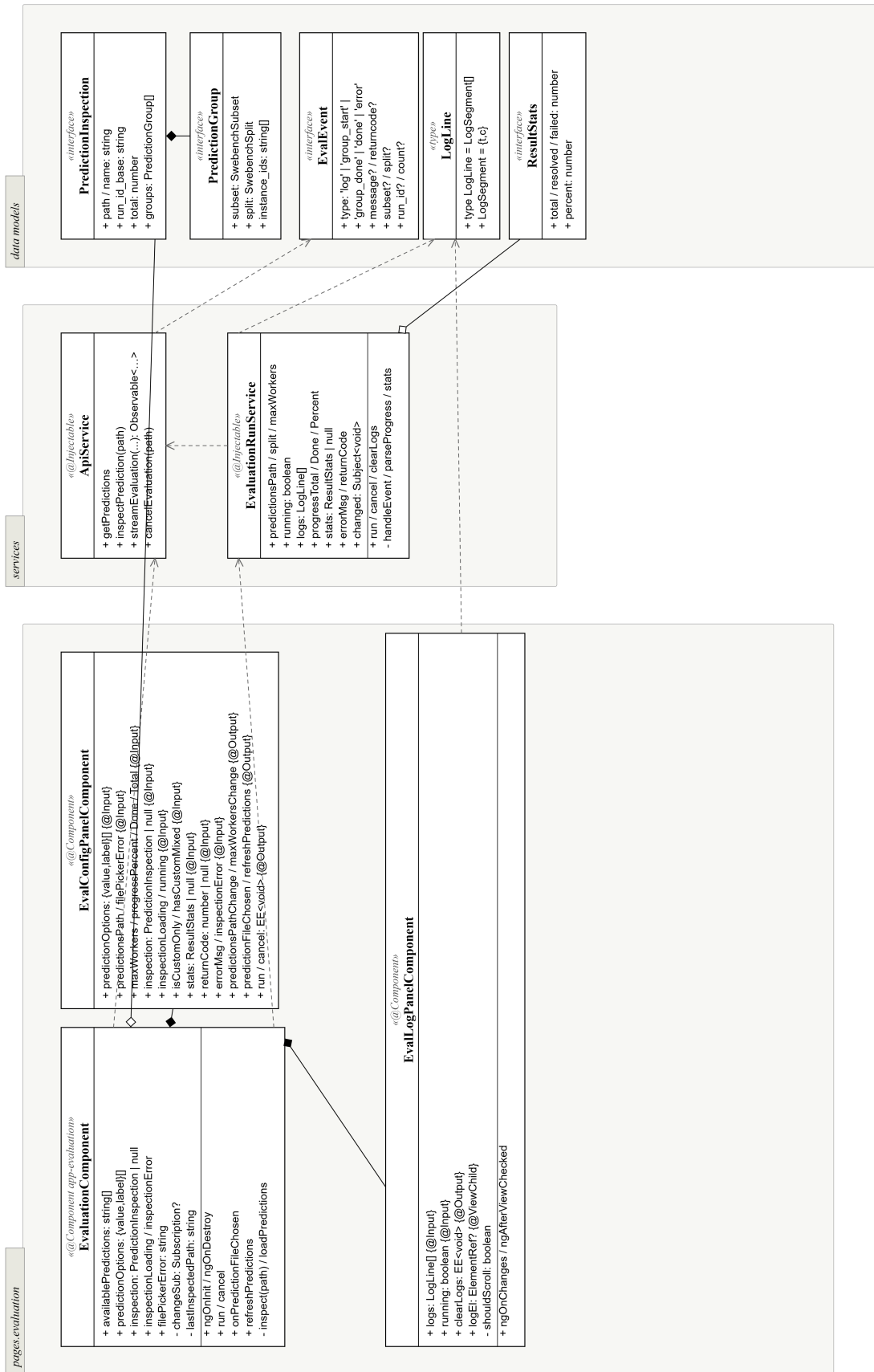


Figure C.10: Frontend level-2 view of the Evaluation page.

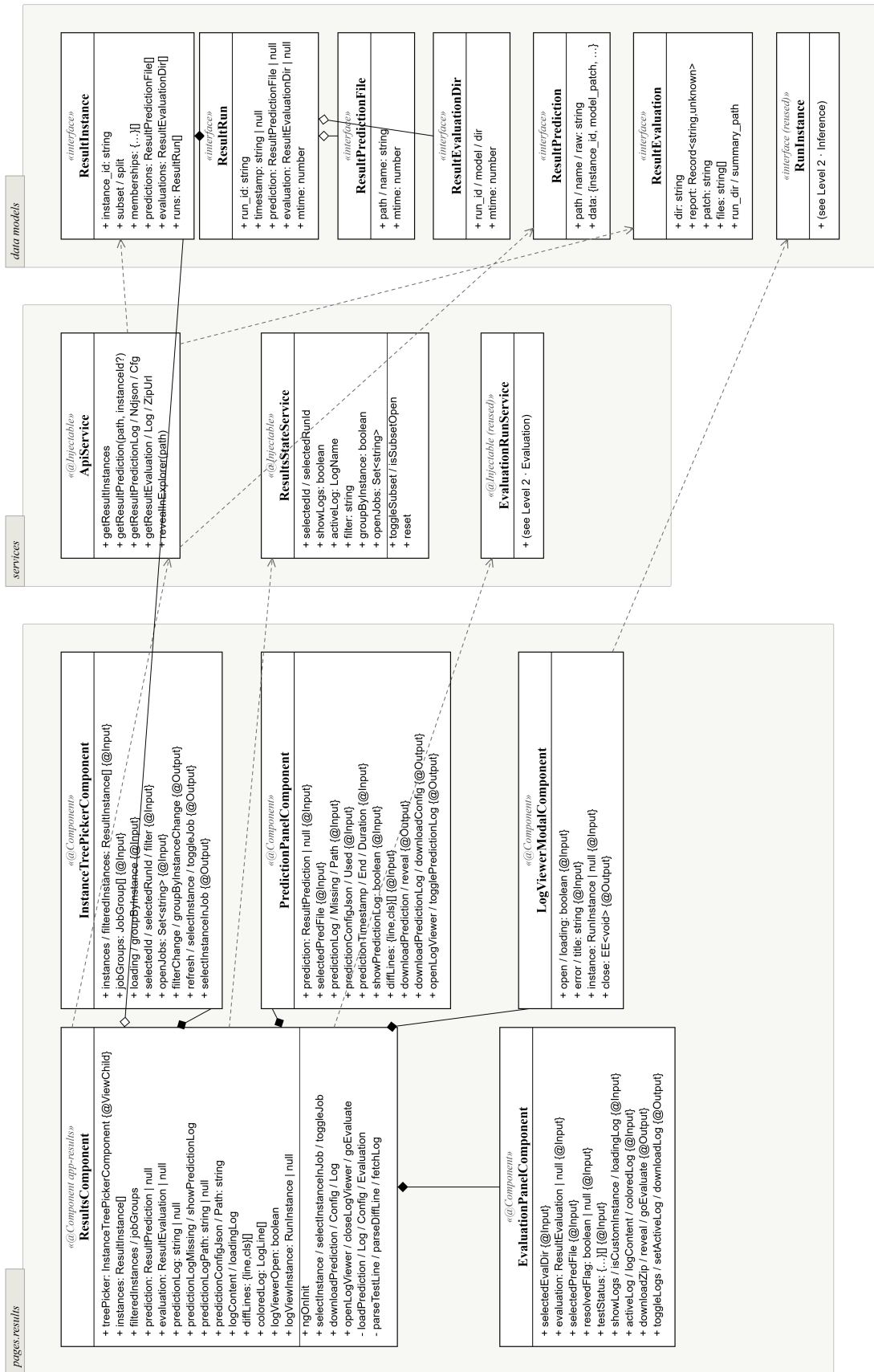


Figure C.11: Frontend level-2 view of the Results page.

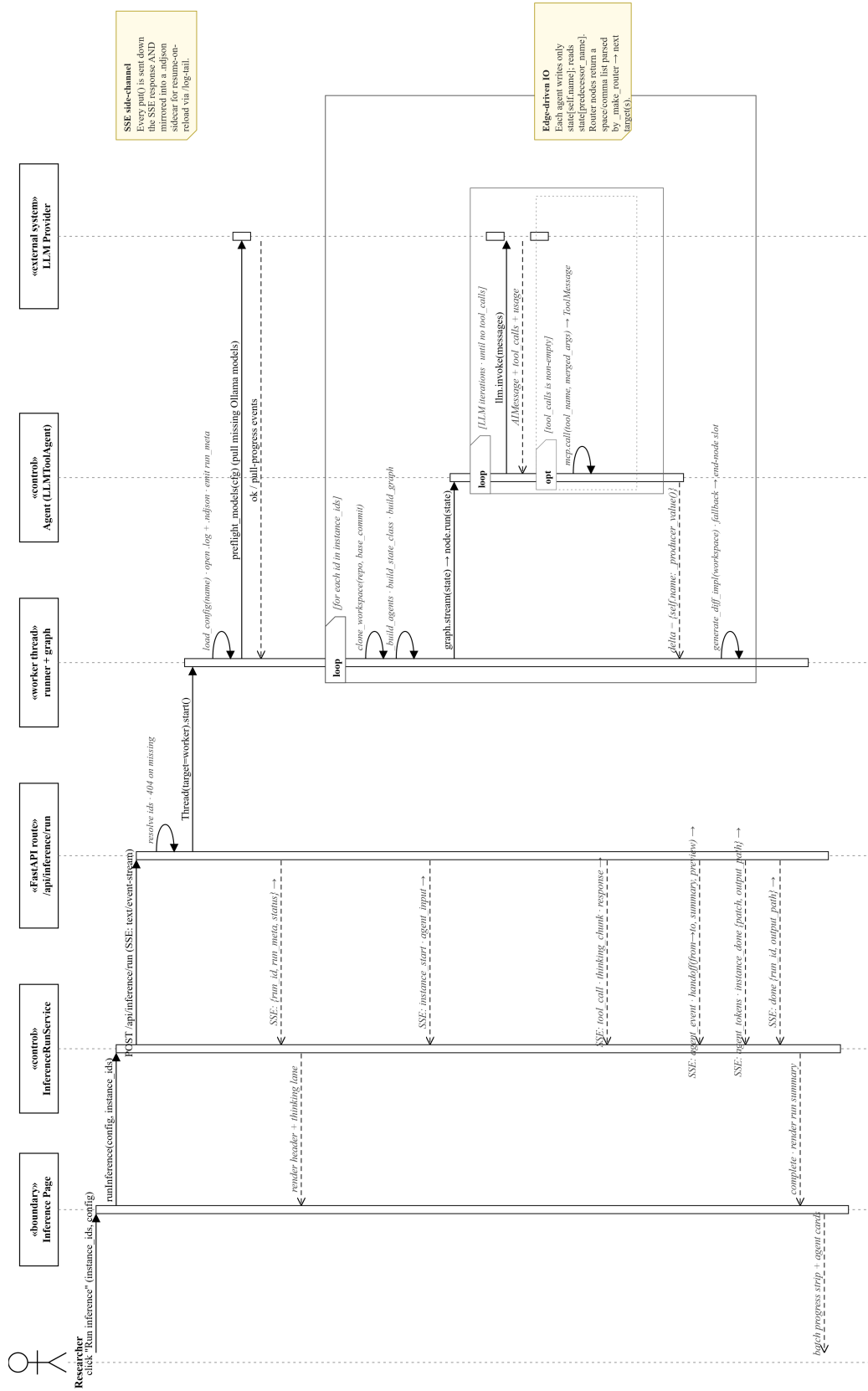
Figure C.12: Sequence diagram of the *Run Inference* use case.



Figure C.13: Sequence diagram of the *Run Evaluation* use case.

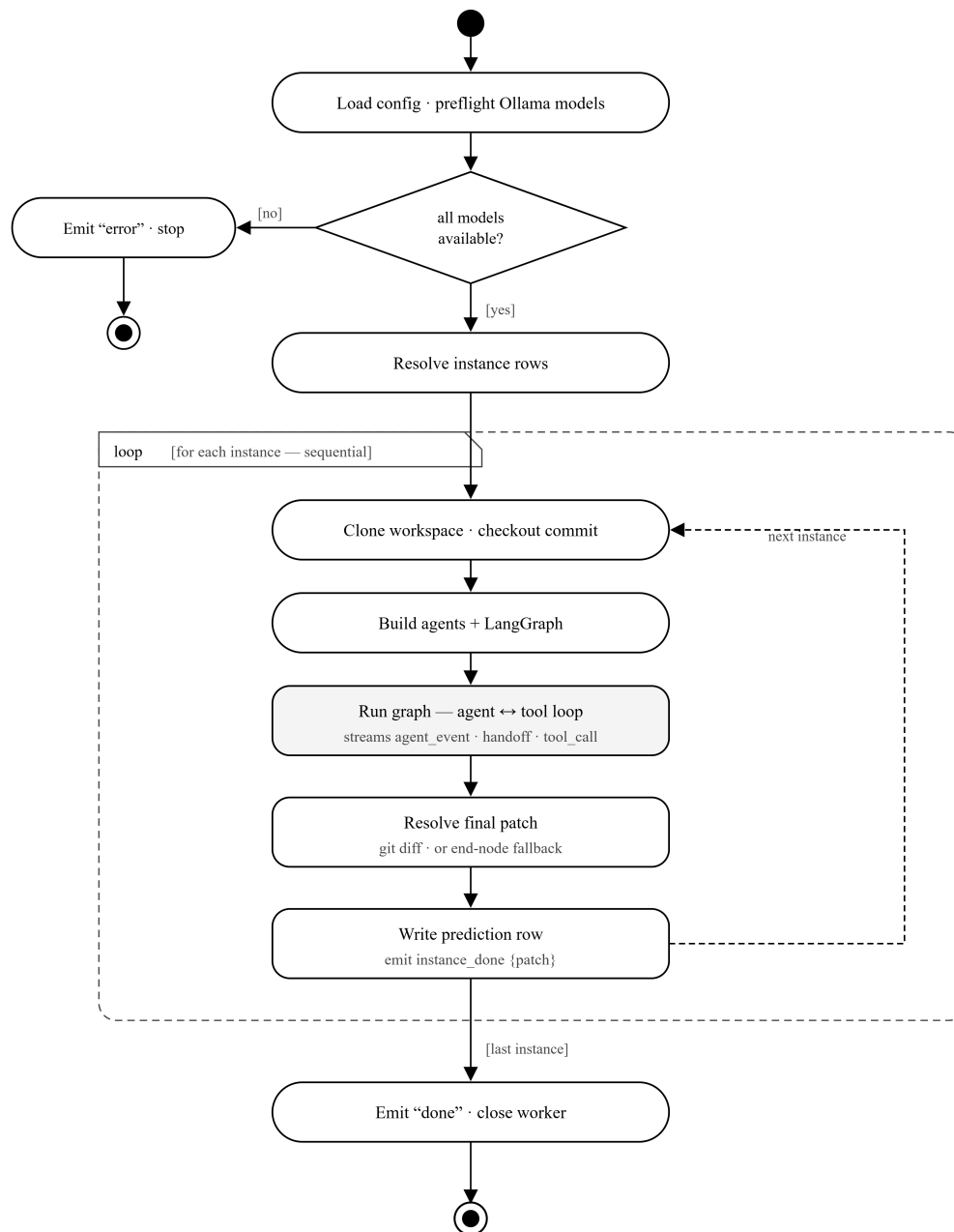


Figure C.14: Activity diagram of the backend repair pipeline.

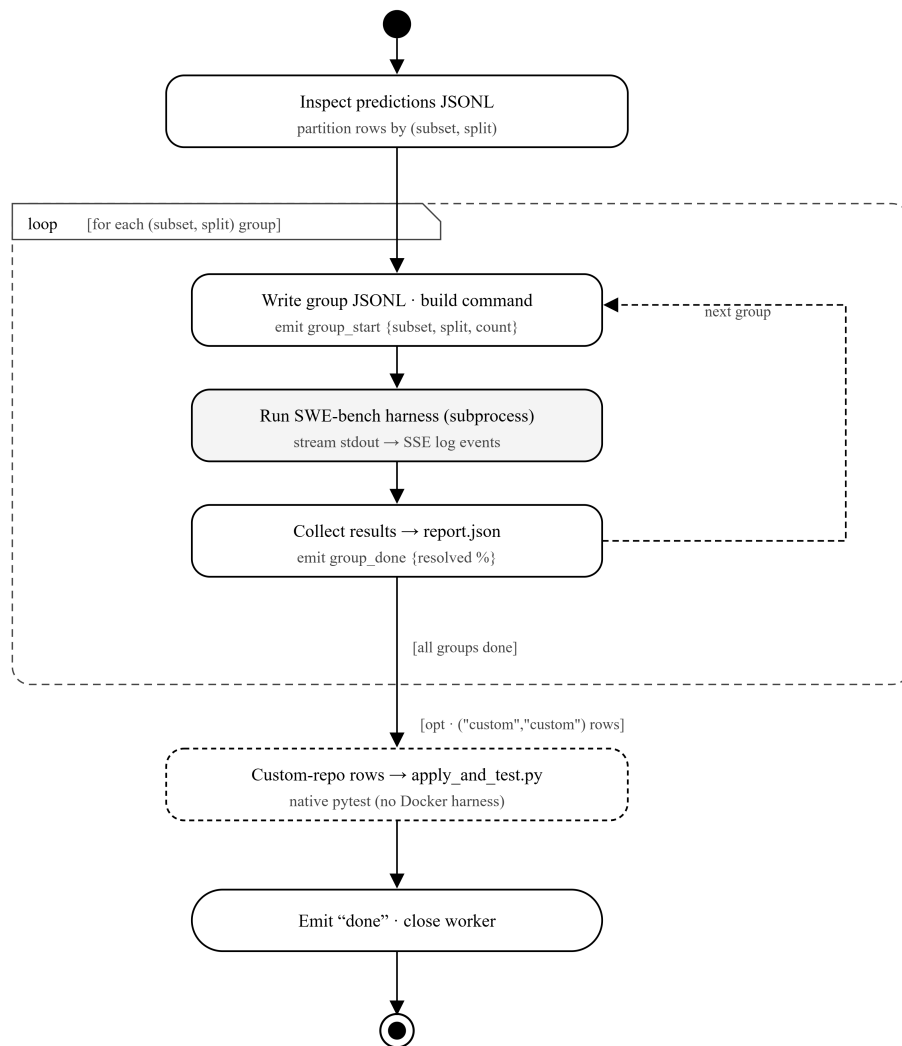


Figure C.15: Activity diagram of the evaluation pipeline driven by the *SWE-bench* Docker harness.

Appendix D

Topology Graphs of the Experiment Notebooks

Table D.1: All 107 instances used in the experiments, grouped by set.

Instance ID	Notes
<i>Synthetic APR set (5 instances)</i>	
evomas-instance-trivial	trivial
evomas-instance-easy	easy
evomas-instance-medium	medium
evomas-instance-hard	hard
evomas-instance-expert	expert
<i>SWE-bench Lite dev subset (23 instances)</i>	
sqlfluff__sqlfluff-1625	sqlfluff
sqlfluff__sqlfluff-2419	sqlfluff
sqlfluff__sqlfluff-1517	sqlfluff
sqlfluff__sqlfluff-1733	sqlfluff
sqlfluff__sqlfluff-1763	sqlfluff
marshmallow-code__marshmallow-1359	marshmallow
marshmallow-code__marshmallow-1343	marshmallow
pvl-lib__pvl-lib-python-1707	pvl-lib
pvl-lib__pvl-lib-python-1072	pvl-lib
pvl-lib__pvl-lib-python-1606	pvl-lib
pvl-lib__pvl-lib-python-1854	pvl-lib
pvl-lib__pvl-lib-python-1154	pvl-lib
pylint-dev__astroid-1978	pylint-astroid
pylint-dev__astroid-1196	pylint-astroid
pylint-dev__astroid-1333	pylint-astroid
pylint-dev__astroid-1866	pylint-astroid
pylint-dev__astroid-1268	pylint-astroid
pyvista__pyvista-4315	pyvista

(continued on next page)

(continued from previous page)

Instance ID	Notes
pydicom__pydicom-1413	pydicom
pydicom__pydicom-1694	pydicom
pydicom__pydicom-1139	pydicom
pydicom__pydicom-1256	pydicom
pydicom__pydicom-901	pydicom
<i>SWE-bench Lite test subset (77 of 300, gold patch < 600 chars)</i>	
astropy__astropy-12907	astropy
astropy__astropy-6938	astropy
django__django-10914	django
django__django-10924	django
django__django-11049	django
django__django-11133	django
django__django-11179	django
django__django-11964	django
django__django-12113	django
django__django-12125	django
django__django-12856	django
django__django-12908	django
django__django-13447	django
django__django-14411	django
django__django-14534	django
django__django-14580	django
django__django-14672	django
django__django-14915	django
django__django-15061	django
django__django-15320	django
django__django-15388	django
django__django-15400	django
django__django-15814	django
django__django-15851	django
django__django-15902	django
django__django-16046	django
django__django-16255	django
django__django-16379	django
django__django-16527	django
django__django-17087	django
matplotlib__matplotlib-23314	matplotlib
matplotlib__matplotlib-23476	matplotlib
matplotlib__matplotlib-23563	matplotlib
matplotlib__matplotlib-25433	matplotlib
mwaskom__seaborn-3010	seaborn
mwaskom__seaborn-3190	seaborn
mwaskom__seaborn-3407	seaborn

(continued on next page)

(continued from previous page)

Instance ID	Notes
psf__requests-1963	requests
psf__requests-863	requests
pydata__xarray-4094	xarray
pydata__xarray-4248	xarray
pydata__xarray-5131	xarray
pylint-dev__pylint-7080	pylint
pylint-dev__pylint-7993	pylint
pytest-dev__pytest-11143	pytest
pytest-dev__pytest-11148	pytest
pytest-dev__pytest-5227	pytest
pytest-dev__pytest-5413	pytest
pytest-dev__pytest-6116	pytest
pytest-dev__pytest-7168	pytest
pytest-dev__pytest-7432	pytest
scikit-learn__scikit-learn-13439	scikit-learn
scikit-learn__scikit-learn-13584	scikit-learn
scikit-learn__scikit-learn-13779	scikit-learn
sphinx-doc__sphinx-7738	sphinx
sphinx-doc__sphinx-8721	sphinx
sympy__sympy-12171	sympy
sympy__sympy-13177	sympy
sympy__sympy-13480	sympy
sympy__sympy-13647	sympy
sympy__sympy-13971	sympy
sympy__sympy-14774	sympy
sympy__sympy-14817	sympy
sympy__sympy-15346	sympy
sympy__sympy-15609	sympy
sympy__sympy-16988	sympy
sympy__sympy-17139	sympy
sympy__sympy-17630	sympy
sympy__sympy-17655	sympy
sympy__sympy-18057	sympy
sympy__sympy-18621	sympy
sympy__sympy-19487	sympy
sympy__sympy-20212	sympy
sympy__sympy-21612	sympy
sympy__sympy-21614	sympy
sympy__sympy-21627	sympy
sympy__sympy-22005	sympy
<i>Non-APR demo instances (2 instances)</i>	
translate-intro-en-es	translate intro.md from English to Spanish

(continued on next page)

(continued from previous page)

Instance ID	Notes
websearch-py-latest	answer "latest stable Python release?"

Legend (agent type, abbreviation tag, and border style): Router [RT] Locator [LO] Patcher [PA] Reviewer [RV] Helper/Proxy [HP] Bug Reproduction [BR] Base Agent [BA].

Note: The "Router" role denotes EvoMas's internal dispatch agent and corresponds to the empirical Planner/Orchestrator category defined in Section 3.2.5.

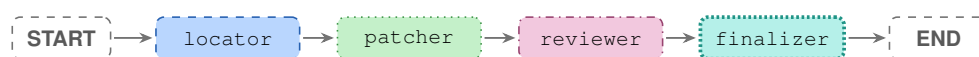


Figure D.1: chain topology graph. Four agents executed in strict sequence; finalizer is the terminal node.

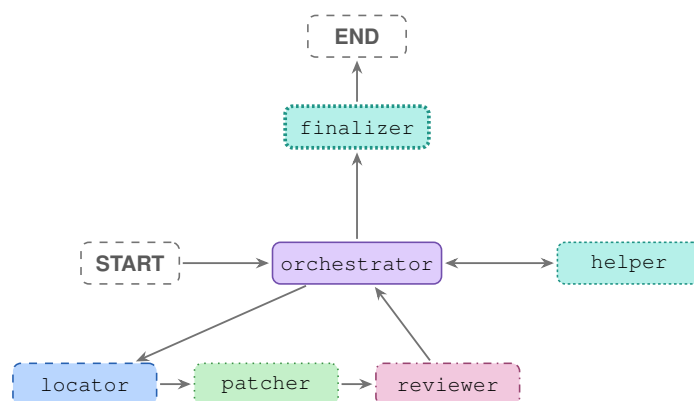


Figure D.2: agentscope_hybrid topology graph. A Router orchestrator delegates to a Locator-Patcher-Reviewer linear subchain and may consult the helper before exiting to finalizer.

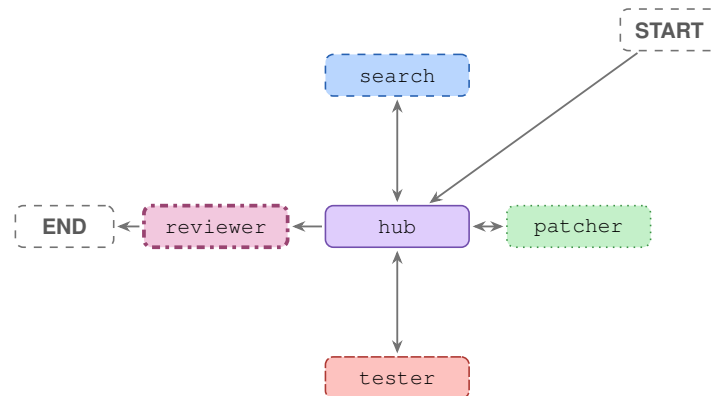


Figure D.3: *experepair_star* topology graph. *search*, *patcher*, and *tester* return to the *hub* between turns; *reviewer* is the terminal node.

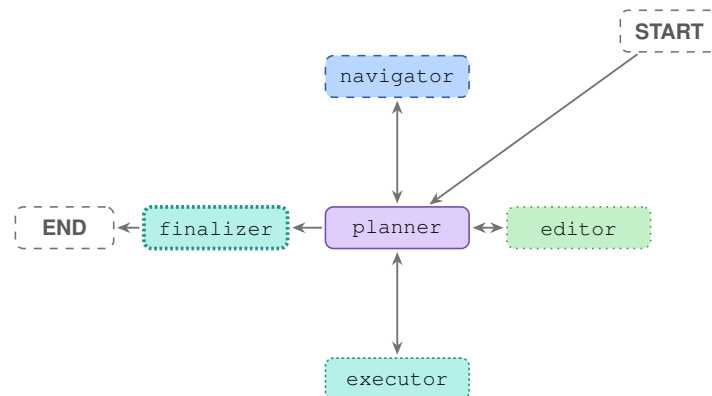


Figure D.4: *hyperagent_star* topology graph. The Router planner re-invokes itself between leaf calls, routing successively to *navigator*, *editor*, or *executor* before exiting to *finalizer*.

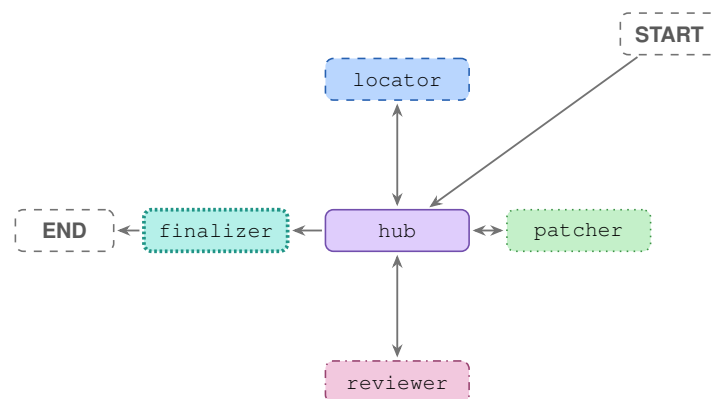


Figure D.5: *joycode_star* topology graph. Hub-and-spoke topology with three bidirectional leaves; *finalizer* terminates.

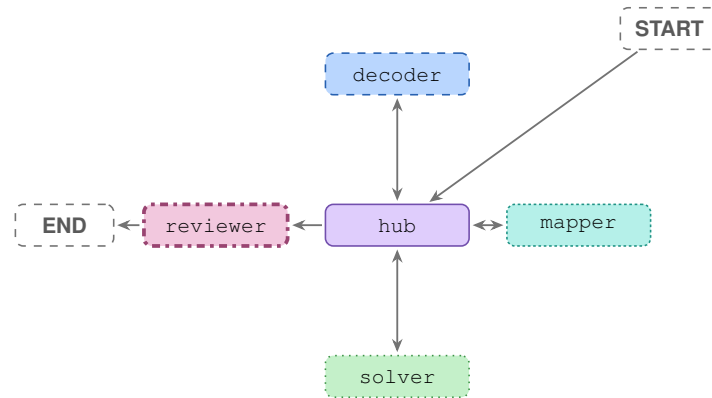


Figure D.6: *lingxi_star* topology graph. decoder, mapper, and solver cycle back through the hub before the run exits to reviewer.

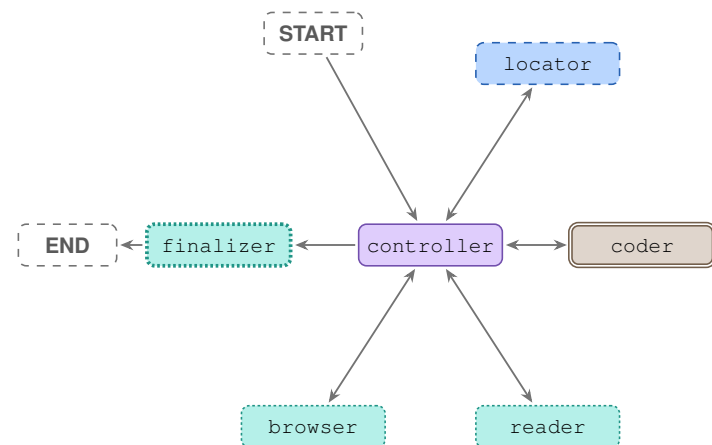


Figure D.7: *openhands_star* topology graph. Four bidirectional leaves plus a terminal finalizer; coder is the only agent that uses the generic Base type (free-form prompt) rather than a typed role.

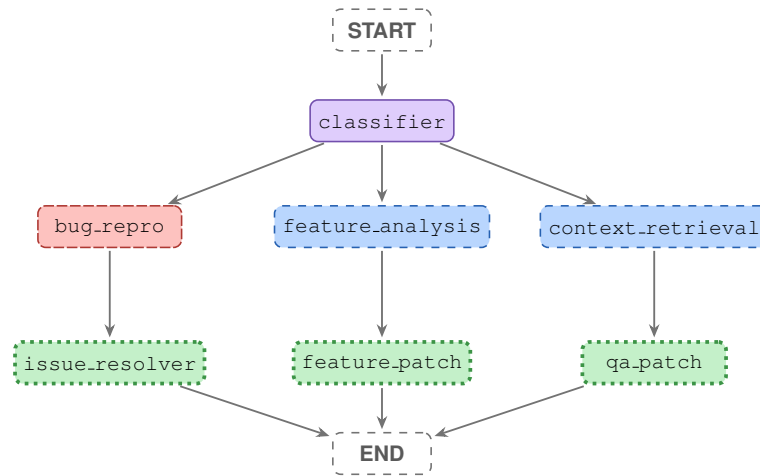


Figure D.8: `prometheus_tree` topology graph. The `classifier` routes the issue into one of three exclusive branches (Bug-reproduction or one of two Locator pre-stages); only the branch's terminal Patcher produces a patch.

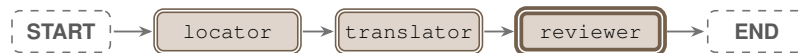


Figure D.9: `translate` topology graph. `locator` picks the target file, `translator` writes the translated body to disk, and `reviewer` flags untranslated chunks or formatting inconsistencies; the unified diff is captured from the workspace mutation.



Figure D.10: `websearch` topology graph. A single agent equipped with two tools (`websearch` via DuckDuckGo and `save_text` for verbatim file writes); the resolution criterion is a non-empty `model_patch` or an `answer*.md` file written as a side effect.

Bibliography

- A. Antoniadou, A. Örwall, K. Zhang, Y. Xie, A. Goyal, and W. Wang. Swe-search: Enhancing software agents with monte carlo tree search and iterative refinement, 2025. URL <https://arxiv.org/abs/2410.20285>.
- E. G. Arias, M. Li, C. Heumann, and M. Aßenmacher. Decoding decoded: Understanding hyperparameter effects in open-ended text generation, 2024. URL <https://arxiv.org/abs/2410.06097>.
- M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- N. Chowdhury, J. Aung, C. J. Shern, O. Jaffe, D. Sherburn, G. Starace, E. Mays, R. Dias, M. Aljube, M. Glaese, C. E. Jimenez, J. Yang, L. Ho, T. Patwardhan, K. Liu, and A. Madry. Introducing SWE-bench verified. <https://openai.com/index/introducing-swe-bench-verified/>, 2024.
- M. E. Conway. How do committees invent? *Datamation*, 14(5):28–31, 1968. URL http://www.melconway.com/Home/Committees_Paper.html.
- Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. URL <https://arxiv.org/abs/2305.14325>.

- Gephi Consortium. Gephi lite: A web-based network visualization tool. <https://lite.gephi.org>, 2024.
- S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. Metagtpt: Meta programming for a multi-agent collaborative framework, 2024. URL <https://arxiv.org/abs/2308.00352>.
- X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang. Large language models for software engineering: A systematic literature review, 2024. URL <https://arxiv.org/abs/2308.10620>.
- C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- Y. Jin, M. Chandra, G. Verma, Y. Hu, M. D. Choudhury, and S. Kumar. Better to ask in english: Cross-lingual evaluation of large language models for healthcare queries, 2023. URL <https://arxiv.org/abs/2310.13132>.
- J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.
- LangChain AI. LangGraph Studio: The first agent IDE. <https://www.langchain.com/blog/langgraph-studio-the-first-agent-ide>, 2024.
- J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye. More agents is all you need, 2024. URL <https://arxiv.org/abs/2402.05120>.
- N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts, 2023. URL <https://arxiv.org/abs/2307.03172>.
- Y. Lu, Y. Hu, X. Zhao, and J. Cao. Dytopo: Dynamic topology routing for multi-agent reasoning via semantic matching, 2026. URL <https://arxiv.org/abs/2602.06039>.

- Microsoft. Pylance: Fast, feature-rich language support for Python in VS Code. <https://marketplace.visualstudio.com/items?itemName=ms-python.vscode-pylance>, 2024.
- Microsoft Research. AutoGen Studio: A no-code developer tool for building and debugging multi-agent systems. <https://microsoft.github.io/autogen/0.2/docs/autogen-studio/getting-started/>, 2024.
- D. J. Moore. A taxonomy of hierarchical multi-agent systems: Design patterns, coordination mechanisms, and industrial applications, 2025. URL <https://arxiv.org/abs/2508.12683>.
- Ollama. Ollama: Run large language models locally. <https://ollama.com>, 2024.
- OpenHands. OpenHands software agent SDK. <https://docs.openhands.dev/sdk>, 2024.
- Payscale. Average junior software engineer salary in Spain. https://www.payscale.com/research/ES/Job=Junior_Software_Engineer/Salary, 2025.
- pytest contributors. pytest: helps you write better programs. <https://docs.pytest.org/>, 2025.
- P. Ralph, N. bin Ali, S. Baltes, D. Bianculli, J. Diaz, Y. Dittrich, N. Ernst, M. Felderer, R. Feldt, A. Filieri, B. B. N. de França, C. A. Furia, G. Gay, N. Gold, D. Graziotin, P. He, R. Hoda, N. Juristo, B. Kitchenham, V. Lenarduzzi, J. Martínez, J. Melegati, D. Mendez, T. Menzies, J. Moller, D. Pfahl, R. Robbes, D. Russo, N. Saarimäki, F. Sarro, D. Taibi, J. Siegmund, D. Spinellis, M. Staron, K. Stol, M.-A. Storey, D. Taibi, D. Tamburri, M. Torchiano, C. Treude, B. Turhan, X. Wang, and S. Vegas. Empirical standards for software engineering research, 2021. URL <https://arxiv.org/abs/2010.03525>.
- F. Sabaté. Módulo 2.1.2: Gestión ágil de proyectos, 2023.
- S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, P. S. Dulepet, S. Vidyadhara, D. Ki, S. Agrawal, C. Pham, G. Kroiz, F. Li, H. Tao, A. Srivastava, H. D. Costa, S. Gupta, M. L. Rogers, I. Goncarenco, G. Sarli, I. Galyunker, D. Peskoff, M. Carpuat, J. White, S. Anadkat, A. Hoyle,

- and P. Resnik. The prompt report: A systematic survey of prompt engineering techniques, 2025. URL <https://arxiv.org/abs/2406.06608>.
- X. Shen, Y. Liu, Y. Dai, Y. Wang, R. Miao, Y. Tan, S. Pan, and X. Wang. Understanding the information propagation effects of communication topologies in llm-based multi-agent systems, 2025. URL <https://arxiv.org/abs/2505.23352>.
- A. Sima. What if AI had corporate culture? <https://alexsima.substack.com/p/ai-multi-agents-with-corporate-structures>, July 2024.
- Skywork. Skywork-SWE-32b: A code repair LLM. <https://huggingface.co/Skywork/Skywork-SWE-32B>, 2025.
- J. Stone, R. Patel, F. Ghiasi, S. Mittal, and S. Rahimi. Navigating mlops: Insights into maturity, lifecycle, tools, and careers. In *2025 IEEE Conference on Artificial Intelligence (CAI)*, page 643–650. IEEE, May 2025. doi: 10.1109/cai64502.2025.00118. URL <https://arxiv.org/abs/2503.15577>.
- SWE-bench Team. SWE-bench: A benchmark for evaluating large language models on real-world software engineering tasks. <https://www.swebench.com/>, 2024a. Accessed: June 2026.
- SWE-bench Team. SWE-bench lite. <https://www.swebench.com/lite.html>, 2024b.
- SWE-bench Team. SWE-bench: Original benchmark. <https://www.swebench.com/original.html>, 2024c.
- K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O’Sullivan, and H. D. Nguyen. Multi-agent collaboration mechanisms: A survey of llms, 2025. URL <https://arxiv.org/abs/2501.06322>.
- L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), Mar. 2024. ISSN 2095-2236. doi: 10.1007/s11704-024-40231-1. URL <https://doi.org/10.1007/s11704-024-40231-1>.

- Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL <https://arxiv.org/abs/2308.08155>.
- C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying llm-based software engineering agents, 2024. URL <https://arxiv.org/abs/2407.01489>.
- J. Yang, M. Zhang, Y. Jin, H. Chen, Q. Wen, L. Lin, Y. He, S. Kumar, W. Xu, J. Evans, and J. Wang. Topological structure learning should be a research priority for llm-based multi-agent systems, 2025. URL <https://arxiv.org/abs/2505.22467>.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- G. Zhang, Y. Yue, X. Sun, G. Wan, M. Yu, J. Fang, K. Wang, T. Chen, and D. Cheng. G-designer: Architecting multi-agent communication topologies via graph neural networks, 2025. URL <https://arxiv.org/abs/2410.11782>.
- H. L. Zhang, C. H. C. Leung, and G. K. Raikundalia. Classification of intelligent agent network topologies and a new topological description language for agent networks. In *Intelligent Information Processing III: IFIP TC12 International Conference on Intelligent Information Processing (IIP 2006)*, volume 228 of *IFIP International Federation for Information Processing*, pages 61–70. Springer, 2006. URL https://link.springer.com/chapter/10.1007/978-0-387-44641-7_3.